

Solving the Shortest Linear Program Problem with Applications to Fast Matrix-Matrix Multiplication

Master's Thesis in Mathematical Modelling and Computation
Department of Applied Mathematics and Computer Science
Technical University of Denmark

MSc Thesis

by

Hugo M. Nielsen

Supervisors

Professor Christian Henriksen
Professor Jakob Lemvig

June 26, 2026

Abstract

This thesis establishes how the classic Shortest Linear Program (SLP) problem can be used to reduce the leading constant in the asymptotic operation count of Strassen-type matrix multiplication algorithms. Improvements to state-of-the-art heuristics and heuristics-based frameworks for solving the SLP problem are proposed and justified both theoretically and experimentally. Of special note here are our novel preprocessing methods, our novel postprocessing method, and our significant improvements to the well-known Boyar-Peralta heuristic for solving instances of the SLP problem. The improved SLP-solver we develop is employed on various databases of distinct types of real-world instances of the SLP problem, from which we derive several state-of-the-art results. Most notably, we find a 52-addition method for a rank-23 general 3×3 matrix multiplication scheme using a Schwartz and Vaknin style basis change [1, 2], improving upon the previous state-of-the-art: a 56-addition scheme with no basis-change [3].

Additionally, we adapt the SAT-based framework of [4] to find multiple new state-of-the-art algorithms for computing the structured matrix product $AA^\intercal$ over $\mathbb{F}_2$, the best of which improves the so-called asymptotic factor from 0.595 found in [5] to 0.5678.

Contents

Glossary v

1 Introduction 1
1.1 Motivation . . . 1
1.2 Contributions . . . 1
1.3 Structure of this Thesis . . . 2

2 Background 3
2.1 Strassen's Algorithm . . . 3
2.2 Beyond Strassen . . . 5
2.3 Addition Minimization . . . 6
2.3.1 Addition Schemes . . . 8
2.3.2 Lifting Addition Schemes . . . 9

3 Data 13
3.1 Dataset for General 3×3 Matrix Multiplication . . . 13
3.1.1 Basis Change . . . 13
3.2 Dataset for Structured Matrix Multiplication . . . 17
3.3 Dataset for Cryptography Matrices . . . 19
3.4 Bernoulli Random Matrices . . . 20
3.4.1 Greedy Potential $\mathbb{F}_2$ Test Dataset . . . 21

4 Shortest Linear Program 22
4.1 Preprocessing . . . 22
4.1.1 Row and Column Reduction . . . 22
4.1.1.1 Row Reduction . . . 22
4.1.1.2 Column Reduction . . . 23
4.1.2 Separability . . . 24
4.2 Heuristics . . . 26
4.2.1 Paar1 & Paar2 . . . 26
4.2.1.1 $\mathbb{F}_2$ Speedups . . . 27
4.2.2 Greedy Potential . . . 28
4.2.2.1 A Potential Difference . . . 29
4.2.2.2 $\mathbb{F}_2$ Speedups . . . 30
4.2.3 Boyar Peralta . . . 31
4.2.3.1 Computing the Distance Vector . . . 32
Brute Force Search. . . . 33
Sparsity-Aware Backtracking. . . . 33
MITM. . . . 34
4.2.4 Random-Normal-BP (RNBP) . . . 36
4.2.5 A1 & A2 . . . 36
4.2.6 MIP . . . 37
4.3 Postprocessing . . . 38
4.3.1 Fixed rules derived from exhaustive search . . . 38
4.3.2 Dynamic reduction rules . . . 39
4.3.2.1 Two-Level Deletion . . . 40
4.4 Iterative SLP Framework . . . 40
4.5 To parallelize or not to parallelize? . . . 42

5 Results 44
5.1 Individual Heuristic Optimizations . . . 44
5.1.1 Greedy Potential . . . 44
5.1.2 Boyar Peralta . . . 46

5.1.3 Comparison of Single-Shot Heuristics . . . 46
5.1.4 Framework . . . 49
5.2 Rank-23 General 3×3 Matrix Multiplication . . . 50
5.3 Fast Structured Matrix Multiplication . . . 51
5.4 Linear Cryptographic Layers . . . 52

6 Conclusion and Outlook **54**
6.1 Future Work . . . 54

Appendix **60**
A Recursive Scheme for $AA^\intercal$ . . . 60
B Strassen's 2×2 matrix multiplication implementation . . . 63
C Rules Covered By New Postprocessing . . . 65
D Rank-23 General 3×3 Matrix Multiplication Schemes . . . 73
D.1 60-addition scheme, no basis change . . . 73
D.2 52-addition scheme, with basis change . . . 74
E Shortest Linear Program Code . . . 80

Glossary

Acronyms

Acronym	Meaning
SLP	Shortest Linear Program
NNZ	Number of nonzero entries
BP	Boyar-Peralta heuristic
RNBP	Random Normal Boyar-Peralta heuristic
MIP	Mixed Integer Programming
DAG	Directed Acyclic Graph
MITM	Meet-in-the-middle
XOR	Exclusive-or
SOTA	State of the art
SAT	Boolean satisfiability problem
CNF	Conjunctive Normal Form
ISD	Information Set Decoding

Notation

Symbol	Meaning
$\mathbb{N}$	Set of positive integers
$\mathbb{N}_0$	Set of non-negative integers
$\mathbb{F}$	A general field
$\mathbb{F}_2$	The field with two elements
R	A ring
$[n]$	The set $\{1, 2, \ldots, n\}$
$\vec{X}$ or $\mathrm{vec}(X)$	Column-wise vectorization of a matrix X
E	Standard basis of the relevant vector space of implicit dimension $n \in \mathbb{N}$
T	Set of target vectors for an instance of the SLP problem
$\&$	The bitwise AND operator
$\oplus$	The XOR symbol
$\odot$	Hadamard product
ω	Matrix multiplication exponent
ρ	Density of matrix or bias of Bernoulli random binary matrix
$P(G)$	The potential of a matrix
D	Distance vector used in Boyar-Peralta type heuristics
$\|\|D\|\|_1$	ℓ_1-norm of the distance vector
$\|\|D\|\|_2$	ℓ_2-norm of the distance vector
$H(\cdot)$	Hamming weight, i.e. number of non-zero entries, of a vector in $\mathbb{F}_2^n$
$\textsc{BlockVec}_k$	Partitions a matrix into a $k \times k$ grid of equal-sized blocks and lists the k^2 blocks as a vector in column-major order. Is the inverse of $\textsc{BlockMat}_k$
$\textsc{BlockMat}_k$	Takes a vector of k^2 equal-sized block matrices and places them back into the corresponding $k \times k$ layout. Is the inverse of $\textsc{BlockVec}_k$

This page was intentionally left blank.

Chapter 1

Introduction

1.1 Motivation

Matrix-matrix multiplication is one of the most fundamental mathematical operations, with widespread applications including deep neural network inference and training, computer graphics, cryptographic encodings, mathematical optimization, and many others [6–8]. Consequently, even modest improvements to matrix multiplication algorithms can have a significant impact across a wide range of applications.

While theoretical approaches have been very successful in reducing the exponent in the arithmetic complexity of matrix multiplication, see Figure 1, practical implementations still predominantly use hardware-optimized versions of the classical $O(n^3)$ matrix multiplication algorithm, see Algorithm 1. This is largely because many asymptotically faster algorithms have large leading constants and implementation overheads. The gap between the asymptotic and practical efficiency suggests that progress on practical algorithms may come not only from lowering ω, but also from reducing the leading constant in the asymptotic operation count for many known sub-cubic matrix multiplication algorithms.

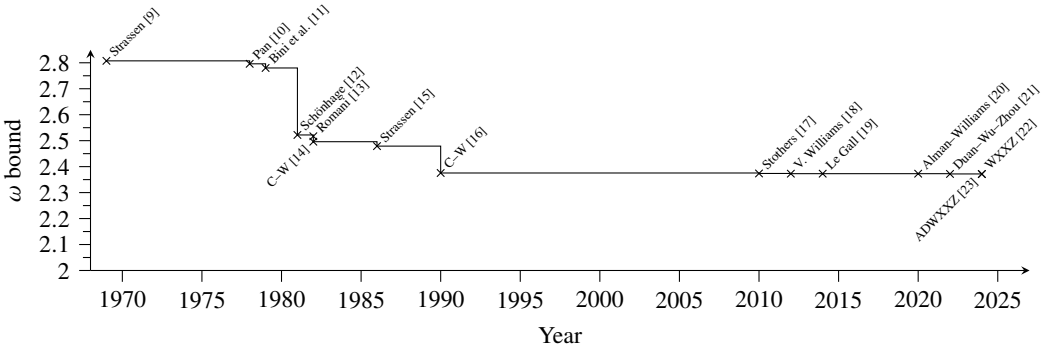


Fig. 1: Progress on the best upper bound on the matrix multiplication exponent ω, where ω is such that two $n \times n$ matrices can be multiplied in $O(n^\omega)$ arithmetic operations. Data points are taken from the cited works, the visualization is inspired by [24].

There is substantial interest in closing the gap between the theoretical advances in the asymptotic operation count and the efficiency of practical implementations of matrix multiplication [2, 3, 5, 25]. This thesis contributes to that effort by improving implementations of the linear combination stage in a class of sub-cubic matrix multiplication algorithms known as Strassen-type schemes.

A second motivation for the work in this thesis comes from the closely related *Shortest Linear Program* (SLP) problem, which asks for a shortest sequence of linear operations, typically over $\mathbb{F}_2$, that computes a given linear map. This problem has been studied extensively in the literature on cryptography [8, 26], because it relates directly to reducing the number of XOR gates required to compute linear layers in small cryptographic circuits, and thereby reducing the area or cost of the circuit, see Figure 10. We show how improving algorithms and heuristics for solving the SLP problem can translate directly to lowering the leading constant for the asymptotic operation count of certain matrix multiplication algorithms with sub-cubic asymptotic operation counts.

1.2 Contributions

In this work, we improve upon several well-known heuristics for optimizing instances of the shortest linear program problem. We do this by optimizing the time complexity of certain aspects of these algorithms, see Section 4. Of special interest are our improvements to the well-known Boyar-Peralta heuristic. Moreover, we develop novel preprocessing and postprocessing steps, and combine these with the previously mentioned heuristics to obtain a state-of-the-art framework for solving SLP instances.

We show how the new framework we have developed can be used to find new best-known solutions to several SLP datasets, with special focus on the addition-minimization problem of the rank-23 general 3×3 matrix multiplication. On the problem of reducing the number of additions required to compute a rank-23 general 3×3 matrix multiplication we find a new best-known addition scheme requiring 52 additions, see Appendix D, improving upon the previous best-known of 56 additions [3].

Finally, in Appendix A we find multiple new matrix multiplication schemes over $\mathbb{F}_2$ for computing $AA^\intercal$, which significantly improve upon the previous state-of-the-art methods, the best of the new methods we find reduces the asymptotic factor from 0.595 [5] to 0.5678.

1.3 Structure of this Thesis

Chapter 2. We begin by introducing the preliminaries necessary to understand the modern literature on Strassen-type matrix multiplication methods, the shortest linear program problem (SLP), and the relation between Strassen-type matrix multiplication schemes and SLP. We also introduce a backtracking method for extending solutions to SLP instances over $\mathbb{F}_2$ to work over general fields $\mathbb{F}$.

Chapter 3. We introduce multiple novel datasets to benchmark various aspects of shortest linear program solvers. We show how to artificially inflate one of the datasets beyond its original size without losing the property of the dataset that reductions in the number of additions used to compute the scheme lead directly to smaller leading constants in the asymptotic arithmetic operation count of the method. A state-of-the-art structured matrix multiplication scheme is introduced as an example in one dataset, the details of its discovery are shown in Appendix A.

Chapter 4. The most well-known heuristics for solving the shortest linear program problem in $\mathbb{F}_2$ are introduced, novel preprocessing methods are proposed and properties about them proven, similarly a novel postprocessing method is proposed, and certain properties of it are proven. We improve upon many of these heuristics in a theoretical manner, and delay the experimental verification of these improvements to the following chapter. We provide an exact mixed-integer programming formulation of the problem, and we show how all of the work in this chapter can be combined into a single coherent framework for solving the shortest linear program problem.

Chapter 5. The improvements of the heuristics proposed in the preceding chapter are experimentally verified. We benchmark various methods for optimizing instances of the shortest linear program problem, and show how our improvements lead to new state-of-the-art results in structured matrix multiplication, linear cipher matrices, and rank-23 general 3×3 matrix multiplication.

Chapter 6. All results from the thesis are put into perspective, and exciting directions to take future research in the topics of Strassen-type matrix multiplication algorithms, and the SLP problem are discussed.

Chapter 2

Background

Throughout this thesis, we restrict our attention to square matrices, and let $n \in \mathbb{N}$ denote their size. Moreover, let $\vec{X}$ denote the column-stacked vectorization of the $n \times n$ matrix X. Finally, we denote by $\mathbb{F}$ a general field and R a ring.

The product of two matrices $A, B \in R^{n\times n}$ is defined as the $n \times n$ matrix C, such that:

$$C_{ij} = (AB)_{ij} = \sum_{k=1}^{n} A_{ik}B_{kj}.$$

This definition naturally lends itself to implementations of matrix multiplication such as Algorithm 1.

Algorithm 1 The standard matrix multiplication algorithm for computing the product AB for matrices $A, B \in R^{n\times n}$ in $O(n^3)$ arithmetic operations.

```
1:  function MATMUL(n ∈ ℕ, A ∈ R^{n×n}, B ∈ R^{n×n})
2:      C ← 0                                   ▷ The n × n zero matrix over R
3:      for i ← 1 to n do
4:          for j ← 1 to n do
5:              C_ij ← A_i1 B_1j
6:              for k ← 2 to n do
7:                  C_ij ← C_ij + A_ik B_kj
8:              end for
9:          end for
10:     end for
11:     return C
12: end function
```

Such algorithms require n^3 multiplications, one for each of the n^2 times line 5 is run, and one for each of the $n^2(n-1)$ times line 7 is run. It also performs $n^2(n-1)$ additions, one for each of the times line 7 is run. For many years matrix multiplication was thus assumed to require $\Theta(n^3)$ arithmetic operations [27], but in 1969 this was shown to be incorrect, when Volker Strassen published an algorithm for general matrix multiplication, which required only $O(n^{\log_2 7})$ arithmetic operations [9].

2.1 Strassen's Algorithm

The standard matrix multiplication algorithm, Algorithm 1, requires 8 multiplications and 4 additions to compute the matrix product of the two matrices $A, B \in R^{2\times 2}$ with elements in the ring R. Let the entries of A and B be given by:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}.$$

The standard algorithm would compute these 8 products:

$$\begin{aligned} M_1 &= A_{11}B_{11}, & M_2 &= A_{12}B_{21}, \\ M_3 &= A_{11}B_{12}, & M_4 &= A_{12}B_{22}, \\ M_5 &= A_{21}B_{11}, & M_6 &= A_{22}B_{21}, \\ M_7 &= A_{21}B_{12}, & M_8 &= A_{22}B_{22}, \end{aligned}$$

and then use 4 additions to construct the desired matrix product:

$$AB = \begin{bmatrix} M_1 + M_2 & M_3 + M_4 \\ M_5 + M_6 & M_7 + M_8 \end{bmatrix}. \tag{2.1}$$

In the seminal work [9] by Volker Strassen, he showed that by making use of the distributive property of multiplication in a ring R, together with some carefully chosen cancellations, we can compute the matrix product AB using only 7 multiplications and 18 additions/subtractions. The scheme proposed by Strassen first computes the following 7 products using 10 additions/subtractions:

$$\begin{aligned} M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), & M_2 &= (A_{21} + A_{22})B_{11}, \\ M_3 &= A_{11}(B_{12} - B_{22}), & M_4 &= A_{22}(B_{21} - B_{11}), \\ M_5 &= (A_{11} + A_{12})B_{22}, & M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}), \end{aligned}$$

so that, by using 8 additional additions/subtractions can compute the desired matrix product as follows:

$$AB = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}.$$

In matrix form, we can represent this 7-multiplication 18-addition/subtraction method as seen in equation (2.2), where we use the column-stacked vectorizations $\vec{A} = \begin{bmatrix} A_{11} & A_{21} & A_{12} & A_{22} \end{bmatrix}^\intercal$ and $\vec{B} = \begin{bmatrix} B_{11} & B_{21} & B_{12} & B_{22} \end{bmatrix}^\intercal$ of our matrices $A, B \in R^{2\times 2}$:

$$\vec{AB} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}^\intercal \left(\left(\begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix} \vec{A} \right) \odot \left(\begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \vec{B} \right) \right), \tag{2.2}$$

where $\odot$ is the Hadamard product. A straightforward expansion of the right-hand side of equation (2.2) shows that equation (2.2) indeed coincides with equation (2.1) and hence is correct. We state this as Lemma 2.1.

Lemma 2.1. *Let $A, B \in R^{2\times 2}$. We can compute the matrix product AB using 7 multiplications and 18 additions/subtractions.*

It is possible to further reduce the number of additions/subtractions required [2], however at least 7 multiplications are required [28]. While this result might only seem immediately interesting if the computational cost of multiplication in the ring R is much higher than the computational cost of performing additions/subtractions in R, it turns out that we can utilize Lemma 2.1 to perform matrix multiplication using an asymptotically sub-cubic number of operations over any field $\mathbb{F}$.

Theorem 2.2. *We can multiply matrices $A, B \in \mathbb{F}^{n\times n}$ using $7n^{\log_2 7} - 6n^2$ arithmetic operations when n is a power of 2, and $O\left(n^{\log_2 7}\right)$ arithmetic operations in general.*

Proof Since $\mathbb{F}^{m\times m}$ is a ring for all $m \in \mathbb{N}$, Lemma 2.1 applies to 2×2 block matrices with elements in $\mathbb{F}^{m\times m}$.

First assume $n = 2^\ell$ for some $\ell \in \mathbb{N}$. We claim that computing AB for $A, B \in \mathbb{F}^{n\times n}$ by repeated use of Lemma 2.1 requires 7^ℓ multiplications, and $6(7^\ell - 4^\ell)$ additions/subtractions. We prove this claim by way of induction.

The claim is immediate for $\ell = 1$, by Lemma 2.1. Suppose the claim holds for some $\ell = k \in \mathbb{N}$, and let $A, B \in \mathbb{F}^{2^{k+1}\times 2^{k+1}}$ be a pair of arbitrary matrices over $\mathbb{F}$. Consider A and B as 2×2 block matrices with elements in the matrix-ring $\mathbb{F}^{2^k\times 2^k}$. Applying Lemma 2.1 at the block level uses 7 block multiplications, and 18 block additions/subtractions. Each block multiplication requires 7^k multiplications and $6(7^k - 4^k)$ additions/subtractions, as per the induction hypothesis, hence the block multiplications contribute a total of 7^{k+1} multiplications and $7 \cdot 6(7^k - 4^k)$ additions/subtractions. Each block addition/subtraction contributes $(2^k)^2 = 4^k$ additions/subtractions. The total number of multiplications required to multiply two matrices $A, B \in \mathbb{F}^{2^{k+1}\times 2^{k+1}}$ is thus 7^{k+1}, and the total number of additions is $7 \cdot 6(7^k - 4^k) + 18 \cdot 4^k = 6(7^{k+1} - 4^{k+1})$, which by the principle of mathematical induction proves the claim. Hence for $n = 2^\ell$ with $\ell \in \mathbb{N}$, we can multiply matrices in $\mathbb{F}^{n\times n}$ using $7 \cdot 7^\ell - 6 \cdot 4^\ell = 7n^{\log_2 7} - 6n^2$ arithmetic operations, as was to be shown.

Finally, for general n, pad A, B with zeros to the size $2^\ell \times 2^\ell$ where $\ell = \lceil \log_2 n \rceil$, and denote the padded matrices $\tilde{A}, \tilde{B}$. Using $2^\ell < 2n$, we can find the matrix product $\tilde{A}\tilde{B}$ in $O\left(n^{\log_2 7}\right)$ arithmetic operations, as per the preceding argument, and after removing the padding part of $\tilde{A}\tilde{B}$ we are left with AB. □

Strassen's algorithm is exactly what we get when we recursively apply equation (2.2) similar to what is done in the proof of Theorem 2.2, see Algorithm 2.

2.2 Beyond Strassen

From this point onward additions will refer to operations of the type $f : R^2 \to R$, given by $f(x, y) = \lambda x + \mu y$, for any $\lambda, \mu \in \{-1, 1\}$, with sign changes regarded as free. This agrees with the convention for counting additions in Strassen-type matrix multiplication schemes [25, 29]. If R is an $\mathbb{F}$-algebra, and one allows arbitrary nonzero coefficients λ, μ from $\mathbb{F}$, counting operations and comparing matrix multiplication schemes becomes more involved [4, 30], and we leave such extensions and further consideration to future work.

We can generalize the setup in equation (2.2) to matrices $A, B \in R^{k\times k}$ with $k \in \mathbb{N}$. Specifically, if we can find matrices $W, U, V \in \{-1, 0, 1\}^{r\times k^2}$, such that

$$\vec{AB} = W^\intercal \left(\left(U\vec{A}\right) \odot \left(V\vec{B}\right)\right), \tag{2.3}$$

then we can multiply A and B using only r multiplications in R and $\alpha \leq 3k^2 r$ additions in R. We refer to such a scheme as a rank-r multiplication scheme. We restrict to coefficient matrices $W, U, V \in \{-1, 0, 1\}^{r\times k^2}$ because this convention aligns with our definition of addition, simplifies comparison between schemes, allows equation (2.3) to be used over arbitrary rings R, and makes implementation simpler [4, 31]. More general coefficient choices are possible [30], but we leave such extensions to future work.

Theorem 2.3. *Let $k, n \in \mathbb{N}$ with $r > k^2$, given a scheme of the form equation* (2.3) *requiring α additions to compute for $W, U, V \in \{-1, 0, 1\}^{r\times k^2}$, we can multiply matrices $A, B \in \mathbb{F}^{n\times n}$ using $n^{\log_k r}$ multiplications, and $\frac{\alpha}{r-k^2}\left(n^{\log_k r} - n^2\right)$ additions when n is a power of k.*

Proof Since $\mathbb{F}^{m\times m}$ is a ring for all $m \in \mathbb{N}$, our scheme applies to $k \times k$ block matrices with elements in $\mathbb{F}^{m\times m}$.

Let $n = k^\ell$, we proceed by induction on ℓ. For $\ell = 1$ we are given an α-addition r-multiplication scheme of the form in equation (2.3), hence covering the base case. Suppose now that we require $n^{\log_k r} = r^\ell$ multiplications, and $\frac{\alpha}{r-k^2}\left(n^{\log_k r} - n^2\right) = \frac{\alpha}{r-k^2}\left(r^\ell - k^{2\ell}\right)$ additions to compute the matrix product for some $\ell \in \mathbb{N}$. Given any two matrices $A, B \in \mathbb{F}^{k^{\ell+1}\times k^{\ell+1}}$, we can view them as $k \times k$ block matrices with entries in $\mathbb{F}^{k^\ell \times k^\ell}$. Applying equation (2.3) at the block level uses r block multiplications, and α block additions. Each block multiplication uses r^ℓ multiplications and $\frac{\alpha}{r-k^2}\left(r^\ell - k^{2\ell}\right)$ additions, by the induction hypothesis, hence the block multiplications contribute a total of $r \cdot r^\ell = r^{\ell+1}$ multiplications, and $\frac{r\alpha}{r-k^2}\left(r^\ell - k^{2\ell}\right)$ additions. Each block addition contributes $(k^\ell)^2 = k^{2\ell}$ additions. The total number of multiplications required to compute the matrix product AB for matrices $A, B \in \mathbb{F}^{k^{\ell+1}\times k^{\ell+1}}$ is thus:

$$r^{\ell+1} = (kn)^{\log_k r}, \tag{2.4}$$

and the total number of additions required:

$$\frac{r\alpha}{r-k^2}\left(r^\ell - k^{2\ell}\right) + \alpha k^{2\ell} = \frac{r\alpha}{r-k^2}\left(r^\ell - k^{2\ell} + \frac{r-k^2}{r}k^{2\ell}\right) \tag{2.5}$$

$$= \frac{\alpha}{r-k^2}\left(r^{\ell+1} - k^{2(\ell+1)}\right) = \frac{\alpha}{r-k^2}\left((kn)^{\log_k r} - (kn)^2\right), \tag{2.6}$$

which by the principle of mathematical induction completes the proof. □

Theorem 2.2 is a special case of Theorem 2.3, and an important corollary of Theorem 2.3 is then Corollary 2.3.1.

Corollary 2.3.1. *Let $k, r \in \mathbb{N}$ with $r > k^2$. Suppose that matrices $U, V, W \in \{-1, 0, 1\}^{r\times k^2}$ satisfy equation* (2.3) *for all $A, B \in R^{k\times k}$. Then matrices $A, B \in \mathbb{F}^{n\times n}$ can be multiplied using $O(n^{\log_k r})$ arithmetic operations.*

Proof Let $A, B \in \mathbb{F}^{n\times n}$ be the matrices whose product we want to compute. If n is a power of k, then Theorem 2.3 tells us that the total number of arithmetic operations required, is $O\left(n^{\log_k r}\right)$.

Suppose now n is not a power of k, then pad A, B with zeros to the size $m = k^{\lceil \log_k n \rceil}$. Denote the padded matrices $\tilde{A}$ and $\tilde{B}$ respectively, so that:

$$\tilde{A} = \begin{bmatrix} A & 0_{n\times(m-n)} \\ 0_{(m-n)\times n} & 0_{(m-n)\times(m-n)} \end{bmatrix}, \qquad \tilde{B} = \begin{bmatrix} B & 0_{n\times(m-n)} \\ 0_{(m-n)\times n} & 0_{(m-n)\times(m-n)} \end{bmatrix},$$

where $0_{a\times b}$ represents the $a \times b$ zero matrix. Since $m < kn$ and m is a power of k, we can compute $\tilde{A}\tilde{B}$ in $O\left(m^{\log_k r}\right) = O\left((nk)^{\log_k r}\right) = O\left(n^{\log_k r}\right)$ arithmetic operations. Finally, we can extract AB from $\tilde{A}\tilde{B}$ as the top-left $n \times n$ submatrix of $\tilde{A}\tilde{B}$. □

The recursive Strassen-like algorithm used to compute AB, given a scheme of the form equation (2.3), is shown in Algorithm 2, see Appendix B for a working Python implementation.

Algorithm 2 Recursive algorithm using $O(n^{\log_k r})$ arithmetic operations for computing the matrix product $AB \in \mathbb{F}^{n\times n}$, given the matrices $A, B \in \mathbb{F}^{n\times n}$ and a scheme of the form equation (2.3), assuming that n is a power of k.

1: **function** MATMUL($n \in \mathbb{N}$, $k \in \mathbb{N}$, $r \in \mathbb{N}$, $A \in \mathbb{F}^{n\times n}$, $B \in \mathbb{F}^{n\times n}$, $U \in \{-1,0,1\}^{r\times k^2}$, $V \in \{-1,0,1\}^{r\times k^2}$, $W \in \{-1,0,1\}^{r\times k^2}$)
2: **if** $n = 1$ **then**
3: return $A \cdot B$ ▷ Regular $\mathbb{F}$ multiplication
4: **end if**
5: $\mathcal{A} \leftarrow \text{BLOCKVEC}_k(A)$ ▷ $\mathcal{A}$ and $\mathcal{B}$ are treated as k^2 element vectors with elements in $\mathbb{F}^{\frac{n}{k}\times\frac{n}{k}}$
6: $\mathcal{A} \leftarrow U\mathcal{A}$
7: $\mathcal{B} \leftarrow \text{BLOCKVEC}_k(B)$
8: $\mathcal{B} \leftarrow V\mathcal{B}$
9: $M \leftarrow 0_r$ ▷ Zero vector in $\left(\mathbb{F}^{\frac{n}{k}\times\frac{n}{k}}\right)^r$
10: **for** $i \leftarrow 1$ **to** r **do**
11: $M_i \leftarrow \text{matmul}\left(\frac{n}{k}, k, r, \mathcal{A}_i, \mathcal{B}_i, U, V, W\right)$ ▷ X_i denotes the i-th element in vector X
12: **end for**
13: $C \leftarrow W^\intercal M$
14: $C \leftarrow \text{BLOCKMAT}_k(C)$ ▷ Reshapes C into an element in $\mathbb{F}^{n\times n}$
15: **return** C
16: **end function**

Given that Strassen's algorithm, the $k = 2$ case, see equation (2.2), has optimal rank, a natural extension to consider would be the case when $k = 3$. Theorem 2.3 tells us that we would need a scheme with rank 21 or less, for $k = 3$ to surpass the asymptotic performance of Strassen's algorithm, since for $r \in \mathbb{N}$ we have $\log_3 r > \log_2 7$ if and only if $r > 21$. Despite a substantial amount of research and compute going into reducing the rank of the $k = 3$ case, no improvement in the rank has occurred since Laderman found a rank-23 scheme in 1979 [4, 31].

2.3 Addition Minimization

Recently, there has been a substantial interest in minimizing the number of additions in 3×3 matrix multiplication schemes of rank-23 [2, 3, 25, 29, 31–33]. Recent work has followed two main approaches to minimizing the number of additions of rank-23 multiplication schemes on 3×3 matrices. One approach used by [3, 29, 33] is to look for new rank-23 multiplication schemes requiring few additions, another is to minimize additions in known rank-23 schemes as in the works [2, 25]. In this work, we focus on the second method.

For a fixed rank-r multiplication scheme, addition minimization decomposes into three independent optimization problems: minimizing the number of additions required to construct the vector $U\vec{A}$, minimizing the number of additions required to construct the vector $V\vec{B}$, and minimizing the number of additions required to construct the vector $W^\intercal((U\vec{A}) \odot (V\vec{B}))$, where $U, V, W \in \{-1,0,1\}^{r\times k^2}$ are given, and are used as in equation (2.3), i.e. computing lines 6, 8 and 13 in Algorithm 2. Minimizing the number of additions required to compute the linear map $x \mapsto Gx$ for $G \in \mathbb{F}^{m\times n}$ is essentially a problem of reusing additions that occur at multiple places. For illustrative purposes, consider the simple example below.

Example 2.1. Consider the matrix-vector product below:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} x_1 + x_2 + x_3 \\ x_2 + x_3 - x_4 \end{bmatrix} \tag{2.7}$$

This would require 4 additions using the simple expansion shown in the right-hand side of equation (2.7), but if we compute it like this:

$$s_1 = x_2 + x_3 \tag{2.8}$$
$$y_1 = x_1 + s_1 \tag{2.9}$$
$$y_2 = s_1 - x_4 \tag{2.10}$$

we only need 3 additions. We can represent this with a $\{-1, +1\}$ edge weighted directed acyclic graph, where each node is either one of the variables x_1, x_2, x_3, x_4, an intermediate sum s_1, or an output y_1, y_2, see Figure 2.

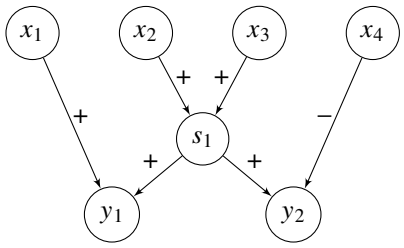


Fig. 2: Illustration of how we can save additions when computing the matrix-vector product Gx shown in equation (2.7). Any node except for x_1, x_2, x_3, x_4 represent the sum induced by the incoming edges. Edges marked with a "+" indicate addition of the source node, and those marked with a "−" indicate addition of the negated source node.

Formally, our matrix-multiplication addition-minimization problem decomposes into three independent addition-minimization problems, analogous to the form in Problem 2.1.

Problem 2.1 (Shortest Linear Program)**.** Given a matrix $G \in \mathbb{F}^{m \times n}$, find the minimum number of additions required to compute the general matrix-vector product Gx for all $x \in \mathbb{F}^n$.

Specifically, denote by $T = \{g_1, g_2, \ldots, g_m\} \subseteq \mathbb{F}^n$ the ordered set of rows in G, and let $E = \{e_1, e_2, \ldots, e_n\}$ be the standard basis of $\mathbb{F}^n$. Then the shortest linear program problem can be expressed as finding the smallest $\ell \in \mathbb{N}_0$ such that an ordered set of vectors $B = (b_1, \ldots, b_{n+\ell})$ satisfies:

1. $b_i = e_i$ for all $i \in [n]$.
2. For all $q > n$ we have some $q > j > i$ such that $b_q = \alpha b_j + \beta b_i$, for some $\alpha, \beta \in \mathbb{F}$.
3. $T \subseteq B$

We will refer to the ordered set B as an optimal *addition scheme* for G if ℓ is minimal and all three conditions are satisfied. Given an optimal addition scheme for G, the number of additions required to compute Gx is $|B| - n$.

The notion of addition in Problem 2.1 is more general than the signed-addition model used for matrix multiplication schemes above. The two notions coincide when the coefficients satisfy $\alpha, \beta \in \{-1, 1\}$. Thus, Problem 2.1 should initially be viewed as a formulation of a type of addition-minimization problem distinct from our setup, later we specialize to settings where this distinction disappears, namely we set $\mathbb{F} = \mathbb{F}_2$.

Any instance of Problem 2.1 is determined completely by the field $\mathbb{F}$ and the matrix $G \in \mathbb{F}^{m \times n}$, as such, when it is clear what the field $\mathbb{F}$ we are working over is, we will refer interchangeably to the matrix G and the instance of Problem 2.1 induced by this G.

Given matrices $U, V, W \in \{-1, 0, 1\}^{r \times k^2}$ such that a matrix multiplication scheme of the form in equation (2.3) holds for all matrices $A, B \in R^{k \times k}$, Algorithm 2 will for such a scheme compute the correct matrix product AB for all matrices $A, B \in \mathbb{F}^{k^\ell \times k^\ell}$, with $\ell \in \mathbb{N}$. This follows immediately from the fact that equation (2.3) makes no assumptions on the commutativity of the entries of $\textsc{BlockVec}_k(A)$ and $\textsc{BlockVec}_k(B)$. Thus minimizing the additions required to compute the matrix product AB using a specific scheme of the form in equation (2.3) reduces to three distinct instances of Problem 2.1, where we additionally restrict $\alpha, \beta \in \{-1, 1\}$.

The matrix G in the formulation of Problem 2.1 corresponds to one of U, V or $W^\intercal$ as they are defined in equation (2.3). To enforce the restriction $\alpha, \beta \in \{-1, 1\}$ in the addition schemes we find for each instance of Problem 2.1 we solve, we will follow the convention in this line of work [4, 7], by working over the field $\mathbb{F} = \mathbb{F}_2$

to reduce the search space. After finding addition schemes in $\mathbb{F}_2$ we attempt to extend the solutions found to have elements in $\{-1, 0, 1\}$ by flipping certain 1-entries to (-1)-entries.

2.3.1 Addition Schemes

A method for computing Gx with $G \in \mathbb{F}^{m\times n}$ and $x \in \mathbb{F}^n$ will throughout the rest of this work be referred to as an *addition scheme*. Addition schemes can be represented in multiple equivalent ways, such as an ordered set B satisfying conditions (1)–(3) in Problem 2.1, a set of explicit additions as shown in equations (2.8)–(2.10), or as a weighted directed acyclic graph as shown in Figure 2. Since we will be working over the field $\mathbb{F}_2$, all the representations simplify. Some instances of Problem 2.1 will be reused throughout this thesis, one of those instances is given by the matrix $G \in \mathbb{F}_2^{4\times 5}$, defined by:

$$G = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \end{bmatrix}. \tag{2.11}$$

An ordered set $B \subseteq \mathbb{F}_2^n$ constitutes an addition scheme if it satisfies the following three conditions:

1. $b_i = e_i$ for all $i \in [n]$.
2. For all $k > n$ there exist indices i, j with $k > j > i$ such that $b_k = b_j + b_i$.
3. $T \subseteq B$, for T the set of target vectors, that is, the rows of G.

The difference between the second condition here and the second condition outlined in Problem 2.1 is that, over $\mathbb{F}_2$, we have $\alpha = \beta = 1$. For the matrix in equation (2.11), we can use the addition scheme:

$$B = \left\{ b_1 = \begin{bmatrix}1\\0\\0\\0\\0\end{bmatrix}, b_2 = \begin{bmatrix}0\\1\\0\\0\\0\end{bmatrix}, b_3 = \begin{bmatrix}0\\0\\1\\0\\0\end{bmatrix}, b_4 = \begin{bmatrix}0\\0\\0\\1\\0\end{bmatrix}, b_5 = \begin{bmatrix}0\\0\\0\\0\\1\end{bmatrix}, b_6 = b_2 + b_3 = \begin{bmatrix}0\\1\\1\\0\\0\end{bmatrix}, \right. \tag{2.12}$$

$$\left. b_7 = b_1 + b_5 = \begin{bmatrix}1\\0\\0\\0\\1\end{bmatrix}, b_8 = b_3 + b_7 = \begin{bmatrix}1\\0\\1\\0\\1\end{bmatrix}, b_9 = b_4 + b_8 = \begin{bmatrix}1\\0\\1\\1\\1\end{bmatrix}, b_{10} = b_6 + b_9 = \begin{bmatrix}1\\1\\0\\1\\1\end{bmatrix} \right\}. \tag{2.13}$$

The target vectors T satisfy $T = \{g_1, g_2, g_3, g_4\} = \{b_8, b_6, b_9, b_{10}\}$, hence B comprises an addition scheme according to the conditions (1-3).

The DAG representation of an addition scheme over $\mathbb{F}_2$ is an unweighted DAG such that all nodes representing intermediate sums have exactly 2 incoming edges, and all nodes representing target vectors in $T \setminus E$ have exactly 2 incoming edges. Moreover, the two incoming edges for any node b_k have to satisfy that the source nodes b_i and b_j satisfy $b_k = b_j + b_i$. Finally, the DAG satisfies that all nodes representing vectors in $T \setminus E$ are formed, and that only nodes representing elements of E has no incoming edges, see Figure 3.

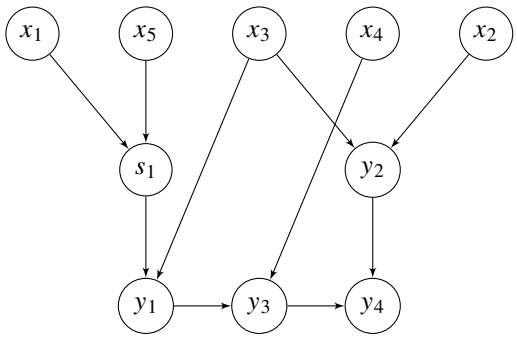


Fig. 3: Directed acyclic graph representation of the addition scheme for the matrix in equation (2.11) also illustrated in equations (2.12) and (2.13). The ordering of $x_1, \ldots, x_5$ is chosen so as to minimize edge overlaps.

Finally, we can represent any addition scheme by a sequence of additions involving the inputs and intermediate variables. Specifically, let $x \in \mathbb{F}_2^n$ be the vector on which the matrix G acts. Then we can represent the addition scheme showcased in equations (2.12) and (2.13), and in Figure 3 by the following operations performed in order:

$$\begin{aligned} y_2 &= x_2 + x_3, \\ s_1 &= x_1 + x_5, \\ y_1 &= x_3 + s_1, \\ y_3 &= x_4 + y_1, \\ y_4 &= y_2 + y_3, \end{aligned}$$

where $T = \{g_1, g_2, g_3, g_4\} = \{y_1, y_2, y_3, y_4\}$.

We will use all three mentioned representations of addition schemes interchangeably throughout, since the translation between the three formats is straightforward. Moreover, we will specify the addition scheme representation methods used sparingly, as it is often straightforward to infer from context.

2.3.2 Lifting Addition Schemes

Although we will work almost exclusively over $\mathbb{F}_2$, some of the 3×3 matrix multiplication schemes obtained over $\mathbb{F}_2$ will later be extended to schemes over a general field $\mathbb{F}$.

Several methods have been proposed for lifting Strassen-type matrix multiplication schemes from $\mathbb{F}_2$ to more general fields. These include methods based on Gröbner bases [4], Hensel lifting [34], and heuristic constraint satisfaction techniques for progressively extending modular schemes [35]. We emphasize that unlike previous work, we are extending addition schemes, not full Strassen-type matrix multiplication schemes.

In this thesis, we restrict attention to lifting addition schemes for multiplication by matrices U, V and W, as they are defined in equation (2.3), i.e. we do not lift full matrix multiplication schemes. Additionally, this is done solely for relatively sparse rank-23 schemes for general 3×3 matrix multiplication. Inspired by [4], we consider only lifts that preserve the DAG structure of the $\mathbb{F}_2$ addition scheme and replace each edge weight 1 by a sign in $\{-1, 1\}$, see Figure 2.

Thus, our goal is to determine which entries equal to 1 in a given $\mathbb{F}_2$ addition scheme should be changed to -1 so that the resulting $\{-1, 0, 1\}$ addition scheme is valid over general fields $\mathbb{F}$. Our approach differs from that of [4] in that rather than using Gröbner basis computation, we introduce a backtracking method that efficiently prunes infeasible solutions. This allows us to quickly search systematically over the possible choices of signs in the lifted scheme.

As motivation for introducing our method of extending $\mathbb{F}_2$ addition schemes to addition schemes over general fields, consider the following examples:

Example 2.2. Suppose $G \in \{-1, 0, 1\}^{2\times 4}$ is given by

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = Gx = \begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}. \tag{2.14}$$

First we find the $\mathbb{F}_2$ version of equation (2.14), namely

$$\begin{bmatrix} y_1' \\ y_2' \end{bmatrix} = G'x = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}. \tag{2.15}$$

An optimal addition scheme for the instance of Problem 2.1 induced by G' with $\mathbb{F} = \mathbb{F}_2$ is given by

$$\begin{aligned} s_1 &= x_2 + x_3, \\ y_1' &= s_1 + x_1, \\ y_2' &= s_1 + x_4. \end{aligned}$$

This scheme, in $\mathbb{F}_2$ DAG form is given in Figure 4.

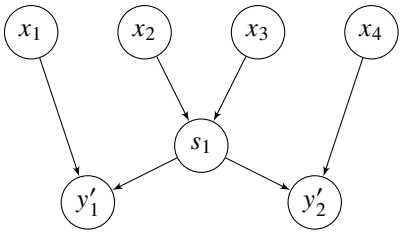


Fig. 4: DAG formulation of addition scheme for G' as it is defined in equation (2.15).

Now we want to extend the addition scheme detailed in Figure 4 for equation (2.15), to one for $G \in \{-1, 0, 1\}^{2\times 4}$ as it is defined in equation (2.14). To do so, we let the weight of each edge be an undetermined variable $\epsilon_i \in \{-1, 1\}$, and let the extension map y'_i to y_i, as shown in Figure 5.

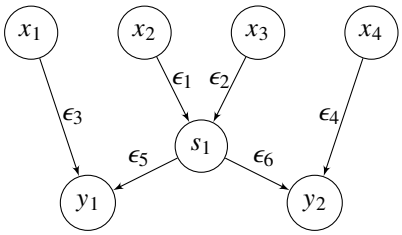


Fig. 5: Variable weighted DAG formulation of addition scheme for G' as it is defined in equation (2.15), but with undetermined ϵ-weighted edges.

To find the variable values $\epsilon_i \in \{-1, 1\}$ that make the addition scheme illustrated in Figure 5 valid, we write out the result of y_1 and y_2 in terms of x_1, x_2, x_3, and x_4 with coefficients that are polynomials in $\epsilon_1, \ldots, \epsilon_6$:

$$\begin{aligned} y_1 &= \epsilon_3 x_1 + \epsilon_5 s_1 = \epsilon_3 x_1 + \epsilon_5\epsilon_1 x_2 + \epsilon_5\epsilon_2 x_3, \\ y_2 &= \epsilon_4 x_4 + \epsilon_6 s_1 = \epsilon_6\epsilon_1 x_2 + \epsilon_6\epsilon_2 x_3 + \epsilon_4 x_4. \end{aligned}$$

By comparing that to the coefficients of directly expanding equation (2.14), we get the set of equations in $\epsilon_1, \ldots, \epsilon_6$:

$$\begin{aligned} \epsilon_3 &= 1, & \epsilon_1\epsilon_5 &= -1, \\ \epsilon_2\epsilon_5 &= 1, & \epsilon_4 &= -1, \\ \epsilon_1\epsilon_6 &= 1, & \epsilon_2\epsilon_6 &= -1, \end{aligned}$$

where $\epsilon_1, \ldots, \epsilon_6 \in \{-1, 1\}$. We can see that solutions exist, and are completely determined by the value of one of $\epsilon_1, \epsilon_2, \epsilon_5$, and ϵ_6, hence there are only two solutions. One solution is given by:

$$\begin{aligned} \epsilon_1 &= -1, & \epsilon_2 &= 1, & \epsilon_3 &= 1, \\ \epsilon_4 &= -1, & \epsilon_5 &= 1, & \epsilon_6 &= -1, \end{aligned}$$

the other is given by:

$$\begin{aligned} \epsilon_1 &= 1, & \epsilon_2 &= -1, & \epsilon_3 &= 1, \\ \epsilon_4 &= -1, & \epsilon_5 &= -1, & \epsilon_6 &= 1. \end{aligned}$$

Using the first solution found, we see that the addition scheme

$$\begin{aligned} s_1 &= -x_2 + x_3, \\ y_1 &= x_1 + s_1, \end{aligned}$$

$$y_2 = -s_1 - x_4,$$

is a valid extension of the addition scheme from $\mathbb{F}_2$ to $\{-1, 0, 1\}$ which works over all fields $\mathbb{F}$.

Example 2.3. Suppose $G \in \{-1, 0, 1\}^{3\times 6}$ is given by

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = Gx = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 & 1 & 1 \\ 1 & 1 & 1 & -1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix}. \tag{2.16}$$

First we find the $\mathbb{F}_2$ version of equation (2.16), namely

$$\begin{bmatrix} y'_1 \\ y'_2 \\ y'_3 \end{bmatrix} = G'x = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix}. \tag{2.17}$$

An addition scheme for the instance of Problem 2.1 induced by G' with $\mathbb{F} = \mathbb{F}_2$ is given by the directed acyclic graph in Figure 6.

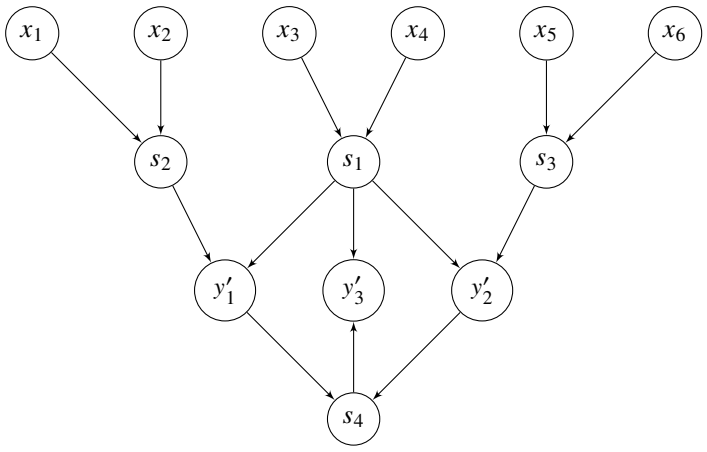


Fig. 6: $\mathbb{F}_2$ DAG formulation of addition scheme for G' as it is defined in equation (2.17).

We now investigate whether the addition scheme detailed in Figure 6 for equation (2.17) can be extended to one for $G \in \{-1, 0, 1\}^{3\times 6}$ as defined in equation (2.16). To do so, we represent the weight of each edge as a variable $\epsilon_i \in \{-1, 1\}$, and let our extension map the $\mathbb{F}_2$ outputs y'_i to $\mathbb{F}$ outputs y_i, as shown in Figure 7.

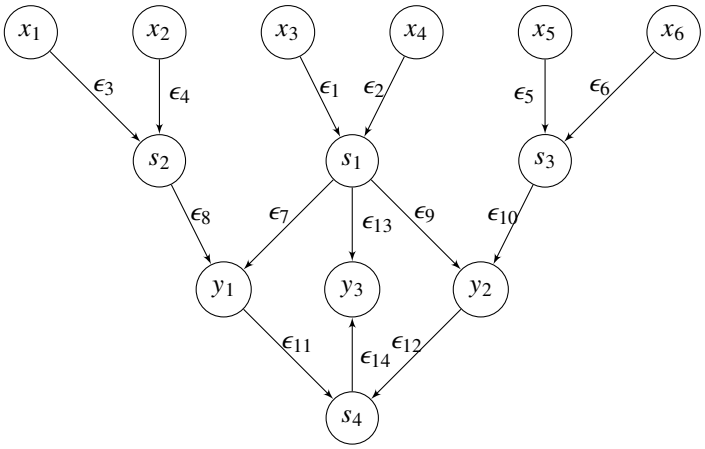


Fig. 7: DAG formulation of addition scheme for G' as it is defined in equation (2.17), but with undetermined ϵ-weighted edges.

By comparing coefficients of $x_1, \ldots, x_6$ in the direct expansion of equation (2.16) with the coefficients derived from the addition scheme illustrated in Figure 7, we obtain the following:

$$x_1 + x_2 + x_3 + x_4 = y_1 = \epsilon_7 s_1 + \epsilon_8 s_2 = \epsilon_7\epsilon_1 x_3 + \epsilon_7\epsilon_2 x_4 + \epsilon_8\epsilon_3 x_1 + \epsilon_8\epsilon_4 x_2 \tag{2.18}$$

$$-x_3 + x_4 + x_5 + x_6 = y_2 = \epsilon_9 s_1 + \epsilon_{10} s_3 = \epsilon_9\epsilon_1 x_3 + \epsilon_9\epsilon_2 x_4 + \epsilon_{10}\epsilon_5 x_5 + \epsilon_{10}\epsilon_6 x_6 \tag{2.19}$$

$$\begin{aligned} x_1 + x_2 + x_3 - x_4 + x_5 + x_6 = y_3 = {} & \epsilon_{14}\epsilon_{11}\epsilon_8\epsilon_3 x_1 + \epsilon_{14}\epsilon_{11}\epsilon_8\epsilon_4 x_2 + (\epsilon_{13}\epsilon_1 + \epsilon_{14}\epsilon_{11}\epsilon_7\epsilon_1 + \epsilon_{14}\epsilon_{12}\epsilon_9\epsilon_1)\, x_3 \\ & + (\epsilon_{13}\epsilon_2 + \epsilon_{14}\epsilon_{11}\epsilon_7\epsilon_2 + \epsilon_{14}\epsilon_{12}\epsilon_9\epsilon_2)\, x_4 + \epsilon_{14}\epsilon_{12}\epsilon_{10}\epsilon_5 x_5 + \epsilon_{14}\epsilon_{12}\epsilon_{10}\epsilon_6 x_6 \end{aligned} \tag{2.20}$$

By comparing coefficients of the x_i in equations (2.18)–(2.20) we obtain the polynomial equations

$$\begin{aligned} \epsilon_8\epsilon_3 &= 1, & \epsilon_8\epsilon_4 &= 1, \\ \epsilon_7\epsilon_1 &= 1, & \epsilon_7\epsilon_2 &= 1, \\ \epsilon_9\epsilon_1 &= -1, & \epsilon_9\epsilon_2 &= 1, \\ \epsilon_{10}\epsilon_5 &= 1, & \epsilon_{10}\epsilon_6 &= 1, \\ \epsilon_{14}\epsilon_{11}\epsilon_8\epsilon_3 &= 1, & \epsilon_{14}\epsilon_{11}\epsilon_8\epsilon_4 &= 1, \\ \epsilon_{13}\epsilon_1 + \epsilon_{14}\epsilon_{11}\epsilon_7\epsilon_1 + \epsilon_{14}\epsilon_{12}\epsilon_9\epsilon_1 &= 1, & \epsilon_{13}\epsilon_2 + \epsilon_{14}\epsilon_{11}\epsilon_7\epsilon_2 + \epsilon_{14}\epsilon_{12}\epsilon_9\epsilon_2 &= -1, \\ \epsilon_{14}\epsilon_{12}\epsilon_{10}\epsilon_5 &= 1, & \epsilon_{14}\epsilon_{12}\epsilon_{10}\epsilon_6 &= 1. \end{aligned}$$

By considering the four equations $\epsilon_9\epsilon_1 = -1, \epsilon_9\epsilon_2 = 1, \epsilon_7\epsilon_2 = 1$, and $\epsilon_7\epsilon_1 = 1$, and the fact that $\epsilon_i \in \{-1, 1\}$, we see that

$$\begin{aligned} \epsilon_9 &= -\epsilon_1 & \epsilon_9 &= \epsilon_2, \\ \epsilon_7 &= \epsilon_2 & \epsilon_7 &= \epsilon_1, \end{aligned}$$

which in turn implies $\epsilon_9 = -\epsilon_1 = -\epsilon_7 = -\epsilon_2 = -\epsilon_9$, a contradiction. Hence we conclude that we cannot extend the $\mathbb{F}_2$ addition scheme to one working in all fields $\mathbb{F}$, by flipping certain of the additions from 1 to -1.

What Examples 2.2 and 2.3 illustrates is that we can express each instance of our version of the lifting problem, where we only consider extending to the ternary set of values $\{-1, 0, 1\}$, as the problem of finding solutions to a set of polynomial equations with values in $\{-1, 1\}$. To determine whether solutions exist we devise a backtracking approach over the set of variables, where we order the variables in such a way that the shortest polynomial equations, i.e. those involving the fewest variables, will be determined as early as possible during the backtracking. Such a setup allows for early pruning of candidate solutions, making the worst-case exponential exhaustive search feasible for our instances, see Appendix D for examples of addition scheme instances lifted using this approach.

Chapter 3

Data

3.1 Dataset for General 3 × 3 Matrix Multiplication

Previous work on reducing the number of additions required to compute existing rank-23 schemes for multiplying 3×3 matrices only considers relatively few well-known schemes [25]. We make use of the database of over 17,000 rank-23 multiplication schemes due to Heule et al. [4].

Each of the schemes found by Heule et al. was first computed in $\mathbb{F}_2$, and then most of them were lifted to general fields $\mathbb{F}$ by systematically changing some of the 1s to −1s [4]. We can therefore easily revert each of the $\mathbb{F}$ schemes to the $\mathbb{F}_2$ schemes from which they were derived, by replacing each entry of U,V, and W by its absolute value, that is, changing each -1 to a 1.

Example 3.1. The first rank-23 matrix-matrix multiplication scheme for 3×3 matrix multiplication was introduced by Laderman in 1976 [31]. In the form of equation (2.3), Laderman's scheme can be written in terms of the matrices illustrated in Figure 8.

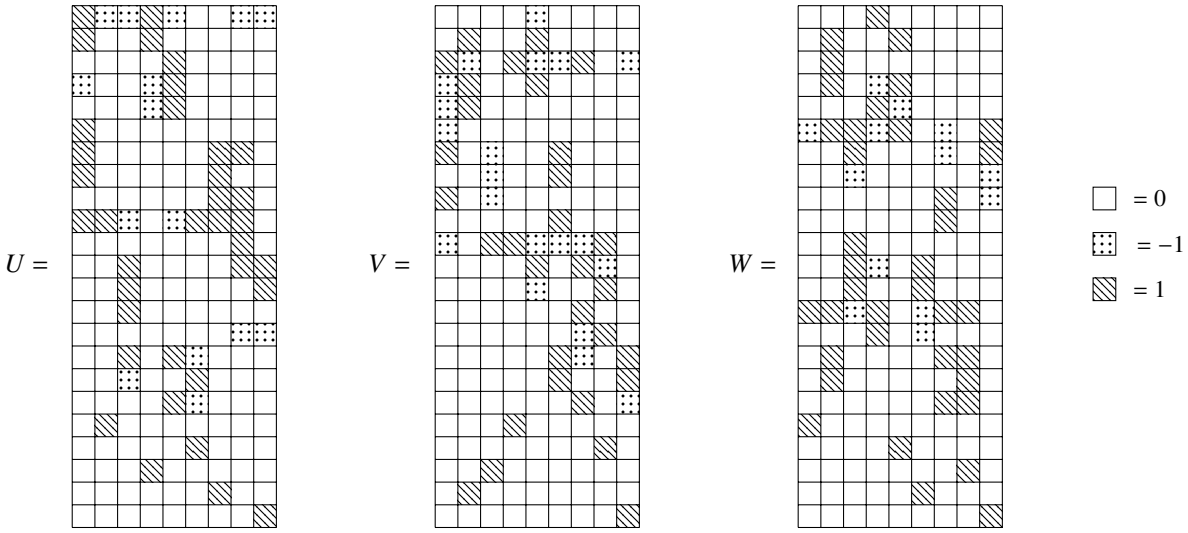


Fig. 8: Illustration of the matrices U, V, and W that constitute Laderman's rank-23 scheme for multiplying 3×3 matrices [31]. These specific matrices U, V, and W were taken from the Laderman scheme provided by [4].

The matrices $U, V, W \in \{-1, 0, 1\}^{23 \times 9}$ that constitute Laderman's scheme, see Figure 8, work over general fields $\mathbb{F}$ to produce a valid 3×3 matrix multiplication scheme, and if we convert each -1 into a 1, then it will still work in $\mathbb{F}_2$. The converse does not necessarily hold: a scheme over $\mathbb{F}_2$ does not necessarily lift to a ternary scheme over general fields [4].

3.1.1 Basis Change

In the work [2] the authors show that one can apply a change of basis to equation (2.3) in $O(n^2 \log n)$ time, which can result in a more advantageous representation in terms of the number of additions required to compute a multiplication scheme of the form equation (2.3).

Consider the same setup as in equation (2.3), given the three invertible matrices $B_U, B_V, B_W \in \{-1, 0, 1\}^{k^2 \times k^2}$ for some fixed $k \in \mathbb{N}$, we can write:

$$\begin{aligned} \vec{AB} &= W^\intercal \left((U\vec{A}) \odot (V\vec{B}) \right) \\ &= B_W \left(B_W^{-1} W^\intercal \right) \left(\left((UB_U^{-1})(B_U\vec{A}) \right) \odot \left((VB_V^{-1})(B_V\vec{B}) \right) \right). \end{aligned}$$

Let us define

$$(W')^\intercal = B_W^{-1} W^\intercal, \qquad U' = U B_U^{-1}, \qquad V' = V B_V^{-1}, \tag{3.1}$$

each requiring $O(k^6 + rk^4)$ arithmetic operations to compute, but these only have to be computed once, and can then be reused at all steps of the recursion in Algorithm 2. At each step of the recursive algorithm, having to apply the basis change matrices B_U, B_V, and B_W directly, quickly ends up using a lot more additions than performing the algorithm with no basis-change. So to make the basis change actually possibly useful, we need a way to compute these basis change matrix products using only few additions.

Theorem 3.1. *Let $k \geq 2$ be fixed, and let $n = k^\ell$ for some $\ell \in \mathbb{N}$. Suppose $U, V, W \in \mathbb{F}^{r \times k^2}$ satisfy equation* (2.3)*. Then the basis change induced by any $B_U, B_V, B_W \in \mathbb{F}^{k^2 \times k^2}$ that are invertible over $\mathbb{F}$ can be implemented by preprocessing the inputs and postprocessing the output, with each such step using $O(n^2 \log n)$ arithmetic operations.*

Proof Define $\mathcal{M}_\ell^{U,V,W}(A, B)$ as the matrix produced by Algorithm 2 applied to two $k^\ell \times k^\ell$ matrices A, B, using coefficient matrices $U, V, W \in \mathbb{F}^{r \times k^2}$. Since U, V, W satisfy equation (2.3), we have $\mathcal{M}_\ell^{U,V,W}(A, B) = AB$.

For each $i \in \{1, 2, \ldots, \ell\}$ we consider any $Y \in \mathbb{F}^{k^\ell \times k^\ell}$ as a $k^{\ell-i} \times k^{\ell-i}$ block matrix whose entries are $k \times k$ block matrices, which in turn has entries in $\mathbb{F}^{k^{i-1} \times k^{i-1}}$. Denote this block decomposition by $Y^{(i)}$, see Figure 9.

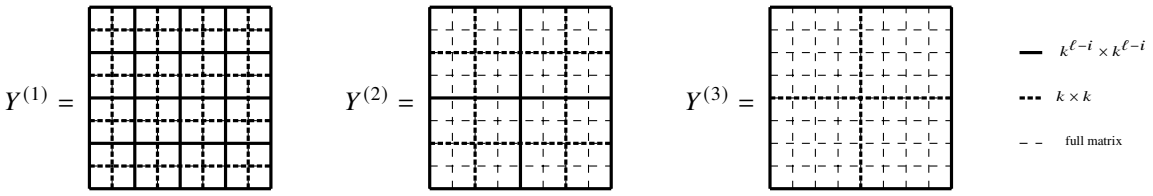


Fig. 9: Illustration of matrices $Y^{(1)}$, $Y^{(2)}$, and $Y^{(3)}$ for matrix $Y \in \mathbb{F}^{8\times 8}$, that is $k = 2$, and $\ell = 3$.

For $Z \in \{U, V, W\}$, define recursively the *linear* matrix operator $T_i^{B_Z} : \mathbb{F}^{k^i \times k^i} \to \mathbb{F}^{k^i \times k^i}$, such that $T_0^{B_Z} Y = Y$, and for each $i = 1, \ldots, \ell$ we have

$$T_i^{B_Z} Y = \textsc{BlockMat}_k \left(B_Z \cdot \textsc{BlockVec}_k \left(\begin{bmatrix} T_{i-1}^{B_Z} Y_{1,1} & \cdots & T_{i-1}^{B_Z} Y_{1,k} \\ \vdots & \ddots & \vdots \\ T_{i-1}^{B_Z} Y_{k,1} & \cdots & T_{i-1}^{B_Z} Y_{k,k} \end{bmatrix} \right) \right),$$

where Y is viewed as a $k \times k$ block matrix with blocks in $\mathbb{F}^{k^{i-1} \times k^{i-1}}$. Note that when computing $T_\ell^{B_Z}$, the operator applies the basis change to the $k^{2(\ell-i)}$ distinct $k \times k$ block matrices outlined by $Y^{(i)}$ for each $i = 1, \ldots, \ell$, see Figure 9.

We claim that given any triple of $k^2 \times k^2$ invertible matrices B_U, B_V, and B_W, for every $\ell \geq 1$ we have

$$T_\ell^{B_W} \mathcal{M}_\ell^{U',V',W'}(T_\ell^{B_U} A, T_\ell^{B_V} B) = \mathcal{M}_\ell^{U,V,W}(A, B) = AB, \tag{3.2}$$

with U', V', W' defined as in equation (3.1). This shows that the basis change can be absorbed into preprocessing of A and B by applying the operators $T_\ell^{B_U}$ and $T_\ell^{B_V}$ on A and B respectively, together with post-processing of the output via the linear operator $T_\ell^{B_W}$.

We prove the claim, equation (3.2), by induction on ℓ. For $\ell = 1$ we have

$$T_1^{B_U} A = \textsc{BlockMat}_k \left(B_U \cdot \textsc{BlockVec}_k(A) \right), \tag{3.3}$$

$$T_1^{B_V} B = \textsc{BlockMat}_k \left(B_V \cdot \textsc{BlockVec}_k(B) \right), \tag{3.4}$$

then lines 5–8 in Algorithm 2 corresponding to $\mathcal{M}_1^{U',V',W'}(T_1^{B_U} A, T_1^{B_V} B)$ become:

$$U' \textsc{BlockVec}_k \left(T_1^{B_U} A \right) \overset{(3.3)}{=} U B_U^{-1} \textsc{BlockVec}_k \left(\textsc{BlockMat}_k \left(B_U \cdot \textsc{BlockVec}_k(A) \right) \right) \tag{3.5}$$

$$= U \textsc{BlockVec}_k(A), \tag{3.6}$$

$$V' \textsc{BlockVec}_k \left(T_1^{B_V} B \right) \overset{(3.4)}{=} V B_V^{-1} \textsc{BlockVec}_k \left(\textsc{BlockMat}_k \left(B_V \cdot \textsc{BlockVec}_k(B) \right) \right) \tag{3.7}$$

$$= V \textsc{BlockVec}_k(B), \tag{3.8}$$

using that $\textsc{BlockMat}_k$ and $\textsc{BlockVec}_k$ are inverses of each other. Now equations (3.6) and (3.8) are exactly the same intermediate output that $\mathcal{M}_\ell^{U,V,W}(A,B)$ would have at lines 5–8 in Algorithm 2. Hence the intermediate r-vector with matrix entries M, see lines 9 and 11 in Algorithm 2, will be the same between $\mathcal{M}_\ell^{U,V,W}(A,B)$ and $\mathcal{M}_\ell^{U',V',W'}(T_1^{B_U}A, T_1^{B_V}B)$. Therefore line 13, in Algorithm 2 for input defined by $\mathcal{M}_\ell^{U',V',W'}(T_1^{B_U}A, T_1^{B_V}B)$ becomes

$$C = \textsc{BlockMat}_k\left((W')^\intercal M\right) = \textsc{BlockMat}_k\left(B_W^{-1}W^\intercal M\right), \tag{3.9}$$

if we then apply $T_1^{B_W}$ to this output, we get

$$T_1^{B_W}C = \textsc{BlockMat}_k\left(B_W\textsc{BlockVec}_k(C)\right) \overset{(3.9)}{=} \textsc{BlockMat}_k\left(W^\intercal M\right),$$

which coincides exactly with the output of $\mathcal{M}_\ell^{U,V,W}(A,B)$, namely the matrix-product AB. This proves the claim in equation (3.2) for $\ell = 1$.

Now assume equation (3.2) holds for some $\ell = m-1 \in \mathbb{N}$, and consider $\ell = m$. Let $\mathcal{A}$ and $\mathcal{B}$ be defined as in lines 6 and 8 in Algorithm 2 for the non-basis changed function call $\mathcal{M}_\ell^{U,V,W}(A,B)$. By the linearity of $T_{m-1}^{B_X}$, we can compute lines 5–8 in Algorithm 2, as follows:

$$\begin{aligned}
X'\textsc{BlockVec}_k\left(T_m^{B_X}Y\right) &= X\textsc{BlockVec}_k\left(\begin{bmatrix} T_{m-1}^{B_X}Y_{1,1} & \cdots & T_{m-1}^{B_X}Y_{1,k} \\ \vdots & \ddots & \vdots \\ T_{m-1}^{B_X}Y_{k,1} & \cdots & T_{m-1}^{B_X}Y_{k,k}\end{bmatrix}\right) \\
&= \begin{bmatrix} \sum_{i=1}^{k}\sum_{j=1}^{k} X_{1,(i-1)k+j}T_{m-1}^{B_X}Y_{i,j} \\ \vdots \\ \sum_{i=1}^{k}\sum_{j=1}^{k} X_{r,(i-1)k+j}T_{m-1}^{B_X}Y_{i,j}\end{bmatrix} = \begin{bmatrix} T_{m-1}^{B_X}\sum_{i=1}^{k}\sum_{j=1}^{k} X_{1,(i-1)k+j}Y_{i,j} \\ \vdots \\ T_{m-1}^{B_X}\sum_{i=1}^{k}\sum_{j=1}^{k} X_{r,(i-1)k+j}Y_{i,j}\end{bmatrix} = \begin{bmatrix} T_{m-1}^{B_X}\mathcal{Y}_1 \\ \vdots \\ T_{m-1}^{B_X}\mathcal{Y}_r\end{bmatrix},
\end{aligned}$$

for $(X,Y) \in \{(U,A),(V,B)\}$, and $\mathcal{Y}_i$ being the corresponding entry $\mathcal{A}_i$ or $\mathcal{B}_i$ for the corresponding function call for $\mathcal{M}_\ell^{U,V,W}(A,B)$. By the induction hypothesis applied to each recursive multiplication of size $k^{m-1}\times k^{m-1}$, the i-th recursive product produced by the transformed scheme at line 11 in Algorithm 2 is

$$M_i' = \left(T_{m-1}^{B_W}\right)^{-1}(M_i) = \left(T_{m-1}^{B_W}\right)^{-1}(\mathcal{A}_i\mathcal{B}_i),$$

with $M_i = \mathcal{A}_i\mathcal{B}_i$ being the corresponding entry of M produced by the non-basis changed function call $\mathcal{M}_\ell^{U,V,W}(A,B)$. Therefore line 14, in Algorithm 2, becomes:

$$\begin{aligned}
C &= \textsc{BlockMat}_k\left((W')^\intercal M'\right) = \textsc{BlockMat}_k\left(B_W^{-1}W^\intercal M'\right) \\
&= \textsc{BlockMat}_k\left(B_W^{-1}W^\intercal \cdot \begin{bmatrix}\left(T_{m-1}^{B_W}\right)^{-1}\mathcal{A}_1\mathcal{B}_1 \\ \vdots \\ \left(T_{m-1}^{B_W}\right)^{-1}\mathcal{A}_r\mathcal{B}_r\end{bmatrix}\right),
\end{aligned}$$

which is the output of $\mathcal{M}^{U',V',W'}(T_m^{B_U}A, T_m^{B_V}B)$, hence if we apply $T_m^{B_W}$ to C, then we get:

$$T_m^{B_W}C = \textsc{BlockMat}_k\left(B_W\textsc{BlockVec}_k\begin{bmatrix} T_{m-1}^{B_W}C_{1,1} & \cdots & T_{m-1}^{B_W}C_{1,k} \\ \vdots & \ddots & \vdots \\ yT_{m-1}^{B_W}C_{k,1} & \cdots & T_{m-1}^{B_W}C_{k,k}\end{bmatrix}\right) = \textsc{BlockMat}_k(W^\intercal M),$$

by linearity of $T_{m-1}^{B_W}$. We see that $T_m^{B_W}C$ coincides directly with $\mathcal{M}_m^{U,V,W}(A,B) = AB$, hence completing the proof of the claim made in equation (3.2).

It remains to show that the precomputation and post-processing of the basis change, i.e. that computing $T_\ell^{B_X}Y$ requires $O(n^2\log n)$ arithmetic operations, for each of $(B_X,Y) = (B_U,A),(B_V,B),(B_W,C)$. Let β be the number of submatrix additions required to apply the $k^2\times k^2$ basis-change matrix to one $k\times k$ block matrix. Since $Y^{(i)}$ has $(k^{(\ell-i)})^2 = k^{2(\ell-i)}$ distinct $k\times k$ matrices with entries in $\mathbb{F}^{k^{i-1}\times k^{i-1}}$, see Figure 9, and each submatrix addition requires $(k^{i-1})^2$ additions, hence the total number of arithmetic operations required to compute the basis change on $Y^{(1)},\ldots,Y^{(\ell)}$ is

$$\beta\sum_{i=1}^{\ell}\left(k^{2(\ell-i)}\right)\cdot\left(k^{2(i-1)}\right) = \frac{\beta}{k^2}\sum_{i=1}^{\ell}k^{2\ell} = \frac{\beta}{k^2}\ell k^{2\ell} = \frac{\beta}{k^2}n^2\log_k n. \tag{3.10}$$

The basis change is represented by a $k^2\times k^2$ matrix, and is applied to a k^2 element vector with matrix-entries, thus requiring at most $(k^2)^2 - k^2 = k^4 - k^2$, submatrix additions, see Algorithm 1. The number of scalar-matrix multiplications is k^4, see Algorithm 1. This in turn makes computing $T_\ell^{B_X}Y$ require at most $O(n^2\log n)$ arithmetic operations for any basis-change matrices $B_U, B_V, B_W \in \mathbb{F}^{k^2\times k^2}$, as per equation (3.10).

This completes the proof that we can compute the preprocessing and postprocessing steps induced by the invertible basis change matrices B_U, B_V, B_W in $O(n^2 \log n)$ arithmetic operations. With the basis change matrices giving rise to new representations of our coefficient matrices U, V, and W, which might lead to more efficient implementations of Algorithm 2. □

Theorem 3.1 works over general fields $\mathbb{F}$, and hence also for the restriction $B_U, B_V, B_W \in \{-1, 0, 1\}^{k^2 \times k^2}$ invertible over $\mathbb{R}$.

Example 3.2. To illustrate how we can precompute the basis-change induced by B_U for all levels of the recursion in Algorithm 2, using only $O(n^2 \log n)$ arithmetic operations, consider the 2×2 case shown in equation (2.2). Let the basis-change matrix B_U and its inverse B_U^{-1} be given, respectively, by

$$B_U = \begin{bmatrix} 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & -1 \\ -1 & 0 & 0 & -1 \\ 0 & -1 & 0 & -1 \end{bmatrix}, \qquad B_U^{-1} = \begin{bmatrix} 0 & 1 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ 1 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{bmatrix},$$

such that $U' = UB_U^{-1}$ is given by

$$U' = UB_U^{-1} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} 0 & 1 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ 1 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 1 & -1 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 0 & 1 & -1 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

Computing $U'x$ requires only 3 additions, compared to 5 additions for Ux. Let both B_V and B_W be the identity matrix, equivalently we only perform a basis change on U in equation (2.2), thus making it a 16-addition scheme, but with a basis change induced by B_U. Define $A, B \in \mathbb{R}^{4\times 4}$ whose product we want to compute:

$$A = \begin{bmatrix} 1 & -1 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ -1 & 0 & 1 & 1 \\ -1 & 1 & -1 & 0 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & -1 & 0 \\ -1 & 1 & 0 & 1 \end{bmatrix}.$$

We then precompute the basis-change $\tilde{A} = T_2^{B_U}(A)$ as detailed in the proof of Theorem 3.1.

$$\tilde{A} = T_2^{B_U} A = \textsc{BlockMat}_2 \left(B_U \textsc{BlockVec}_2 \begin{bmatrix} T_1^{B_U} \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} & T_1^{B_U} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \\ T_1^{B_U} \begin{bmatrix} -1 & 0 \\ -1 & 1 \end{bmatrix} & T_1^{B_U} \begin{bmatrix} 1 & 1 \\ -1 & 0 \end{bmatrix} \end{bmatrix} \right) \tag{3.11}$$

$$= \textsc{BlockMat}_2 \left(B_U \textsc{BlockVec}_2 \begin{bmatrix} \begin{bmatrix} -2 & -2 \\ -1 & -1 \end{bmatrix} & \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \\ \begin{bmatrix} -1 & 0 \\ -1 & 0 \end{bmatrix} & \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \end{bmatrix} \right) \tag{3.12}$$

$$= \textsc{BlockMat}_2 \left(\begin{bmatrix} 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & -1 \\ -1 & 0 & 0 & -1 \\ 0 & -1 & 0 & -1 \end{bmatrix} \begin{bmatrix} \begin{bmatrix} -2 & -2 \\ -1 & -1 \end{bmatrix} \\ \begin{bmatrix} -1 & 0 \\ -1 & 0 \end{bmatrix} \\ \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \\ \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \end{bmatrix} \right) = \begin{bmatrix} 0 & 1 & 1 & 3 \\ 0 & -2 & 1 & 0 \\ -1 & 1 & 0 & 1 \\ 0 & -1 & 1 & -1 \end{bmatrix} \tag{3.13}$$

Finally, by applying Algorithm 2 to $\tilde{A}, B \in \mathbb{R}^{4\times 4}$ with $U' = UB_U^{-1}$ in place of U, we get the desired matrix product AB. See Appendix B for a Python implementation of Algorithm 2 with $k = 2$ using this exact example as input.

We will mostly be working over $\mathbb{F}_2$, in this thesis, hence we consider mainly $B_U, B_V, B_W \in \mathbb{F}_2^{k^2\times k^2}$. We artificially enlarge the dataset introduced in Section 3.1 by applying random basis changes induced by invertible Bernoulli-random matrices B_U, B_V, and B_W to $U, V, W \in \mathbb{F}_2^{r\times k^2}$ respectively, using the process illustrated in the proof of Theorem 3.1, and in Example 3.2. For $Z \in \{U, V, W\}$ we will draw the basis change matrices $B_Z \in \mathbb{F}_2^{k^2\times k^2}$ from the distribution of Bernoulli-random binary matrices conditioned on being invertible: $B_Z \sim \left(\text{Bernoulli}(\rho)^{k^2\times k^2} \mid B_X \in \text{GL}_{k^2}(\mathbb{F}_2)\right)$, where ρ is the Bernoulli-bias used. We update the matrices $U, V, W \in \mathbb{F}_2^{r\times k^2}$ by

$$\begin{aligned} U &\leftarrow UB_U^{-1}, \\ V &\leftarrow VB_V^{-1}, \\ W^\intercal &\leftarrow B_W^{-1}W^\intercal. \end{aligned}$$

Together with the preprocessing and postprocessing maps induced by B_U, B_V, and B_W, these transformed matrices give basis-changed representations of the original schemes. Thus, each original scheme can generate many additional data points, with the number of generated data points proportional to the number of sampled basis-change triples.

Importantly, Theorem 3.1 shows that the data points generated by this basis-change procedure remain meaningful targets for addition minimization. If a basis-changed representation of a rank-r matrix multiplication scheme admits an α-addition implementation, then the associated recursive algorithm has the same leading constant in the asymptotic operation count as a rank-r non-basis-changed matrix multiplication scheme that admits an α-addition implementation.

3.2 Dataset for Structured Matrix Multiplication

Structured matrix-matrix multiplication is a category of matrix multiplication, where we have some prior structural information about the matrices we are multiplying. Examples include squaring matrices, multiplying a matrix by its transpose, multiplying an upper-triangular and a lower-triangular matrix, and many others. The recent work [5] identifies state-of-the-art Strassen-type schemes for a large class of structured matrix multiplication problems. The categories of structured matrices considered in [5], and by extension in our work, are summarized in Table 1.

Table 1: The types of structured matrices considered.

Type	**Shorthand**	3×3 **Example**
General	g	$\begin{bmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,1} & A_{3,2} & A_{3,3} \end{bmatrix}$
Upper Triangular	u	$\begin{bmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ 0 & A_{2,2} & A_{2,3} \\ 0 & 0 & A_{3,3} \end{bmatrix}$
Lower Triangular	l	$\begin{bmatrix} A_{1,1} & 0 & 0 \\ A_{2,1} & A_{2,2} & 0 \\ A_{3,1} & A_{3,2} & A_{3,3} \end{bmatrix}$
Symmetric	s	$\begin{bmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ A_{1,2} & A_{2,2} & A_{2,3} \\ A_{1,3} & A_{2,3} & A_{3,3} \end{bmatrix}$
Skew Symmetric	k	$\begin{bmatrix} 0 & A_{1,2} & A_{1,3} \\ -A_{1,2} & 0 & A_{2,3} \\ -A_{1,3} & -A_{2,3} & 0 \end{bmatrix}$
Skew Symmetric + Diagonal	w	$\begin{bmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ -A_{1,2} & A_{2,2} & A_{2,3} \\ -A_{1,3} & -A_{2,3} & A_{3,3} \end{bmatrix}$

The main distinction between structured matrix multiplication and Strassen-type general matrix multiplication is the combined use of both recursive structured matrix multiplication calls and general matrix multiplication calls, see Example 3.3. Reducing the total number of general matrix multiplication calls and recursive structured matrix multiplication calls will reduce the leading constant in the asymptotic operation count for multiplying structured matrices of the given form. Furthermore, reducing the ratio of general matrix multiplication calls to recursive structured matrix multiplication calls, will also reduce the leading constant in the asymptotic arithmetic operation count for multiplying the matrices, see Appendix A. Reducing the total number of additions used to compute a structured matrix multiplication scheme only has an effect on lower-order terms, see Example 3.3 below, and see Appendix A. Let R be a ring of $m \times m$ matrices over some field $\mathbb{F}$, then left-multiplying a matrix $B \in R^{n\times n}$ of type $x \in \{g, u, l, s, k, w, t\}$ by a matrix $A \in R^{n\times n}$ of type $y \in \{g, u, l, s, k, w\}$ will be referred to as a matrix multiplication of type xy. Where we remark that xt denotes multiplying the matrix A of type x with its transpose.

Example 3.3. Consider the gt structured matrix multiplication category for 3×3 block matrices over the field $\mathbb{F}_2$, that is, we want to compute:

$$AA^\intercal = \begin{bmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,1} & A_{3,2} & A_{3,3} \end{bmatrix} \begin{bmatrix} A_{1,1}^\intercal & A_{2,1}^\intercal & A_{3,1}^\intercal \\ A_{1,2}^\intercal & A_{2,2}^\intercal & A_{3,2}^\intercal \\ A_{1,3}^\intercal & A_{2,3}^\intercal & A_{3,3}^\intercal \end{bmatrix} =$$
$$\begin{bmatrix} A_{1,1}A_{1,1}^\intercal + A_{1,2}A_{1,2}^\intercal + A_{1,3}A_{1,3}^\intercal & A_{1,1}A_{2,1}^\intercal + A_{1,2}A_{2,2}^\intercal + A_{1,3}A_{2,3}^\intercal & A_{1,1}A_{3,1}^\intercal + A_{1,2}A_{3,2}^\intercal + A_{1,3}A_{3,3}^\intercal \\ A_{2,1}A_{1,1}^\intercal + A_{2,2}A_{1,2}^\intercal + A_{2,3}A_{1,3}^\intercal & A_{2,1}A_{2,1}^\intercal + A_{2,2}A_{2,2}^\intercal + A_{2,3}A_{2,3}^\intercal & A_{2,1}A_{3,1}^\intercal + A_{2,2}A_{3,2}^\intercal + A_{2,3}A_{3,3}^\intercal \\ A_{3,1}A_{1,1}^\intercal + A_{3,2}A_{1,2}^\intercal + A_{3,3}A_{1,3}^\intercal & A_{3,1}A_{2,1}^\intercal + A_{3,2}A_{2,2}^\intercal + A_{3,3}A_{2,3}^\intercal & A_{3,1}A_{3,1}^\intercal + A_{3,2}A_{3,2}^\intercal + A_{3,3}A_{3,3}^\intercal \end{bmatrix}.$$

Since $AA^\intercal$ is symmetric $(AA^\intercal)_{i,j} = (AA^\intercal)_{j,i}^\intercal$, and since transposing a matrix can be done in time proportional to the number of entries in the matrix, i.e. $(n/3)^2$ for each submatrix $(AA^\intercal)_{i,j}^\intercal$ given the submatrix $(AA^\intercal)_{j,i}$ we only need to compute the upper-triangular submatrices of $AA^\intercal$. To do so, we first compute the 6 recursive structured matrix multiplication calls denoted $S_1, \ldots, S_6$, and then the 9 general matrix products $M_7, \ldots, M_{15}$:

$$\begin{aligned}
S_1 &= A_{2,2}A_{2,2}^\intercal, \\
S_2 &= A_{3,1}A_{3,1}^\intercal, \\
S_3 &= A_{1,3}A_{1,3}^\intercal, \\
S_4 &= \left(A_{1,1} + A_{1,2} + A_{2,2}\right)\left(A_{1,1} + A_{1,2} + A_{2,2}\right)^\intercal, \\
S_5 &= \left(A_{1,3} + A_{2,2} + A_{3,2} + A_{3,3}\right)\left(A_{1,3} + A_{2,2} + A_{3,2} + A_{3,3}\right)^\intercal, \\
S_6 &= \left(A_{2,1} + A_{2,3} + A_{3,1}\right)\left(A_{2,1} + A_{2,3} + A_{3,1}\right)^\intercal, \\
M_7 &= \left(A_{1,3} + A_{3,3}\right)\left(A_{2,2} + A_{3,2} + A_{3,3}\right)^\intercal, \\
M_8 &= \left(A_{1,1} + A_{1,2} + A_{2,1} + A_{2,2}\right)A_{1,1}^\intercal, \\
M_9 &= A_{2,3}A_{1,3}^\intercal, \\
M_{10} &= A_{2,2}A_{3,2}^\intercal, \\
M_{11} &= \left(A_{2,1} + A_{2,3} + A_{3,1} + A_{3,3}\right)A_{2,3}^\intercal, \\
M_{12} &= \left(A_{2,1} + A_{2,3}\right)\left(A_{2,1} + A_{3,1}\right)^\intercal, \\
M_{13} &= \left(A_{1,2} + A_{2,2}\right)\left(A_{1,1} + A_{1,2}\right)^\intercal, \\
M_{14} &= \left(A_{2,1} + A_{2,2} + A_{3,1} + A_{3,2}\right)\left(A_{1,1} + A_{2,1} + A_{2,3}\right)^\intercal, \\
M_{15} &= \left(A_{1,1} + A_{1,2} + A_{1,3} + A_{2,1} + A_{2,2} + A_{2,3} + A_{3,2} + A_{3,3}\right)\left(A_{2,2} + A_{3,2}\right)^\intercal.
\end{aligned}$$

After which we can compute $AA^\intercal$ as follows, where we only show the upper-triangular part of the matrix, as the lower triangular part is simply given by the relation $(AA^\intercal)_{ij} = (AA^\intercal)^\intercal_{ji}$, by the symmetry of $AA^\intercal$:

$$AA^\intercal = \begin{bmatrix} S_1 + S_3 + S_4 & S_1 + S_4 + M_8^\intercal & S_1 + S_3 + S_4 + S_5 \\ + M_{13} + M_{13}^\intercal & + M_9^\intercal + M_{13} & + M_7^\intercal + M_8^\intercal + M_{12} \\ & & + M_{13} + M_{14}^\intercal + M_{15} \\ \star & S_1 + S_2 + S_6 & S_2 + S_6 + M_{10} \\ & + M_{12} + M_{12}^\intercal & + M_{11}^\intercal + M_{12}^\intercal \\ \star & \star & S_1 + S_2 + S_3 + S_5 \\ & & + M_7 + M_7^\intercal \\ & & + M_{10} + M_{10}^\intercal \end{bmatrix}.$$

This scheme requires 6 recursive calls, denoted $S_1, \ldots, S_6$, and 9 general matrix-matrix multiplication calls denoted $M_7, \ldots, M_{15}$. Hence the total number of operations required to compute $AA^\intercal$ with $A \in \mathbb{F}_2^{n \times n}$ for some $n = 3^\ell$ where $\ell \in \mathbb{N}$, is given by the recurrence relation:

$$S(n) = 6S(n/3) + 9M(n/3) + O(n^2), \quad S(1) = M(1) = 1, \tag{3.14}$$

where $S(n)$ and $M(n)$ denote the number of operations required to compute the structured $n \times n$ matrix product $AA^\intercal$, and the general $n \times n$ matrix product AB respectively, for $A, B \in \mathbb{F}^{n \times n}$. The solution of the recurrence relation depends on the number of operations required for general matrix multiplication, which in the context of work on structured matrix multiplication is usually assumed to be $Cn^\omega + O(n^2)$ for some $C > 0$ and either $\omega = \log_2 7$ or $\omega = 3$ [5]. Under this assumption the solution to equation (3.14) is given by the following:

$$S(n) = \frac{9}{3^\omega - 6} Cn^\omega + O(n^2). \tag{3.15}$$

A proof is provided in Appendix A. From equation (3.15) we see that the relative operation count of the provided recursive scheme for computing $AA^\intercal$, namely $S(n)$, as compared to regular matrix-matrix multiplication $M(n)$ is given by:

$$\frac{S(n)}{M(n)} = \frac{9}{3^\omega - 6} + o(1).$$

We remark that this exact scheme significantly improves upon the previous state-of-the-art gt matrix multiplication schemes [5, 36], and the details for how we discovered it can be found in Appendix A.

Since the efficiency of structured matrix multiplication schemes primarily depends on the ratio of recursive calls, to general matrix multiplication calls, most work on structured matrix multiplication, including [5], focuses only on optimizing this aspect. Khoruzhii et al. explicitly state that they leave reducing the total addition count of their schemes to future work [5]. In this work we apply the methods discussed in Section 4 to this secondary problem of minimizing additions.

The dataset consists of sets of linear equations, some of which have coefficients in $\mathbb{Q}$, and therefore do not reduce to schemes in $\mathbb{F}_2$. We collect all the structured matrix multiplication schemes that reduce to $\mathbb{F}_2$ when forming our dataset, so that the final dataset consists of 136 structured matrix multiplication schemes, and by extension 376 matrices to optimize over. We remark that whenever the structured matrix multiplication is of the form xt with $x \in \{g, u, l, s, k, w\}$ we only get 2 matrices to optimize over as there only are 2 unique inputs, see the gt example above, whereas when the scheme is of the form xy for $y \in \{g, u, l, s, k, w\}$, we get 3 distinct matrices.

3.3 Dataset for Cryptography Matrices

We remind the reader that most previous work on the SLP problem with $\mathbb{F} = \mathbb{F}_2$ has been done in the context of cryptography. This is due to the immediate applicability of such work to the physical hardware design of various linear layers in certain cryptographic circuits. Each addition required to compute a linear layer $G \in \mathbb{F}_2^{m \times n}$ for a given input $x \in \mathbb{F}_2^n$ corresponds directly to a physical XOR gate [8, 26, 37].

Example 3.4. Consider having to compute Gx over $\mathbb{F}_2$ for $G \in \mathbb{F}_2^{4\times 5}$ given by

$$G = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \end{bmatrix},$$

noting that this is the same example used in Equation (2.11). Since addition in $\mathbb{F}_2$ corresponds to performing the exclusive or, $\oplus$, operation, we can express Problem 2.1 as wanting to find the fewest number of XOR-gates required to compute Gx. Analogously, given a circuit consisting exclusively of XOR-gates, finding the fewest number of XOR gates required to perform the same operation on the input as the given circuit, corresponds exactly to Problem 2.1.

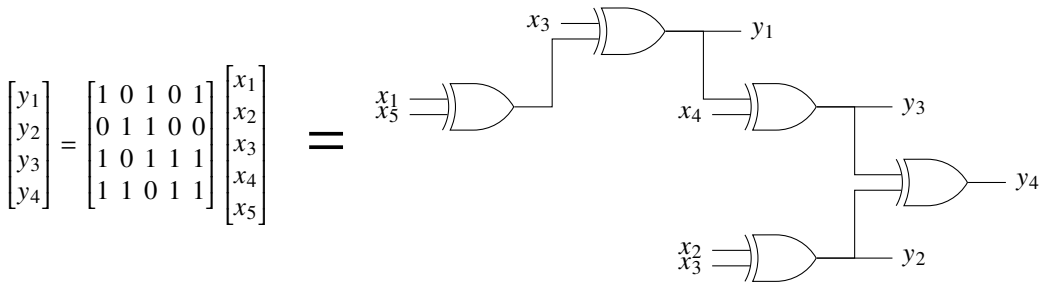


Fig. 10: Equivalence between XOR circuit and addition scheme. The addition scheme represented here is the same as for the example used in Section 2.3.1.

Figure 10 makes it clear that computing a representation of an XOR circuit using as few XOR gates as possible reduces to being an instance of Problem 2.1 with $\mathbb{F} = \mathbb{F}_2$. Note here the similarity to the DAG representation of an addition scheme.

Even though Problem 2.1 is a widespread problem with a large body of research in cryptographic literature, there seem to be no standardized datasets to test heuristics on. The best we could find is the set of matrices provided in the github repository for the work [26], and this is what we will use. There are 63 distinct matrices provided in this dataset together with, what was at the time, the smallest addition schemes for each of these 63 matrices.

3.4 Bernoulli Random Matrices

When introducing a new heuristic for solving Problem 2.1, or when proving theoretical results related to Problem 2.1 with $\mathbb{F} = \mathbb{F}_2$, it is common to test it on square Bernoulli-random matrices with biases $\rho = 0.1, 0.2, \ldots, 0.9$ [8, 37–39]. The work [39] uses a subset of the dataset from [38], by drawing matrices at random for each pair of matrix size and bias (n, ρ), such that $n = 15, 16, 17, 18, 19, 20$, and $\rho = 0.1, \ldots, 0.9$. Inspired by [39], we draw a matrix from a larger set of pairs (n, ρ), namely $n = 12, \ldots, 28$, and $\rho = 0.025, 0.05, \ldots, 0.95, 0.975$, such that our dataset of Bernoulli-random matrices ends up consisting of 663 distinct matrices.

```
1  import itertools, random, os
2  random.seed(628)
3  for n, betax40 in itertools.product(range(12, 29), range(1, 40)):
4      M = [[int(40 * random.random() < betax40) for j in range(n)] for i in range(n)]
5      os.makedirs("./benchmarks/bernoulli/dataset", exist_ok=True)
6      with open(f"./benchmarks/bernoulli/dataset/n_{n}_bx40_{betax40}.txt", "w") as f:
7          f.write(f"{n} {n}\n" + "\n".join([" ".join([str(v) for v in row]) for row in M]))
```

Listing 1: Code used for generating Bernoulli random dataset, shown here for reproducibility.

The code used to generate the datasets in [38, 39] is not provided, thus for future reproducibility we provide our dataset generation code above.

3.4.1 Greedy Potential $\mathbb{F}_2$ Test Dataset

We also use a slight modification of the Bernoulli-random matrix dataset to test some unexpected results we find from running the greedy potential method described in Section 4.2.2. For each matrix-size $n = 1, 2, \ldots, 64$, we construct the ternary $\{-1, 0, 1\}$ dataset of square matrices $M \in \{-1, 0, 1\}^{n \times n}$ with the entry of matrix $i, j \in [n] \times [n]$ being determined by

$$M_{i,j} = \begin{cases} 1, & 0 \leq p_{ij} \leq \frac{\rho}{2} \\ -1, & \frac{\rho}{2} \leq p_{ij} \leq \rho \\ 0, & \rho \leq p_{ij} \leq 1 \end{cases} \tag{3.16}$$

for p_{ij} drawn uniformly on the unit interval $[0, 1]$, where ρ is a *density* parameter we very. We specifically construct our dataset using Section 3.4.1.

```
import itertools, random, os
random.seed(628)
for n, betax40 in itertools.product(range(1, 65), range(1, 40)):
    M = [[2 * int(random.random() < 0.5) - 1 for j in range(n)] for i in range(n)]
    os.makedirs("./dataset_ii", exist_ok=True)
    os.makedirs("./dataset_F2_ii", exist_ok=True)
    with open(f"./dataset_ii/n_{n}_bx40_{betax40}.txt", "w") as f:
        f.write(f"{n} {n}\n" + "\n".join([" ".join([str(v * (40 * random.random() < betax40)
    ) for v in row]) for row in M]))
    with open(f"./dataset_F2_ii/n_{n}_bx40_{betax40}.txt", "w") as f:
        f.write(f"{n} {n}\n" + "\n".join([" ".join([str(abs(v) * (40 * random.random() <
    betax40)) for v in row]) for row in M]))
```

Listing 2: Code used to generate test dataset for comparing the ternary and binary greedy potential methods described in Section 4.2.2. See Figure 27 for the use of this dataset.

We remark that the greedy potential $\mathbb{F}_2$ Bernoulli-random dataset was generated for the sole purpose of generating Figure 27, and is used nowhere else in the thesis, except for in generating this figure.

Chapter 4

Shortest Linear Program

Solving the shortest linear program (SLP) problem, i.e. Problem 2.1, is in general NP-hard [37], which motivates the need for heuristics that quickly and reliably find good, but possibly non-optimal, solutions. While finding the shortest linear program over $\mathbb{F}_2$ is a well-known and highly studied problem [8, 37, 39], most studies on the SLP problem are to be found in literature related to cryptography. To the best of our knowledge, no existing work on minimizing the additions in Strassen-type matrix multiplication has taken into consideration the work on the SLP problem from the cryptography-related research and the other way around. We aim to bridge this gap by implementing and improving upon the most prominent and efficient heuristics proposed for solving the SLP problem in research on cryptography and research on Strassen-type matrix multiplication.

All heuristics will be described for the $\mathbb{F}_2$ case, as this is the field over which most of the heuristics were developed. A few of the heuristics will be described first over some non-$\mathbb{F}_2$ structure, and then converted to an $\mathbb{F}_2$ SLP heuristic, it will be made clear to the reader when this is the case and what the initial structure we are working over for each heuristic is.

The heuristic implementations are combined into a unified framework to allow for easy use in other projects, see Appendix E for the code.

4.1 Preprocessing

Even though Problem 2.1 is widely studied, we are not aware of any prior work on preprocessing SLP instances in order to simplify them before optimization. In this work we introduce several simple preprocessing steps that reduce the complexity of the instances passed to the heuristics described in Section 4.2. Some of the preprocessing steps proposed here are lossless, in that they preserve the optimum and allow any optimal solutions for reduced/preprocessed instances to be translated into optimal solutions for the original instance. Some of the proposed preprocessing methods are lossy. For each preprocessing method, we will make clear whether it is lossless or not. Throughout this section, let $G \in \mathbb{F}_2^{m \times n}$ denote the matrix representing an instance of Problem 2.1.

4.1.1 Row and Column Reduction

Certain row-forms and certain column-forms lend themselves to simple reduction rules that can simplify the given instance of Problem 2.1. These patterns are common in small low-density matrices, which fits most of the datasets considered, see Section 3.

4.1.1.1 Row Reduction. Suppose G contains a row indexed by $i \in [m]$ which has at most one nonzero entry $G_{i,j}$ for some $j \in [n]$. The effect induced by this row on the output Gx for $x \in \mathbb{F}_2^n$ is to insert $G_{i,j}x_j$ into the i-th entry of the output vector.

Lemma 4.1 (Row Reduction Preprocessing)**.** *If $G \in \mathbb{F}_2^{m \times n}$ contains a row indexed by $i \in [m]$ with at most one nonzero entry $G_{i,j}$ for some $j \in [n]$, and $G' \in \mathbb{F}_2^{(m-1) \times n}$ is obtained from G by removing row i from G, then there is a bijective correspondence between addition schemes for G' and addition schemes for G, which preserves the number of additions required to compute Gx.*

Proof Any addition scheme for G' can be translated into one for G by inserting $G_{i,j}x_j$ in the output vector at the position where row i was removed. Similarly any addition scheme for G can be translated into one for G' by simply removing the i-th entry of the output Gx that the addition scheme for G computes:

$$Gx = \begin{bmatrix} \vdots \\ G_{i-1,:}x \\ G_{i,j}x_j \\ G_{i+1,:}x \\ \vdots \end{bmatrix} \quad \xleftrightarrow{\text{add / remove } G_{i,j}x_j} \quad G'x = \begin{bmatrix} \vdots \\ G_{i-1,:}x \\ G_{i+1,:}x \\ \vdots \end{bmatrix}.$$

As such there is a one-to-one correspondence between addition schemes for G and addition schemes for G', and as neither inserting nor removing the i-th entry, $G_{i,j}x_j$, affects the number of additions required to compute an addition scheme, this completes the proof. □

Lemma 4.1 tells us that an optimal addition scheme for $G' \in \mathbb{F}_2^{(m-1)\times n}$, the matrix corresponding to G without row i, can be directly translated into an optimal addition scheme for G.

4.1.1.2 Column Reduction. Suppose that all rows in G have at least two nonzero entries. If this is not the case, then we can simply apply Lemma 4.1 until it is the case. Assume G contains a column indexed by $j \in [n]$ which has at most one nonzero entry $G_{i,j}$ for some $i \in [m]$. The effect induced by this column on the output Gx for $x \in \mathbb{F}_2^n$ is to add $G_{i,j}x_j$ onto the i-th entry of the output vector.

Lemma 4.2 (Column Reduction)**.** *Suppose $G \in \mathbb{F}_2^{m\times n}$ has no row with fewer than two nonzero entries, it contains a column indexed by $j \in [n]$ which has at most one nonzero entry $G_{i,j}$ for some $i \in [m]$, and $G' \in \mathbb{F}_2^{m\times(n-1)}$ is obtained by removing column j from G, then any optimal addition scheme for G' can be translated into an optimal addition scheme for G.*

Proof Suppose $G_{i,j} = 0$, any addition scheme for G' then translates directly to one for G with no change, except accepting an extra input x_j which we do nothing with. Similarly, we can translate any addition scheme for G to one for G', by removing x_j as an input. Clearly this translation between addition schemes for G and G' has no effect on the number of additions required, hence an optimal addition scheme for G' can be translated into an optimal addition scheme for G this way:

$$Gx = \sum_{c=1}^{n} G_{:,c}x_c = \sum_{\substack{c=1\\c\neq j}}^{n} G_{:,c}x_c = G'x',$$

due to $G_{:,j} = 0$, where x' is x without the j-th entry.

Now suppose $G_{i,j} = 1$, and $G_{i,j'} = 1$ for some $j' \in [n] \setminus \{j\}$, which exists as per the lemma statement. Let α be the number of additions required to compute an optimal addition scheme for G, let α' be the number of additions required to compute an optimal addition scheme for G'. The input x_j occurs in at least one addition in the addition scheme for G, as otherwise we could compute $(G(ae_j + be_{j'}))_i = a + b$ for all $a, b \in \mathbb{F}_2$ using no additions:

$$G(ae_j + be_{j'}) = aG_{:,j} + bG_{:,j'} = ae_i + bG_{:,j'} = \begin{bmatrix} bG_{1,j'} \\ \vdots \\ bG_{i-1,j'} \\ aG_{i,j} + bG_{i,j'} \\ bG_{i+1,j'} \\ \vdots \\ bG_{m,j'} \end{bmatrix}.$$

Specifically assume that x_j occurs in no addition in the optimal addition scheme $B \subseteq \mathbb{F}_2^n$ for G, that is, for all $b_k \in B$ there are no $b_i \in B$ such that $b_k = e_j + b_i$. For each $b_k \in B \setminus E$ with $b_k = b_{i_1} + b_{i_2}$ for some $k > i_1 > i_2$ we have two cases. Namely the case that $j \in \{i_1, i_2\}$, but this would violate the assumption of j not being in any addition. The alternative case, that $j \notin \{i_1, i_2\}$ for all b_k with $k > n$ implies that $e_j^\intercal b_k = 0$ for all $b_k \neq e_j$. This follows easily by induction on k starting at $k = n$. Since for all $b_k \in B \setminus E$ we have $e_j^\intercal b_k = 0$, we must have that all rows of G, $g_k \in T \subseteq B$, satisfies

$$e_j^\intercal g_k = \begin{cases} 1, & g_k = e_j \\ 0, & \text{otherwise} \end{cases}$$

contradicting the existence of $j' \in [n] \setminus \{j\}$ with $G_{i,j} = G_{i,j'} = 1$. We conclude that x_j occur in at least one addition in any addition scheme for G. Given an optimal addition scheme for G, we can remove all additions of x_j to obtain an addition scheme of G', requiring at most $\alpha - 1$ additions, as we remove at least one addition by x_j in our scheme, hence $\alpha' \leq \alpha - 1$

Any optimal addition scheme for G' can be turned into an addition scheme for G requiring $\alpha' + 1$ additions, as per:

$$x_j e_i + G'x' = x_j G_{:,j} + \sum_{\substack{c=1\\c\neq j}} x_c G_{:,c} = Gx.$$

Therefore we must have $\alpha \leq \alpha' + 1$.

We conclude $\alpha = \alpha' + 1$. Thus we can compute an optimal addition scheme for G' and then translate it to an optimal addition scheme for G by adding x_j to the i-th entry of the output. □

From Lemma 4.2 we see that an optimal addition scheme for $G' \in \mathbb{F}_2^{m\times(n-1)}$, the matrix corresponding to G without column j, can be directly translated into an optimal addition scheme for G, when G has at least two nonzero entries in each row.

4.1.2 Separability

Let $I \subseteq [m]$ and $J \subseteq [n]$, and define $I^c = [m] \setminus I$ and $J^c = [n] \setminus J$. If the entries of G are such that $G_{i,j} = 0$ whenever $(i, j) \in (I \times J^c) \cup (I^c \times J)$, and I, J are non-empty proper subsets of $[m]$ and $[n]$ respectively, then we can split the instance of Problem 2.1 induced by G into two separate smaller instances of Problem 2.1, and call the instance *separable*.

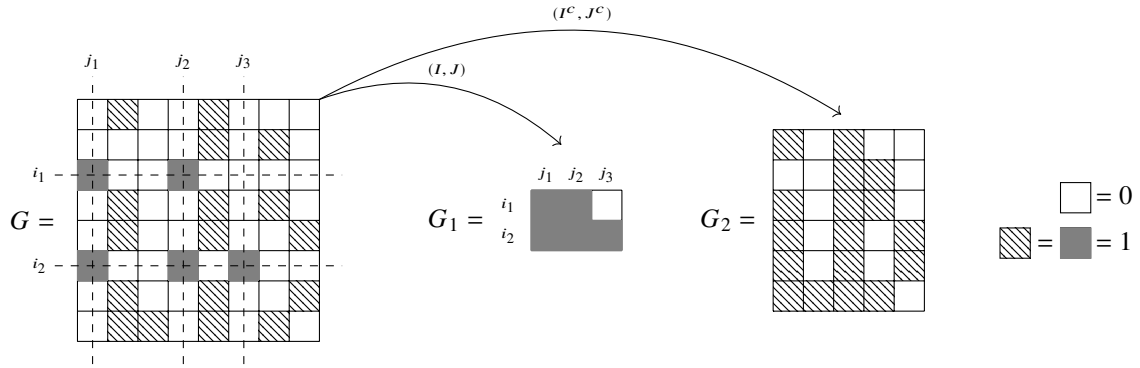


Fig. 11: Illustration of separability of matrix G into two distinct smaller matrices G_1 and G_2 from index sets $I = \{i_1, i_2\}$ and $J = \{j_1, j_2, j_3\}$.

Specifically, we can consider separately the submatrix G_1 of G indexed by I and J, and the submatrix G_2 of G indexed by I^c and J^c, see Figure 11.

We remark that separable matrices correspond exactly to block-diagonal matrices whose rows and columns are permuted, that is $G \in \mathbb{F}_2^{m \times n}$ is separable if and only if there exists permutation matrices $P \in \mathbb{F}_2^{m \times m}$, and $Q \in \mathbb{F}_2^{n \times n}$ such that:

$$G = P \begin{bmatrix} G_1 & 0 \\ 0 & G_2 \end{bmatrix} Q,$$

for some matrices $G_1 \in \mathbb{F}_2^{m_1 \times n_1}, G_2 \in \mathbb{F}_2^{(m-m_1) \times (n-n_1)}$.

Lemma 4.3 (Worst Case XOR)**.** *For all $n \in \mathbb{N}_{\geq 2}$ there exists a matrix $A_n \in \mathbb{F}_2^{n \times n}$ requiring at least $\left\lceil \frac{n^2}{4 \log_2 n} \right\rceil - n$ additions to compute.*

Proof Consider the DAG formulation of addition schemes. The number of distinct addition schemes with v nodes, S_v, satisfies

$$S_v \leq \binom{v-1}{2}^{v-n} \cdot v^n \cdot v^n \leq v^{2v}, \tag{4.1}$$

since for each of the $v - n$ non-input nodes in such a scheme there are at most $v - 1$ nodes to choose between for its two in-neighbors, for each of the n outputs we can choose any v of our nodes, and similarly for each of the n inputs we can choose between at most v of our nodes. Each addition scheme in S_v computes one of the 2^{n^2} matrices in $\mathbb{F}_2^{n \times n}$, hence the matrix with the longest optimal addition scheme requiring v' nodes, must satisfy

$$2^{n^2} \leq \sum_{u=1}^{v'} S_u \overset{(4.1)}{\leq} \sum_{u=1}^{v'} u^{2u} \leq (v')^{2v'+1}.$$

From which it follows that

$$2^{n^2} \leq (v')^{2v'+1} = 2^{(2v'+1)\log_2 v'} \iff n^2 \leq (2v'+1)\log_2 v' \implies$$
$$v' \geq \frac{n^2}{4 \log_2 n}.$$

Since the number of additions required by an addition scheme with v' nodes is $v' - n$, we have the desired existence of a matrix $A_n \in \mathbb{F}_2^{n \times n}$ requiring at least $\left\lceil \frac{n^2}{4 \log_2 n} \right\rceil - n$ additions to compute. □

The proof of Lemma 4.3 was inspired by [40, 41], we do remark that it is possible to improve the bound in Lemma 4.3, but the shown bound will suffice for this work.

Intuitively, the optimal addition scheme for a separable matrix G should require the same number of additions as the total number of additions required for G_1 and G_2. Surprisingly, it turns out that Lemma 4.3 actually implies the opposite, namely that there are separable matrices G for which we can compute the addition scheme for G using fewer total additions than what is required for the addition schemes of G_1 and G_2.

Theorem 4.4 (Separability Upper Bound)**.** *Given a separable instance of Problem 2.1 defined by the matrix $G \in \mathbb{F}_2^{m\times n}$ which is separable by non-empty proper subsets $I \subseteq [m]$ and $J \subseteq [n]$. Any addition schemes for G_1 and G_2 separately can be combined to give an upper bound on the number of additions required for the optimal addition scheme for G. The upper bound provided by G_1 and G_2 need not be tight, where G_1 is the submatrix of G indexed by (I, J), and G_2 is the submatrix of G indexed by (I^c, J^c).*

Proof Let $J = (j_1, j_2, \ldots, j_{|J|})$, and $I = (i_1, i_2, \ldots, i_{|I|})$ be ordered subsets of $[n]$ and $[m]$ respectively, and let $x \in \mathbb{F}_2^n$.

We first show how an addition scheme for G_1 requiring α_1 additions to compute, and an addition scheme for G_2 requiring α_2 additions to compute directly gives an addition scheme for G requiring $\alpha_1 + \alpha_2$ additions to compute. To that end define the output vector $y = Gx \in \mathbb{F}_2^m$, and the four projection matrices $P_J^x \in \mathbb{F}_2^{|J|\times n}$, $P_{J^c}^x \in \mathbb{F}_2^{(n-|J|)\times n}$, $P_I^y \in \mathbb{F}_2^{m\times|I|}$, and $P_{I^c}^y \in \mathbb{F}_2^{m\times(m-|I|)}$, given by:

$$(P_J^x)_{i,j} = \begin{cases} 1, & j = j_i \in J, \\ 0, & \text{otherwise} \end{cases} \qquad (P_{J^c}^x)_{i,j} = \begin{cases} 1, & j = j_i \in J^c, \\ 0, & \text{otherwise,} \end{cases}$$

$$(P_I^y)_{i,j} = \begin{cases} 1, & i = i_j \in I, \\ 0, & \text{otherwise,} \end{cases} \qquad (P_{I^c}^y)_{i,j} = \begin{cases} 1, & i = i_j \in I^c, \\ 0, & \text{otherwise,} \end{cases}$$

with i, j running over an index set matching the specified size of the projection matrix. This way we have

$$y = Gx = P_I^y G_1 P_J^x x + P_{I^c}^y G_2 P_{J^c}^x x, \tag{4.2}$$

and since each of the projections require 0 additions to compute, only the additions required to compute $G_1\left(P_J^x x\right)$ and $G_2\left(P_{J^c}^x x\right)$ count towards the total number of additions to compute Gx. This shows that any two addition schemes for G_1 and G_2 requiring α_1 and α_2 additions respectively can be used to construct an addition scheme for G requiring $\alpha_1 + \alpha_2$ additions.

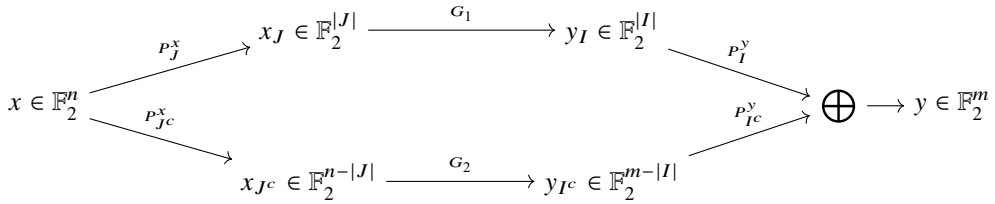


Fig. 12: Illustration of computing an addition scheme of a separable instance of Problem 2.1 induced by the matrix G, by decomposing into instances of Problem 2.1 induced by G_1 and G_2.

We remark here that the addition between $y_1 = P_I^y G_1 P_J^x x$ and $y_2 = P_{I^c}^y G_2 (P_{J^c}^x x)$ does not count towards the total number of additions required, as we know that the nonzero entries in $y_1 \in \mathbb{F}_2^m$ will definitely be zero in $y_2 \in \mathbb{F}_2^m$ and the other way around, hence the "+" in equation (4.2) amounts to simply combining two vectors by copying entries from either y_1 or y_2.

Finally, we show that the reverse inequality $\alpha \geq \alpha_1 + \alpha_2$ does not work. To that end consider a matrix $A \in \mathbb{F}_2^{k\times k}$ which requires $\Omega(k^2/\log k)$, existence of which is given by Lemma 4.3. Now define $G \in \mathbb{F}^{k^2\times k^2}$ as the block-diagonal matrix, with each diagonal entry coinciding with A:

$$G = \begin{bmatrix} A & & \\ & \ddots & \\ & & A \end{bmatrix}.$$

Then computing Gx for $x \in \mathbb{F}_2^n$ with $n = k^2$ by iteratively using the previously shown decomposition would require $\Omega(k^3/\log k)$ operations, but if we instead compute it the following way, using Strassen's algorithm for the matrix-product:

$$Gx = \textsc{BlockVec}_n\left(A \cdot \textsc{BlockMat}_k(x)\right),$$

we would compute Gx in only $O(k^{\log_2 7})$ additions, see Theorem 2.2, which is asymptotically less than $O(k^3/\log k)$, implying that for large enough k we can find an addition scheme for G that require fewer additions to compute than iteratively applying separability, hence completing the proof. □

We remark that the argument for the negative in the proof of Theorem 4.4 was derived from [42].

Most methods introduced for optimizing Problem 2.1 have running times that increase exponentially with the input size, hence optimizing the instances of Problem 2.1 induced by matrices G_1 and G_2 separately will for many methods be faster than optimizing the instance induced by G itself.

Determining whether or not a matrix $G \in \mathbb{F}_2^{m\times n}$ is separable is straightforward. First we let each nonzero entry $G_{i,j}$ denote a node in a graph, next for each nonzero entry $G_{i,j}$ we connect it with the closest nodes due north, south, east and west that correspond to another nonzero entry of G, see Figure 13. After constructing this graph, the separable parts of matrix G correspond exactly to the distinct connected components in the graph, which can be found in time proportional to the number of edges and nodes in the graph using standard graph traversal algorithms such as depth first search or breadth first search, see Figure 13.

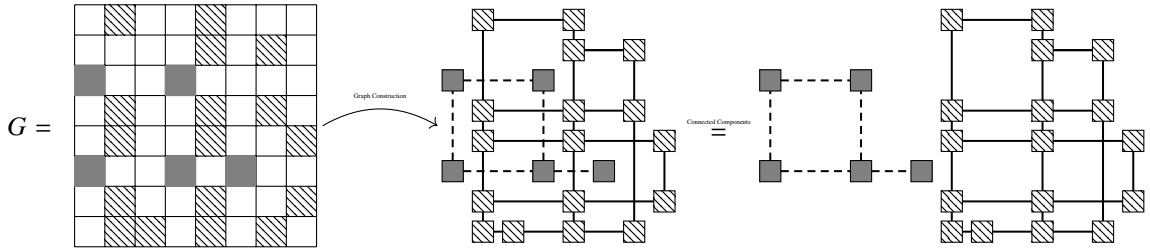


Fig. 13: Constructing the graph from the corresponding matrix and finding the connected components of the corresponding graph. Here, ■ denotes the (I, J)-indexed entries, ▨ denotes the non-(I, J) entries, and □ denotes zero entries. We remark that the example matrix G is the same used in Figure 11.

4.2 Heuristics

In this section we introduce the most well-known heuristics for solving the shortest linear program problem. Each heuristic introduced is designed to take as input the matrix $G \in \mathbb{F}_2^{m\times n}$ defining the instance of Problem 2.1 we are to solve, and outputs an addition scheme for G.

Although the heuristics described below work for all matrix sizes, in practice we work with matrices $G \in \mathbb{F}_2^{m\times n}$ with $m, n \leq 64$. This allows us to represent each column or row of G using a single unsigned 64-bit integer, which in turn can entail significant speedups of certain heuristics when compared to representing G as an $m \times n$ array. While this may seem unnecessarily restrictive, solving Problem 2.1 is a very difficult problem and most existing works already conform to this restriction [8, 39, 43].

4.2.1 Paar1 & Paar2

The Paar1 and Paar2 heuristics were proposed by Christof Paar in 1997 with the purpose of minimizing the number of XOR operations required for Reed-Solomon encoders [44]. The Paar1 and Paar2 heuristics are among the earliest proposed heuristics for solving Problem 2.1 still in use today. Their main advantage lies in their simplicity and speed with which they find solutions. We start by describing the Paar1 heuristic.

Given a set of linear expressions over $\mathbb{F}_2$, a natural heuristic for minimizing the number of additions is to greedily combine additions common to multiple linear expressions. Among all pairs of variables $\{x_i, x_j\}$, we choose the pair $\{x_i, x_j\}$ with x_i and x_j occurring simultaneously in most of the expressions at once, and compute the temporary variable $s_k = x_i + x_j$. Finally, for each linear expression in which both x_i and x_j occur we remove x_i and x_j from the sum and add s_k in their place, where $k \in \mathbb{N}$ is the smallest positive integer such that no variable s_k occurs in the linear expressions of the given instance. Computing s_k costs one addition, but replacing $x_i + x_j$ with s_k in ℓ expressions saves ℓ additions, for a net saving of $\ell - 1$ additions, where ℓ is the number of linear expressions we substitute s_k into. We obtain a new instance of Problem 2.1 this way and keep iterating the algorithm until no pair of variables occurs in more than one linear expression, then we terminate the algorithm and compute the rest of the additions directly.

Example 4.1. Consider the instance of Problem 2.1 given by the matrix in equation (2.11), that is we want to compute the outputs y_1, y_2, y_3, and y_4 in as few additions as possible, where:

$$\begin{aligned} y_1 &= x_1 + x_3 + x_5, \\ y_2 &= x_2 + x_3, \\ y_3 &= x_1 + x_3 + x_4 + x_5, \\ y_4 &= x_1 + x_2 + x_4 + x_5. \end{aligned}$$

The most frequently occurring addition pair is $\{x_1, x_5\}$, hence the first step is to compute $s_1 = x_1 + x_5$ and transform the shortest linear program we want to compute into:

$$\begin{aligned} s_1 &= x_1 + x_5, \\ y_1 &= s_1 + x_3, \\ y_2 &= x_2 + x_3, \\ y_3 &= s_1 + x_3 + x_4, \\ y_4 &= s_1 + x_2 + x_4. \end{aligned}$$

The pairs $\{s_1, x_3\}$ and $\{s_1, x_4\}$ occur with the same frequency in the new linear expressions, we choose one of them arbitrarily, let's say $\{s_1, x_3\}$, and compute $s_2 = s_1 + x_3$. Then by substitution we reduce the instance of Problem 2.1 to:

$$\begin{aligned} s_1 &= x_1 + x_5, \\ s_2 &= s_1 + x_3, \\ y_1 &= s_2, \\ y_2 &= x_2 + x_3, \\ y_3 &= s_2 + x_4, \\ y_4 &= s_1 + x_2 + x_4. \end{aligned}$$

Finally, no pair of variables occur in any of the linear expressions with frequency greater than 1, hence we are done and can count the number of additions required to be 6. We remark here that when evaluating we will compute s_i before s_j whenever $i < j$, to avoid dependency issues. Refer to Figure 14 to see the provided example in matrix form.

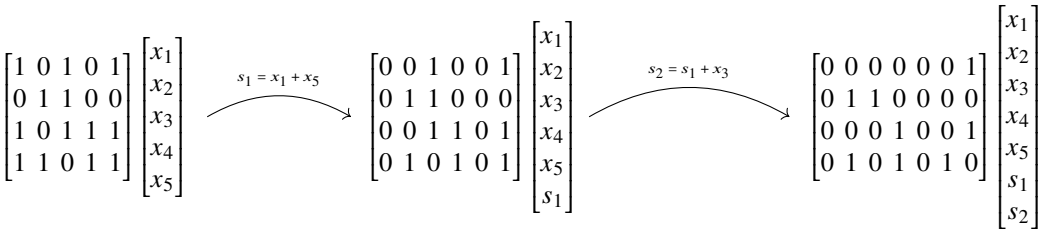


Fig. 14: Matrix form of Paar1 on the example instance of Problem 2.1.

This greedy algorithm is referred to as the *Paar1* heuristic. If instead of deterministically choosing a specific pair $\{x_i, x_j\}$ out of multiple pairs occurring equally frequently, see pairs $\{s_1, x_3\}$ and $\{s_1, x_4\}$ in the second step of the preceding example, we recursively try all of them. Using backtracking in this way is referred to as the *Paar2* heuristic [44].

4.2.1.1 $\mathbb{F}_2$ **Speedups.** From the matrix form of the Paar1 algorithm, see Figure 14, we see that each new column corresponding to the addition s_k of variables x_i and x_j is exactly the bitwise AND operation of the columns corresponding to these variables. Similarly, the updated columns corresponding to variables x_i and x_j are the XOR of the new column and the original columns corresponding to x_i and x_j, see equations (4.3)–(4.5).

Hence when $G \in \mathbb{F}_2^{m\times n}$ has $m \leq 64$, we can find the new column in a single word-level operation by storing each column of G as a 64-bit unsigned integer and performing a bitwise AND between these columns, namely:

$$G_{:,n+1} \leftarrow G_{:,i} \;\&\; G_{:,j}, \tag{4.3}$$
$$G_{:,i} \leftarrow G_{:,n+1} \oplus G_{:,i}, \tag{4.4}$$
$$G_{:,j} \leftarrow G_{:,n+1} \oplus G_{:,j}, \tag{4.5}$$

where & is the bitwise AND operator, and $\oplus$ is the bitwise XOR operator. In most modern programming languages targeting common 64-bit architectures, the bitwise operations in equations (4.3)–(4.5) can be implemented using word-level logical instructions, such as the `AND` and `XOR` instructions on Intel 64 architectures [45].

4.2.2 Greedy Potential

The greedy potential heuristic is one of the few heuristics for optimizing instances of Problem 2.1, which originates from research on minimizing the number of additions required to compute Strassen-type matrix multiplication algorithms. The heuristic was proposed by Mårtensson and Wagner in their work [25]. They illustrate the effectiveness of their proposed heuristic by reducing the number of additions required to compute a given 3×3 matrix multiplication scheme of rank-23.

The greedy potential heuristic was proposed for a variant of Problem 2.1 over the field of real numbers $\mathbb{R}$, where $G \in \{-1, 0, 1\}^{m\times n}$. As such, we first describe the greedy potential algorithm in this context, before detailing how it can be used over $\mathbb{F}_2$.

To understand the proposed heuristic we write out the setup in Problem 2.1 as follows:

$$Gx = \begin{bmatrix} g_{1,1} & g_{1,2} & \cdots & g_{1,n} \\ g_{2,1} & g_{2,2} & \cdots & g_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ g_{m,1} & g_{m,2} & \cdots & g_{m,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

Suppose $G \in \{-1, 0, 1\}^{m\times n}$, and there exist distinct column indices $j_1, j_2 \in [n]$, and a set of row indices $I \subseteq [m]$, such that either:

$$G_{i,j_1} = G_{i,j_2} \in \{-1, 1\} \quad \forall i \in I, \tag{4.6}$$

or

$$G_{i,j_1} = -G_{i,j_2} \in \{-1, 1\} \quad \forall i \in I, \tag{4.7}$$

then we can precompute the addition $s = x_{j_1} + x_{j_2}$ or $s = x_{j_1} - x_{j_2}$ respectively, and reuse it in all the rows indexed by I. To represent the use of the precomputed variable s, we augment G with a new column that has entries in $\{-1, 0, 1\}$ determined by

$$G_{i,n+1} = \begin{cases} G_{i,j_1}, & i \in I, \\ 0, & \text{otherwise}, \end{cases} \tag{4.8}$$

after which we set all entries $G_{i,j}$ where $(i, j) \in I \times \{j_1, j_2\}$ to zero. The variable corresponding to the new column of G is the precomputed sum s. It is informative to see the example provided for Paar's heuristic, as this augmentation of G is analogous to what is shown in that example, we also refer the reader to Figure 15. This transformation removes $2|I|$ nonzero entries and introduces $|I|$ new nonzero entries, so G' is sparser whenever $|I| \geq 1$. The immediate net saving in additions is $|I| - 1$, which is positive whenever $|I| \geq 2$.

Suppose for a given row-index set $I \subseteq [m]$, column indices $j_1, j_2 \in [n]$, and $\sigma_1, \sigma_2 \in \{-1, 1\}$ that

$$\sigma_1 G_{i,j_1} = \sigma_2 G_{i,j_2} \neq 0 \quad \forall i \in I, \tag{4.9}$$

and define $\sigma = \sigma_1/\sigma_2 \in \{-1, 1\}$. In equations (4.6) and (4.7) we can multiply both sides of the equality by -1 without affecting the augmentation of G induced by the row index set I and the column indices j_1, j_2, hence equation (4.9) corresponds to equation (4.6) if $\sigma = 1$, and to equation (4.7) if $\sigma = -1$. Given a pair of column

indices $j_1, j_2 \in [n]$, Mårtensson and Wagner always choose I such that it satisfies either equation (4.6) or equation (4.7), while maximizing $|I|$ [25], see Figure 15.

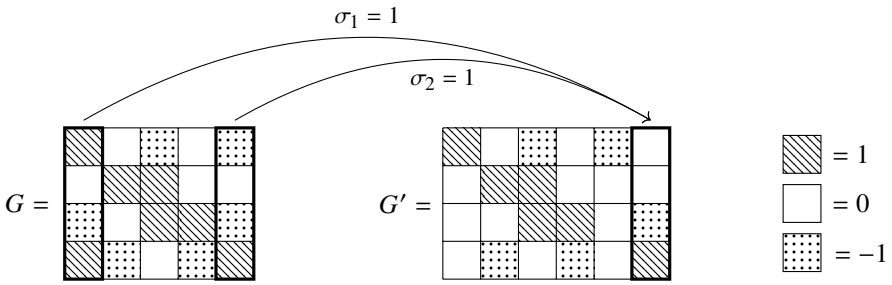


Fig. 15: Depiction of how the vector $G_{:,n+1}$ is constructed, and how the matrix G is augmented, when $\sigma = 1$ and $(j_1, j_2) = (1, 5)$.

The number of additions saved for each choice of $j_1, j_2 \in [n]$ is given by $|I| - 1$, since we remove $|I|$ additions from G, but also add the intermediate addition $s = x_{j_1} \pm x_{j_2}$. Let $\sigma \in \{-1, 1\}$, and denote by $I_\sigma(j_1, j_2)$ a row-index set of maximal cardinality such that $G_{i,j_1} = \sigma G_{i,j_2}$ for all $i \in I_\sigma(j_1, j_2)$. A natural upper-bound on the total number of additions we can save from this process can be found by considering $|I_\sigma(j_1, j_2)|$ for all pairs $\{j_1, j_2\} \subseteq [n]$, that is:

$$P(G) = \sum_{\sigma \in \{-1,1\}} \sum_{j_1=1}^{n-1} \sum_{j_2=j_1+1}^{n} \max\left(0, |I_\sigma(j_1, j_2)| - 1\right) \tag{4.10}$$

$$= \sum_{\sigma \in \{-1,1\}} \sum_{j_1=1}^{n-1} \sum_{j_2=j_1+1}^{n} \max\left(0, -1 + \sum_{k=1}^{m} \delta_{G_{k,j_1} = \sigma G_{k,j_2} \in \{-1,1\}}\right), \tag{4.11}$$

where

$$\delta_{G_{k,j_1} = \sigma G_{k,j_2} \in \{-1,1\}} = \begin{cases} 1, & G_{k,j_1} = \sigma G_{k,j_2} \wedge G_{k,j_1} \in \{-1, 1\}, \\ 0, & \text{otherwise.} \end{cases}$$

Then $P(G)$ is an upper bound on the number of additions we can save using the greedy-potential method, compared to the naive expansion of Gx. Following the convention in [25], we call $P(G)$ the potential of the matrix G.

Finally, we can state the greedy-potential algorithm based on the potential detailed in equations (4.10) and (4.11). For each step in the algorithm, we consider pairs of columns $j_1, j_2 \in [n]$, and $\sigma \in \{-1, 1\}$, and compute the following heuristic value, which we will denote as the score of the candidate augmentation:

$$\text{score} \leftarrow \alpha P(G') + \text{NNZ}(G_{:,n+1}) - 1. \tag{4.12}$$

The first term estimates future savings, while the second term is the immediate saving from the current augmentation. In equation (4.12), $\alpha \in \mathbb{R}^+$ is a hyperparameter, $G_{:,n+1}$ is the augmentation vector defined as in equation (4.8) over a set $I \subseteq [m]$ with maximal cardinality satisfying equations (4.6) and (4.7), and G' is the resulting augmented matrix. The pair of distinct column indices $j_1, j_2 \in [n]$ that induces the largest score is then applied, and G is augmented accordingly, see Figure 15. If the potential of the new augmented matrix is 0, then we terminate the algorithm, otherwise we continue with the augmented matrix as our new input G.

4.2.2.1 A Potential Difference. Existing implementations of the greedy-potential algorithm compute the score directly via equation (4.12) for all pairs of columns. Specifically, if $G \in \{-1, 0, 1\}^{m \times n}$, the current implementations carry out one iteration of the greedy-potential algorithm, converting G to G' for each pair of columns $j_1, j_2 \in \{1, 2, \ldots, n\}$ with $j_1 < j_2$, then they compute the potential $P(G')$ directly from the definition, which in the worst case has time complexity $O(mn^2)$ per candidate, and finally they revert back to G to continue with the next pair of columns.

We show that for any pair of columns j_1 and j_2, given the potential $P(G)$, we can compute the potential difference $\Delta_{j_1,j_2,\sigma}P(G) = P(G') - P(G)$, and hence $P(G')$ in $O(nm)$ time, thus we can significantly speed up the current implementations of the greedy-potential algorithm. To this end, given $G \in \{-1,0,1\}^{m\times n}$, we define the contribution $C(G,c_1,c_2)$ of the pair of distinct columns c_1, c_2 to the potential $P(G)$, as

$$C(G,c_1,c_2) = \sum_{\sigma\in\{-1,1\}} \max\left(0, -1 + \sum_{k=1}^{m} \delta_{G_{k,c_1}=\sigma G_{k,c_2}\in\{-1,1\}}\right), \tag{4.13}$$

such that

$$P(G) = \sum_{c_1=1}^{n-1}\sum_{c_2=c_1+1}^{n} C(G,c_1,c_2).$$

When constructing $G' \in \{-1,0,1\}^{m\times(n+1)}$ from $G \in \{-1,0,1\}^{m\times n}$ via the greedy-potential algorithm, using columns j_1 and j_2 with $j_1 < j_2$, the only columns that differ between matrices G and G' are columns j_1, j_2 and $n+1$, see Figure 15 for intuition. We can exploit this as follows:

$$\Delta_{j_1,j_2,\sigma}P(G) = P(G') - P(G) = \left(\sum_{\substack{c=1\\c\neq j_1}}^{n} C(G',c,j_1) + \sum_{\substack{c=1\\c\neq j_2}}^{n} C(G',c,j_2) - C(G',j_1,j_2) + \sum_{c=1}^{n} C(G',c,n+1)\right) \tag{4.14}$$

$$-\left(\sum_{\substack{c=1\\c\neq j_1}}^{n} C(G,c,j_1) + \sum_{\substack{c=1\\c\neq j_2}}^{n} C(G,c,j_2) - C(G,j_1,j_2)\right). \tag{4.15}$$

The terms $C(G,j_1,j_2)$ and $C(G',j_1,j_2)$ are subtracted because the pair (j_1,j_2) is included in both sums over columns involving j_1 and columns involving j_2. Computing $C(G,j_1,j_2)$ for any columns j_1, j_2 requires $O(m)$ operations, see equation (4.13), hence computing $\Delta_{j_1,j_2,\sigma}P(G)$ must require only $O(mn)$ operations, see equation (4.14).

4.2.2.2 $\mathbb{F}_2$ Speedups. We will consider the $\mathbb{F} = \mathbb{F}_2$ version of Problem 2.1, as is common in this type of work [7, 37, 44]. Computing the potential is by far the most time-consuming part of implementations of the greedy-potential algorithm, even when using the potential-difference method discussed in Section 4.2.2.1, see Figure 25. It turns out that by using bitwise operators, we can significantly speed up the computation of $\Delta_{j_1,j_2,\sigma}P(G)$ for all pairs of columns $j_1 < j_2$, and all $G \in \mathbb{F}_2^{m\times n}$, see Figure 27.

The first important speedup we immediately gain from considering the $\mathbb{F}_2$ case, is that -1 is the same as 1, and hence we only need to concern ourselves with $\sigma = 1$ when computing equation (4.13). In $\mathbb{F}_2$ the G matrix will have binary entries and hence the inner sum in equation (4.13)

$$\sum_{k=1}^{m} \delta_{G_{k,c_1}=G_{k,c_2}\wedge G_{k,c_1}\in\{-1,1\}}, \tag{4.16}$$

is simply the number of set bits in the result of the bitwise AND operation applied to the two columns represented by indices c_1 and c_2. By representing the columns of $G \in \{0,1\}^{m\times n}$ as 64-bit unsigned integers, which is only possible if $m \leq 64$, we can compute the bitwise AND of two column representations using a single word-level CPU instruction, denoted by:

$$G_{:,c_1} \& G_{:,c_2}.$$

Similarly, the number of set bits, i.e. the *Hamming weight*, of a 64-bit unsigned integer g can typically be computed using a single word-level instruction. We denote this quantity by

$$H(g). \tag{4.17}$$

Hence for m small enough, we can compute the sum in equation (4.16) using only 2 word-level CPU instructions, namely over $\mathbb{F}_2$ we have:

$$H\left(G_{:,c_1} \;\&\; G_{:,c_2}\right) = \sum_{k=1}^{m} \delta_{G_{k,c_1}=G_{k,c_2} \wedge G_{k,c_1}}.$$

In contrast, direct implementations over the set of ternary $-1, 0, 1$ matrices, such as the direct method shown in Algorithm 3, require $O(m)$ operations.

Algorithm 3 Naive row-wise computation of the matching-pairs contribution.

Require: $G \in \{-1, 0, 1\}^{m \times n}$, columns c_1, c_2
1: $s \leftarrow 0$
2: **for** $k = 1, \ldots, m$ **do**
3: **if** $G_{k,c_1} = G_{k,c_2}$ **and** $G_{k,c_1} \neq 0$ **then**
4: $s \leftarrow s + 1$
5: **end if**
6: **end for**
7: **return** s

The effects of these speedups are shown in Figure 27.

4.2.3 Boyar Peralta

One of the most used heuristics, from which many of the heuristics we will introduce later on are derived, is called the Boyar-Peralta (BP) heuristic [37, 39, 46]. The first step of the BP heuristic is to construct the set of target vectors $T = \{g_1, \ldots, g_m\} \setminus \{0\} \subseteq \mathbb{F}^n$ from rows of the given matrix $G \in \mathbb{F}^{m \times n}$, see Figure 16.

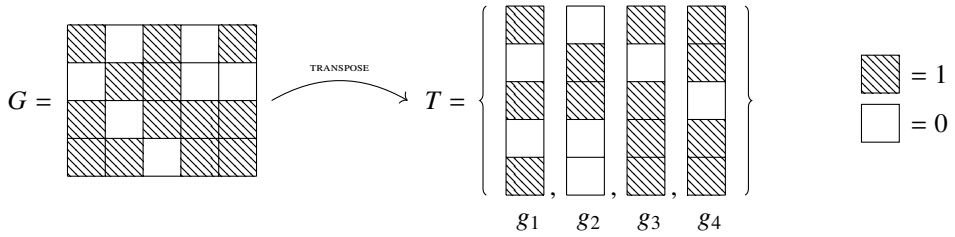


Fig. 16: Illustration of how the set of target vectors described in Problem 2.1 is constructed. In this example, we work over $\mathbb{F}_2$, in general the ▨ - entries would simply indicate nonzero entries.

Given the set of target vectors T corresponding to the input matrix G, the BP heuristic sets up the ordered set B, which in the BP heuristic is initialized as the ordered set of standard basis vectors $B = E$, with E defined as in the definition of Problem 2.1. For each target vector $g_i \in T$, we compute the minimal number of nonzero elements in B required to form g_i from vectors in B i.e. the minimum number of nonzero scalars $x_i \in \mathbb{F}$ such that

$$g_i = \sum_{j=1}^{|B|} x_j b_j.$$

We define a distance vector $D \in \mathbb{N}_0^m$, such that entry i of D is given by

$$D_i = \min_{x \in \mathbb{F}^{|B|}} \left\{ \max\left(0, \mathrm{NNZ}(x) - 1\right) \mid g_i = \sum_{j=1}^{|B|} x_j b_j \right\}.$$

Then for each possible vector $b_{|B|+1}$ we can create in a single addition from the vectors in B, that is for an element $b_{|B|+1} \in \{b_p + b_q | 1 \leq p < q \leq |B|\}$, we compute a new distance vector D' using the ordered set $B \cup \{b_{|B|+1}\}$ as our ordered set B, see Figure 16.

The vector $b_{|B|+1} = b_p + b_q$ which gives rise to the distance vector D' with the smallest ℓ_1-norm is then added to our B set i.e. $B \leftarrow B \cup \{b_p + b_q\}$. If multiple vectors result in the same value for $||D'||_1$, then we pick the $b_{|B|+1} = b_p + b_q$ which gives the largest ℓ_2-norm i.e. maximizes $||D'||_2$, the idea being that if two distance vectors have the same ℓ_1 norm, but different ℓ_2 norms, then the higher ℓ_2 norm have less uniformly distributed elements and hence the upcoming steps is more likely to quickly yield elements of T more quickly. Moreover, later research shows experimentally that prioritizing elements $b_{|B|+1} = b_p + b_q$, which focus on reducing the smaller elements in D tend to do better, when the ℓ_1 norm is fixed [39].

The BP heuristic iteratively performs this process of picking new elements $b_{|B|+1} = b_p + b_q$ based on minimizing the resulting ℓ_1-norm distance vector D' and tie-breaking based on maximizing the ℓ_2-norm of the resulting distance vector D'. In case of ties on both the ℓ_1 norm and the ℓ_2 norm we choose the first element $b_{|B|+1} = b_p + b_q$ giving us the optimal D'.

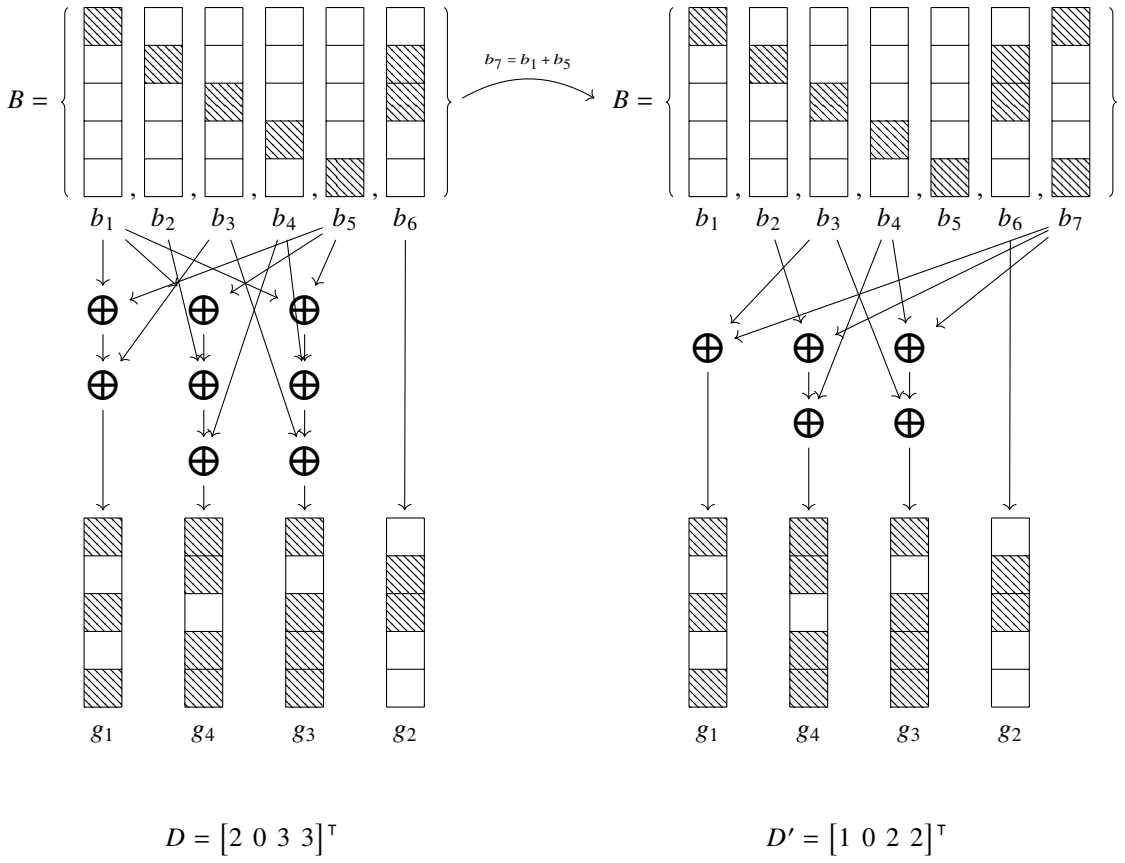


Fig. 17: We continue the example from Figure 16, working over $\mathbb{F}_2^5$, and show the effect of adding a new element $b_{|B|+1}$ to B on the distance vector D. The illustration shows the second step of the BP heuristic, the first step added $b_6 = b_2 + b_3$ to our set B. Each target vector is calculated independently, and each distance entry D_i corresponds exactly to the number of $\oplus$ above it. Note that the target vectors $g_1, ..., g_4$ are not ordered according to their index in the illustration, this is simply to avoid too much clutter.

The only exception to this iterative process occurs when a target vector has distance $D_k = 1$. In this case, there exist indices $1 \leq p < q \leq |B|$ such that $g_k \notin B$ and $g_k = b_p + b_q$. When this is the case we greedily add this vector to our set B.

4.2.3.1 Computing the Distance Vector.

We work over $\mathbb{F}_2$, hence using the same vector more than once is never necessary, as $v + v = 0$ for all $v \in \mathbb{F}_2^n$.

Lemma 4.5 (Distance Vector Change)**.** *If it requires at least D_i additions of elements in a set $B \subseteq \mathbb{F}_2^n$ to compute a vector $g_i \in \mathbb{F}_2^n$, then for any two vectors $b_p, b_q \in B$ it requires either at least $D_i - 1$ additions or at least D_i additions of elements in the set $B \cup \{b_p + b_q\}$ to compute g_i.*

Proof Suppose $D_i - h$ additions are required to compute g_i from elements in the set $B \cup \{b_p + b_q\}$, where $h \in \mathbb{N}_0$. Denote $b_{|B|+1} = b_p + b_q$, then we can write

$$g_i = \sum_{j=1}^{D_i-h+1} b_{k_j},$$

for some index tuple $K = (k_1, k_2, \ldots, k_{D_i-h+1}) \subseteq [|B| + 1]$. If $|B| + 1 \notin K$, then we can also construct g_i using $D_i - h$ additions with elements in B using exactly the same sum, hence making $h = 0$. If, on the other hand, $|B| + 1 \in K$, then we can without loss of generality assume that $k_{D_i-h+1} = |B| + 1$. It follows that:

$$g_i = \sum_{j=1}^{D_i-h+1} b_{k_j} = b_p + b_q + \sum_{j=1}^{D_i-h} b_{k_j},$$

which is a sum of $D_i - h + 1$ additions of elements in B implying $h \leq 1$, which completes the proof. □

Let $b_p, b_q \in B$, a consequence of Lemma 4.5 is that we can reduce finding D'_i to simply checking whether it is possible to compute g_i in $D_i - 1$ additions of elements in $B \cup \{b_p + b_q\}$. If this is possible, then $D'_i = D_i - 1$, otherwise $D'_i = D_i$.

Brute Force Search. To determine whether it is possible to reach g_i using $D_i - 1$ additions of elements in $B \cup \{b_p + b_q\} \subseteq \mathbb{F}_2^n$, it is equivalent to ask whether g_i can be written as a sum of at most D_i vectors from this enlarged set. Almost all existing work searches the space of all possible $D_i - 1$ additions blindly using a strategy akin to the one illustrated in Algorithm 4.

Algorithm 4 Standard recursive algorithm for searching the space of ways to add $d \in \mathbb{N}_0$ vectors in $B \subseteq \mathbb{F}_2^n$ together to obtain a target vector $t \in \mathbb{F}_2^n$.

```
1:  function REACHABLE(t ∈ F_2^n, d ∈ N_0, B ⊆ F_2^n, INDEX = |B|)
2:      if d = 0 ∨ INDEX = 0 then
3:          return t = 0                          ▷ TRUE if t is 0, FALSE otherwise
4:      end if
5:      TEMP ← REACHABLE(t + B[INDEX], d − 1, B, INDEX − 1)        ▷ Use B[INDEX]
6:      if TEMP = TRUE then
7:          return TRUE
8:      end if
9:      return REACHABLE(t, d, B, INDEX − 1)                      ▷ Don't use B[INDEX]
10: end function
```

While Algorithm 4 is very simple, it can easily lead to a combinatorial explosion of the search space, making the REACHABLE function infeasible to compute. Many resources which build upon the BP heuristic, state that the main limiting factor of the BP heuristic, and heuristics derived from it, is that it is very time consuming to compute the new distance vector D', when adding the vector $b_{|B|+1} = b_p + b_q$ to the set B [37, 39, 46].

Sparsity-Aware Backtracking. The decision problem underlying REACHABLE is a form of syndrome decoding over $\mathbb{F}_2$, which is NP-complete in general [47, 48]. However, we have some structure in the ordered set $B \subseteq \mathbb{F}_2^n$ which we can use to our advantage. Specifically, most of the data in our datasets consists of relatively sparse matrices $G \in \mathbb{F}_2^{m \times n}$, see Section 3, and when generating B, we start with $B = E$. Thus initially the collection of vectors $B = E$ has a density of only $\rho = \frac{1}{n}$.

Given a target vector $t \in \mathbb{F}_2^n$, a set $B \subseteq \mathbb{F}_2^n$ and a target distance $d \in \mathbb{N}$, if the i-th entry of the target vector, t_i, is 1, then we know that we need an odd number of vectors $b \in B$ with $b_i = 1$, to form t. Let $Y_i \subseteq B$ be the set of vectors $b \in B$ with $b_i = 1$. We search over every odd-sized subset $X \subseteq Y_i$, to find whether t_{new} is reachable using at most $d - |X|$ elements from $B \setminus X$, where t_{new} is given by:

$$t_{new} = t + \sum_{b \in X} b,$$

and if this is the case we return that it is possible to reach t in $d - 1$ additions over elements in B, otherwise we return that it is not possible. This search method may in the worst case examine $O(2^{|Y_i|})$ subsets, hence we should heuristically choose the index i with $t_i = 1$ according to how small the resulting $|Y_i|$ becomes. We denote this search method as *sparsity-aware backtracking*, see Algorithm 5.

Algorithm 5 Sparsity-aware backtracking algorithm for finding whether $t \in \mathbb{F}_2^n$ can be constructed as a sum of at most d elements in $B \subseteq \mathbb{F}_2^n$.

function REACHABLE($t \in \mathbb{F}_2^n$, $d \in \mathbb{N}_0$, $B \subseteq \mathbb{F}_2^n$, $n \in \mathbb{N}$)
  if $d \leq 0$ **then**
    return $t = 0 \wedge d = 0$ ▷ TRUE if t and d are both 0, FALSE otherwise
  end if
  $Y_{best} \leftarrow B$
  for i=1:n **do**
    if $t_i = 1$ **then**
      $Y \leftarrow \emptyset$
      for all $b \in B$ **do**
        if $b_i = 1$ **then**
          $Y \leftarrow Y \cup \{b\}$
        end if
      end for
      if $Y = \emptyset$ **then**
        return FALSE ▷ No elements in B can form a 1 at index i
      end if
      if $|Y| < |Y_{best}|$ **then**
        $Y_{best} \leftarrow Y$
      end if
    end if
  end for
  for all $X \subseteq Y_{best} : |X| \equiv 1 \pmod 2 \wedge |X| \leq d$ **do**
    $t_{new} \leftarrow t + \sum_{b \in X} b$
    if REACHABLE($t_{new}, d - |X|, B \setminus X, n$) **then**
      return TRUE
    end if
  end for
  return FALSE ▷ No way to reach t using at most d elements of B
end function

We remark that Algorithm 5, as written, assuming $t \notin B$, returns whether t is reachable in d terms, and hence $\max(0, d-1)$ additions. This is due to there being $d-1$ additions in a sum

$$\sum_{i=1}^{d} b_i$$

of d individual terms. In practice we handle this by always adding $b_{|B|+1}$ to t before passing it as input to REACHABLE. Adding d entries of B to form t in $d-1$ additions will always involve $b_{|B|+1}$ if it is possible, as without $b_{|B|+1}$ we already knew that it is reachable in $d+1$ additions and not reachable in d additions.

MITM. Both the brute force method and sparsity aware backtracking, see Algorithms 4 and 5, have worst case time complexities of $O(2^{|B|})$, and the problem is as previously mentioned a classic NP-hard problem, so we cannot hope to find a polynomial time algorithm unless $P = NP$. We introduce here a so called meet-in-the-middle (MITM) algorithm with worst case time complexity $O\left(n|B|\sqrt{2^{|B|}}\right)$, hence significantly improving the asymptotic worst case performance as compared to the brute force and sparsity aware backtracking methods.

Assume for simplicity that $|B|$ is even, the idea of the MITM algorithm is to first partition $B \subseteq \mathbb{F}_2^n$ into two distinct subsets B_1 and B_2 with $|B_1| = |B_2|$. Suppose a target vector $t = g_i$ is reachable in $d = D_i - 1$ additions of elements in B:

$$t = b_1 \oplus b_2 \oplus \cdots \oplus b_{d+1},$$

for some $b_1, \ldots, b_{d+1} \in B$. Partition the sum into those b's that are in B_1, and those in B_2 i.e.

$$t = \left(b_1^{(1)} \oplus \cdots \oplus b_{d_1+1}^{(1)}\right) \oplus \left(b_1^{(2)} \oplus \cdots \oplus b_{d_2+1}^{(2)}\right),$$

where

$$\{b_1^{(1)}, \ldots, b_{d_1+1}^{(1)}\} \subseteq B_1 \cap \{b_1, b_2, \ldots, b_{d+1}\},$$
$$\{b_1^{(2)}, \ldots, b_{d_2+1}^{(2)}\} \subseteq B_2 \cap \{b_1, b_2, \ldots, b_{d+1}\},$$
$$\{b_1^{(1)}, \ldots, b_{d_1+1}^{(1)}\} \cup \{b_1^{(2)}, \ldots, b_{d_2+1}^{(2)}\} = \{b_1, b_2, \ldots, b_{d+1}\},$$

with $d_1 + d_2 + 1 = d$. Now, we can compute the set of all subset sums of B_1, $\mathcal{S}$, that is $b \in \mathcal{S}$ if and only if there exists $U \subseteq B_1$ with $b = \sum_{u \in U} u$. Computing $\mathcal{S}$ requires $O(n|B_1|2^{|B_1|})$ operations, see Algorithm 6. Group the elements of $\mathcal{S}$ into sets $\mathcal{S}^{(0)}, \mathcal{S}^{(1)}, \ldots, \mathcal{S}^{(|B_1|)}$, where $\mathcal{S}^{(i)}$ is the set of subset sums that were formed using $i - 1$ additions over i distinct vectors. If $d + 1 \leq |B_1|$ and $t \in \mathcal{S}^{(d+1)}$ we are done and can conclude that t is reachable in d additions.

Finally, for each subset sum of $j - 1$ additions over j distinct elements in B_2, let the evaluation of the sum be denoted by b. Check if that sum evaluates to t and $j = d + 1$, in which case we are done and can conclude that t is reachable in d additions, otherwise check if $0 \leq d + 1 - j \leq |B_1|$, and $t \oplus b \in \mathcal{S}^{d+1-j}$, in which case we can also conclude t reachable in d additions. After checking all subset sums of B_2, and none of them imply reachability of t in d additions, we conclude the negative, namely that it is not possible to reach t in d additions of elements in B.

As previously stated, computing all of the sets $\mathcal{S}^{(i)}$ require a total of $O(n|B_1|2^{|B_1|})$ operations. Similarly the number of operations required to compute each subset sum of B_2 is $O(n|B_2|2^{|B_2|})$, each of which does a lookup in a set $\mathcal{S}^{(i)}$, which in practice can be accomplished in $O(1)$ time using a hash set. Hence the total number of operations required is

$$O\left(n|B_1|2^{|B_1|}\right) + O\left(n|B_2|2^{|B_2|}\right) \cdot O\left(1\right) = O\left(n|B|\sqrt{2^{|B|}}\right),$$

using $|B_1| = |B_2| = |B|/2$ and the fact that vector addition in $\mathbb{F}_2^n$ require $O(n)$ operations. The full algorithm is illustrated in Algorithm 6.

Algorithm 6 Meet-in-the-middle algorithm for determining reachability of a target vector $t \in \mathbb{F}_2^n$ in $d \in \mathbb{N}_0$ additions of elements in a given set of vectors B.

function REACHABLE($t \in \mathbb{F}_2^n$, $d \in \mathbb{N}$, $B \subseteq \mathbb{F}_2^n$, $n \in \mathbb{N}$)
  Choose $B_1, B_2 \subseteq B$ such that $B_1 \cap B_2 = \emptyset \wedge B_1 \cup B_2 = B \wedge |B_1| = \lfloor |B|/2 \rfloor$
  $\mathcal{S}^{(i)} \leftarrow \emptyset \quad \forall i \in [|B_1|]$ ▷ Hash sets
  $\mathcal{S}^{(0)} \leftarrow \{0\}$
  $V_1 \leftarrow [(0, 0)]$ ▷ Dynamic array
  for all $b \in B_1$ **do**
    $V' \leftarrow V_1$ ▷ Deep copy
    for all $b_v, d_v \in V_1$ **do**
      PUSH(V', $(b_v \oplus b, d_v + 1)$)
      $\mathcal{S}^{(d_v+1)} \leftarrow \mathcal{S}^{(d_v+1)} \cup \{b_v \oplus b\}$
    end for
    $V_1 \leftarrow V'$
  end for
  $V_2 \leftarrow [(0, 0)]$ ▷ Dynamic array
  for all $b \in B_2$ **do**
    $V' \leftarrow V_2$ ▷ Deep copy
    for all $b_v, d_v \in V_2$ **do**
      PUSH(V', $(b_v \oplus b, d_v + 1)$)
    end for
    $V_2 \leftarrow V'$
  end for
  for all $b_v, d_v \in V_2$ **do**
    if $0 \leq d + 1 - d_v \leq |B_1| \wedge (t \oplus b_v \in \mathcal{S}^{(d-d_v+1)})$ **then**
      return TRUE
    end if
  end for
  return FALSE
end function

In Algorithms 4 and 5 we can get lucky and quickly terminate after finding a way to form the target vector $t \in \mathbb{F}_2^n$ using d additions, but the MITM algorithm does not have this property. Another thing to keep in mind is that the MITM algorithm trades time for space, and requires $O\left(\sqrt{2^{|B|}}\right)$ machine words of space when $n \leq 64$.

We consider the dataset of Bernoulli random matrices, see Section 3.4, and apply each of the Algorithms 4–6 to the dataset. Mapping out what the optimal REACHABLE method is for each pair of matrix size n and bias ρ, see Figure 18.

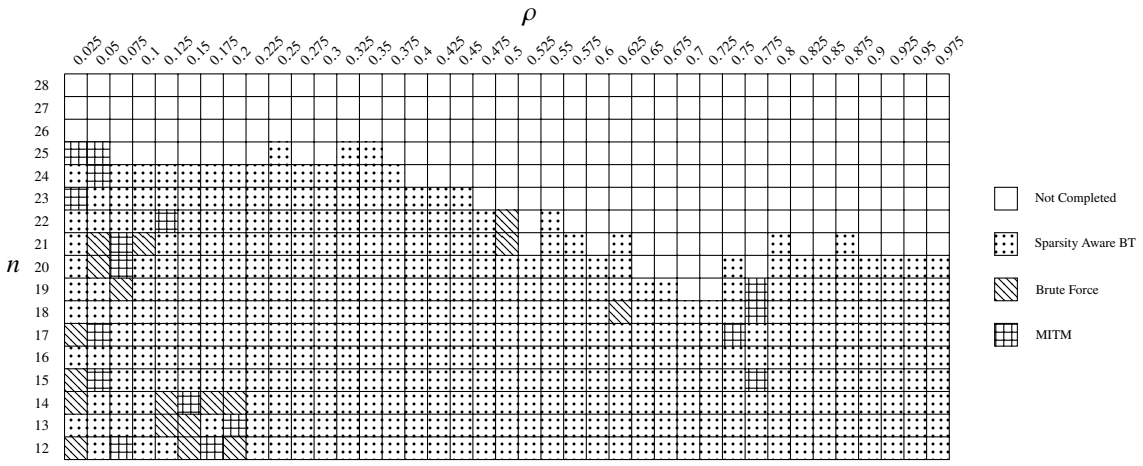


Fig. 18: Map over the best performing REACHABLE strategy out of Algorithms 4–6 used in the Boyar-Peralta heuristic on the Bernoulli random matrices from the dataset described in Section 3.4. All instances were run on a single core, on a XeonE5_2660v3 CPU, and were terminated if it required more than 3 hours to complete. If no method terminated within the timelimit we mark the square as *Not Completed*.

From Figures 18 and 28 we infer that for some instances of Problem 2.1 the brute force, or meet in the middle approaches are better suited, but by far the majority of instances are computed the fastest by the sparsity-aware backtracking REACHABLE method. The state-of-the-art efficiency of our newly proposed sparsity-aware backtracking REACHABLE method is an interesting result in and of itself. It also gives weight to our decision of choosing sparsity-aware backtracking as our default REACHABLE method.

4.2.4 Random-Normal-BP (RNBP)

The BP heuristic is deterministic, as all ties on both the ℓ_1 and ℓ_2 norms of D' are broken by simply taking the first new element $b_{|B|+1}$ giving rise to an optimal D' vector. Since there might be many distinct candidate vectors $b_{|B|+1}$ which give rise to an optimal distance vector D' it stands to reason that the BP heuristic might not pick the optimal such entry. While this could be solved using an exhaustive backtrack search over the BP heuristic, this can quickly become unwieldy for even modestly sized instances of Problem 2.1, therefore the work [39] propose selecting between all optimal new candidates for $b_{|B|+1}$ with equal probability. This way the algorithm can be run multiple times with different seed values, and each run may outperform previous runs, making it possible to optimize an instance of Problem 2.1 even further as compared to simply running a deterministic heuristic once.

4.2.5 A1 & A2

The A1 and A2 heuristics are based on the RNBP heuristic in that they also iteratively build the ordered set B, and both A1 and A2 still use the distance vector and its ℓ_1 norm in determining which vector to add to B. Both heuristics A1 and A2 were proposed in [39], and what differentiates these two heuristics from RNBP is the addition of a filtering step which ignores all possible new vectors $b_{|B|+1}$ which do not reduce the distance to one of the nearest targets. Specifically, we require $b_{|B|+1}$ to be such that

$$\exists i \in [m] : D'_i < D_i \wedge i \in \underset{\substack{j \in [m] \\ D_j > 0}}{\arg\min} D_j.$$

Algorithm A1 first filters the candidates as above, then minimizes $|D'|_1$, and finally breaks ties by maximizing $|D'|_2$. If multiple candidates still remain, one is chosen uniformly at random.

Algorithm A2 uses the same filter and also minimizes $|D'|_1$, but then chooses uniformly at random among the remaining candidates. In particular, A2 does not use the ℓ_2 norm of D' as a tie-breaker.

The filtering step can make this method much faster than the RNBP heuristic as it significantly reduces the number of candidates that have to be evaluated, see Figures 29–32.

4.2.6 MIP

As illustrated in Figure 2, we can represent Problem 2.1 as the problem of finding a suitable edge-weighted DAG. Since we work over $\mathbb{F}_2$, the only weight which can contribute to our scheme is 1, hence we can consider Problem 2.1 with $\mathbb{F} = \mathbb{F}_2$ as an unweighted DAG identification problem. Specifically, we can set up $2^n - 1$ nodes, each node corresponding to a distinct nonzero vector $v \in \mathbb{F}_2^n$. For each set of three distinct nodes $v_i \in \mathbb{F}_2^n, v_j \in \mathbb{F}_2^n, v_k \in \mathbb{F}_2^n$ such that $v_k = v_i + v_j$ we add an edge between the pair of nodes $\{v_i, v_j\}, \{v_i, v_k\}$, and $\{v_j, v_k\}$. Let $a, b, c \in \{i, j, k\}$ be distinct, for each pair of edges $\{\{v_a, v_b\}, \{v_a, v_c\}\}$ associate a binary variable t_{v_a, v_b, v_c}, which is 1 if the addition $v_\alpha = v_\beta + v_\gamma$ is part of the optimal addition scheme we are constructing for the given instance of Problem 2.1, and 0 otherwise.

In this graph we mark the input nodes, i.e. those nodes $x \in \mathbb{F}_2^n$ that have Hamming weight $H(v) = 1$, and output nodes $y \in \mathbb{F}_2^n$, i.e. the rows in the input matrix $G \in \mathbb{F}_2^{m \times n}$, see the definition of T in Problem 2.1. Associate with each node $v_k \in \mathbb{F}_2^n \setminus \{0\}$ a binary variable $s_{v_k} \in \{0, 1\}$, which is 0 if there is no edge-pair variable $t_{k,i,j}$ with $t_{k,i,j} = 1$. The binary variable s_{v_k} signifies whether the variable v_k is constructed as an intermediate step for the addition scheme, or not, as such we also require $t_{k,i,j} = 0$ whenever $s_{v_i} = 0$ or $s_{v_j} = 0$ for all non-input nodes $v_k \in \mathbb{F}_2^n$.

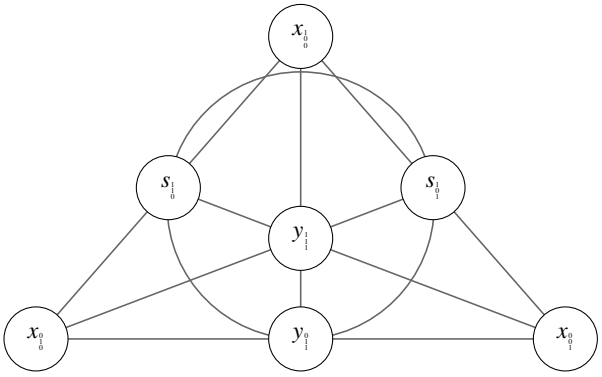


Fig. 19: Representation of the union over all possible addition schemes in DAG formulation, such that there are 3 inputs x_1, x_2, and x_3, and two outputs, namely $y_1 = x_1 + x_2 + x_3$, and $y_2 = x_2 + x_3$. Any two nodes on the same line or circle XOR to the third.

The objective of Problem 2.1, namely minimizing the number of additions, can thus be written as:

$$\min \sum_{\substack{0<\alpha<\beta \\ H(\alpha\oplus\beta)>1}} t_{\alpha\oplus\beta,\alpha,\beta}, \tag{4.18}$$

where $\oplus$ is the exclusive-or operator, $\alpha < \beta$ if and only if $\alpha \neq \beta$, and the first index $i \in [n]$, where $\alpha_i \neq \beta_i$ has $\alpha_i = 0$, and the Hamming weight function $H : \mathbb{F}_2^n \to \mathbb{N}_0$ is as defined in equation (4.17).

The minimization objective equation (4.18) is subject to the constraint that we want each output element y constructed, hence we want $s_y = 1$ for each y in the set of target vectors. Define the set

$$A = \{(\gamma, \alpha, \beta) \in \mathbb{F}_2^n \times \mathbb{F}_2^n \times \mathbb{F}_2^n \mid 0 < \alpha < \beta \wedge \gamma = \alpha \oplus \beta \wedge H(\gamma) > 1\},$$

With this notation, we can express the final binary integer programming problem as:

$$\text{Obj:} \quad \min \sum_{(\gamma,\alpha,\beta)\in A} t_{\gamma,\alpha,\beta}$$

$$
\begin{aligned}
\text{Constraints:} \quad & s_{v_\gamma} = \sum_{\substack{(\gamma,\alpha,\beta)\in A \\ \gamma=\gamma'}} t_{\gamma,\alpha,\beta} && \forall \gamma' \in \{\gamma \in \mathbb{F}_2^n | H(\gamma) > 1\} \\
& t_{\gamma,\alpha,\beta} \leq s_{v_\alpha} && \forall (\gamma,\alpha,\beta) \in A \\
& t_{\gamma,\alpha,\beta} \leq s_{v_\beta} && \forall (\gamma,\alpha,\beta) \in A \\
& s_x = 1 && \forall x \in \{x \in \mathbb{F}_2^n : H(x) = 1\} \\
& s_y = 1 && \forall y \in T \\
& t_{\gamma,\alpha,\beta} \in \{0,1\} && \forall (\gamma,\alpha,\beta) \in A \\
& s_{v_\alpha} \in \{0,1\} && \forall \alpha \in \mathbb{F}_2^n \setminus \{0\}
\end{aligned}
$$

with T as defined in Problem 2.1. We remark here that whenever any input $x \in \mathbb{F}_2^n$ coincides with an output $y \in T$ we can simply remove that output y from T before setting up the problem, as we are already trivially given this output for free in this case.

4.3 Postprocessing

A lot of work has been done coming up with heuristics to solve Problem 2.1 [8, 25, 37, 39, 44], whereas comparatively less work has been done on performing local optimization over a given addition scheme [8, 39]. This section describes, to the best of our knowledge, all postprocessing methods that has been developed for optimizing addition schemes. All the postprocessing methods described here are designed for $\mathbb{F}_2$.

Example 4.2. Let there be given an addition scheme $B \subseteq \mathbb{F}_2^n$ for the matrix $G \in \mathbb{F}_2^{m\times n}$, which has $s_1, s_2, s_3, x_1, x_2, x_3 \in B$, and

$$
\begin{aligned}
s_1 &= x_1 + x_2, \\
s_2 &= s_1 + x_3, \\
s_3 &= x_1 + x_3,
\end{aligned}
$$

with no element $x \in B \setminus \{s_2\}$ having $x = s_1 + y$ for some $y \in B$, and also s_1 is not a target vector, that is, $s_1 \notin T$. Since s_1 is an intermediate variable used only in forming $s_2 \in \mathbb{F}_2^n$, we can reorder the computation and eliminate s_1 in the process by instead computing:

$$
\begin{aligned}
s_3 &= x_1 + x_3, \\
s_2 &= s_3 + x_2,
\end{aligned}
$$

saving one addition, namely $s_1 = x_1 + x_2$. This postprocessing method, called the *swapping orders* method, is an adaptation of one of the postprocessing methods introduced by Tan and Peyrin in their work [39].

4.3.1 Fixed rules derived from exhaustive search

Tan and Peyrin introduce a local postprocessing optimization framework in [39]. Given an addition scheme $B \subseteq \mathbb{F}_2^n$ for the matrix $G \in \mathbb{F}_2^{m\times n}$, they choose an arbitrary element $r \in B$ as the 'root' and an integer $\ell \in \mathbb{N}$ as the max leaf count. Reverse all the edges in the DAG-formulation of the addition scheme, then perform a breadth first search from the root r, adding all elements traversed to $T_{\ell,r}$. We stop the search when we reach ℓ leaf nodes, or the graph with root r has been fully traversed.

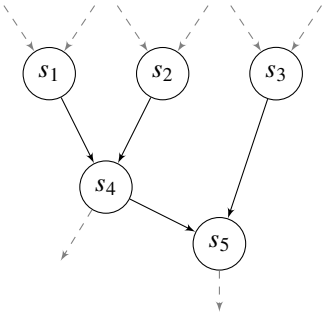


Fig. 20: Illustration of the construction of the subgraph with root s_5 and $\ell = 3$ according to the breadth-first search described. The dashed gray edges show that this is possibly part of a larger addition scheme, but the breadth first search only considers s_1, s_2, s_3, s_4, and s_5.

In the work [39] they set $\ell = 5$, then they perform an exhaustive search over all permutations of the leaves and pick the permutation that minimizes the number of additions in the subgraph without affecting the overall correctness of the addition scheme, see Figure 21. Specifically, if an element b in the subgraph is in the set of target vectors T, or is used outside the subgraph, then it cannot be changed in any non-trivial manner without possibly affecting correctness of the whole addition scheme.

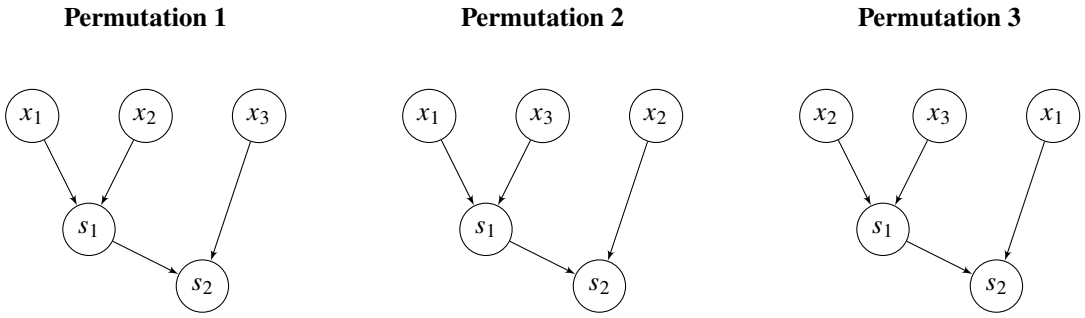


Fig. 21: Example of permutations of processing subgraph from Figure 20 to reduce the number of additions required to compute the addition scheme. Note that the variable s_2 is computed the same way, independent of the way the leaves are permuted, so if one of $s_1 = x_1 + x_2$, $s_1 = x_1 + x_3$, and $s_1 = x_2 + x_3$ is a target vector, or is used elsewhere in the addition scheme, then the right permutation of the leaves might save us an addition. This illustration was inspired by Figure 1 in [39].

In the subsequent work [8], they search over the set of all subgraphs that could arise with $\ell = 5$ and explicitly write out rules that can reduce the number of additions in the addition scheme by permuting the leaves in these subgraphs, and also write out the conditions the subgraphs should satisfy for these reduction rules to apply. This way they construct 11 reduction rules similar in form to the swapping orders method described in the example above. We remark that these 11 reduction rules are not complete and some are redundant, as was shown in [43].

4.3.2 Dynamic reduction rules

Building on [8], Yuan et al. introduce an improvement to the idea of using a fixed set of reduction rules, by using state of the art symmetry-detection tools for dynamically finding reduction rules based on the given addition scheme [43]. The proposed dynamic reduction rule method was designed for schemes using only in-place operations, i.e., all additions are of the form $x_i = x_i + x_j$ for some $i, j \in [n]$. We propose a different dynamic postprocessing algorithm for reducing the number of additions in a specified addition scheme. While the method we propose is not as versatile and extensive as the one proposed in [43], it works for addition schemes where the additions are not in-place, i.e. the addition schemes we work over are of the form $s = x_j + x_i$ with $s \notin \{x_i, x_j\}$. Moreover, our proposed postprocessing method is significantly easier to implement than both the fixed rule based method proposed in [8], and the in-place dynamic method proposed in [43].

Assuming that the elements of B are topologically sorted, according to the DAG-representation of B, the postprocessing method we propose is to first write down the set of all intermediate sums, that is $S = B \setminus (E \cup T)$, with E and T as in Problem 2.1. We go through each intermediate sum $s \in S$ and tentatively remove it. For each removed intermediate sum s, if we can still form all elements $b \in B \setminus \{s\}$ that are the out-neighbors of s in the DAG representation of the addition scheme B using only elements of the addition scheme that do not depend on s, then we do so, see Figure 22. If, on the other hand, we cannot form such values b, then we add s back in to our set B. Here, we say that $b \in B$ depends on s, if there exists a path from s to b in the DAG representation of the addition scheme B.

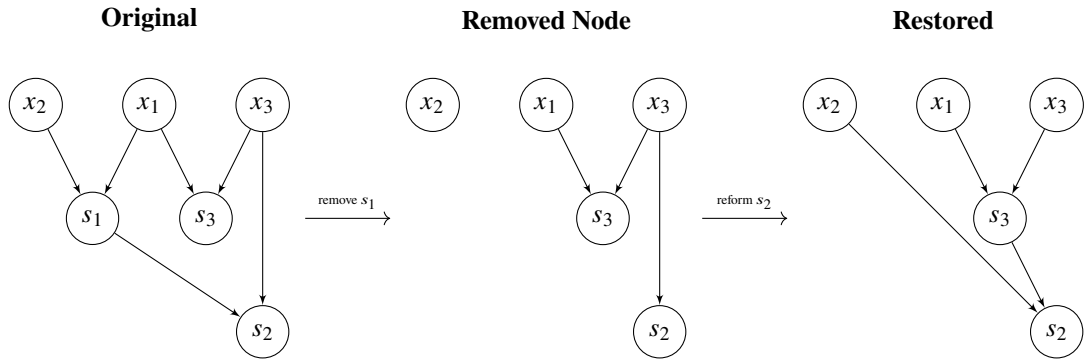


Fig. 22: Illustration of how the *swapping orders* postprocessing method is a special case of the proposed postprocessing scheme. The shown DAG might be part of a larger DAG, but as long as s_1 is used only in forming s_2 the shown reduction holds. Note that we could also form $x_2 = s_2 + s_3$, giving a new connected DAG, but this would not preserve the previous topological ordering, and is hence not considered by the described postprocessing approach.

We keep iteratively applying this postprocessing method until it is no longer possible to reduce the number of additions in an addition scheme using this method.

While the proposed postprocessing method is not complete, in that there are some reductions possible which cannot be found by the method described, it does cover all the manual reduction methods described in [8] except for rule R7, see Appendix C. That is, the set of 10 fixed postprocessing rules, all rules except R7, is a proper subset of the postprocessing rules we can find with our proposed postprocessing method. Our postprocessing method also covers certain reductions not covered by the fixed reduction rules from [8], see Appendix C.

4.3.2.1 Two-Level Deletion. We also implemented an extension of the dynamic reduction rule we proposed in Section 4.3.2. In this extension, two nodes are removed, and the DAG is restored by adding at most one node. This, in principle, could reduce some addition schemes even further, but after some experimentation, we quickly found that the increased running time required by this postprocessing rule affected both the framework introduced in Section 4.4 and the method of repeating the randomized heuristics at different seeds, to such an extent that the method was far from beneficial overall.

This extended dynamic postprocessing rule was therefore dropped.

4.4 Iterative SLP Framework

The heuristics described in section 4.2 are all single-shot optimization methods which provide good solution proposals to Problem 2.1. The heuristics relying on randomness can be run multiple times, which might give rise to new and improved solutions. However, [8] shows experimentally that by introducing various postprocessing reduction steps, and by performing an iterative search with each iteration informed by previous iterations, we can do better than this.

The framework proposed in [8] considers a given addition scheme $B \subseteq \mathbb{F}_2^n$ for a matrix $G \in \mathbb{F}_2^{m \times n}$. We can pick a subset $B' \subseteq B$, such that $e_i \notin B'$ for all $i \in [n]$. From B' we construct two ordered sets S_i, S_o of so-called *real* inputs and *real* outputs respectively. We first populate the real outputs, that is S_o, with all elements $b_i \in B'$ such that b_i is a target vector, that is $b_i \in T$, or b_i is used to form elements in $B \setminus B'$.

By definition of the addition scheme $B \subseteq \mathbb{F}_2^n$, we can write each element $b_k \in S_o$ as multiple different sums, by recursively expanding the variables used to construct it. For instance, if $b = b_{i_5} + b_{i_6}$, $b_{i_5} = b_{i_1} + b_{i_2}$,

and $b_{i_6} = b_{i_3} + b_{i_4}$, then we can write b in the following different ways:

$$\begin{aligned} b &= b_{i_5} + b_{i_6}, \\ b &= b_{i_5} + b_{i_3} + b_{i_4}, \\ b &= b_{i_1} + b_{i_2} + b_{i_6}, \\ b &= b_{i_1} + b_{i_2} + b_{i_3} + b_{i_4}. \end{aligned}$$

By expanding all the elements in $S_o \subseteq B$ this way we can find a common subset $S_i \subseteq B \setminus B'$ which generates S_o. This generating set is the set of so-called real inputs, S_i, and is what we get when we expand each element $b \in S_o$ until every value used in forming b is outside B', see Figure 23. The ordering of elements in S_i and in S_o follows the ordering in B. Denote by $e(b)$ the set of elements in the expansion of $b \in S_o$ used in constructing S_i, that is, if $b = b_{i_1} + b_{i_2} + \cdots + b_{i_j}$ for $b_{i_k} \in S_i$ for all $k \in [j]$, then from S_i and S_o we can construct the matrix $G' \in \mathbb{F}_2^{|S_o| \times |S_i|}$ via:

$$G'_{j,k} = \begin{cases} 1, & s_k \in e(h_j) \\ 0, & \text{otherwise} \end{cases} \quad \forall (j,k) \in [|S_o|] \times [|S_i|],$$

where s_k is the k-th element of S_i, and h_j is the j-th element of S_o. Refer to Figure 23 for intuition based on the DAG representation of addition schemes.

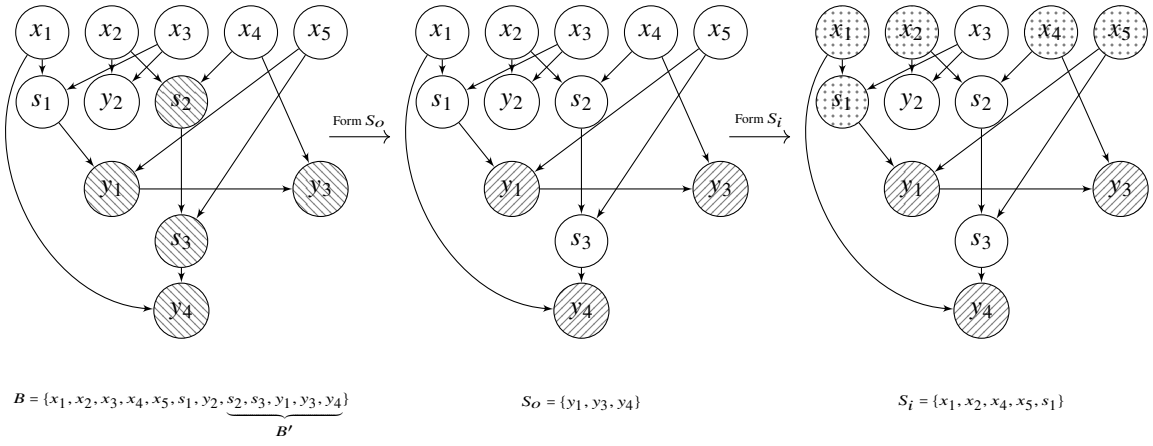


Fig. 23: Illustration of how a certain slice B', illustrated with ▨, of an addition scheme B for the example instance of Problem 2.1 introduced in Section 2.3.1 gives rise to the sets S_o, illustrated with ▨, and S_i, illustrated with ⣿.

Any addition scheme for the matrix G' can be translated directly back into the addition scheme for G by removing the set B' from B, and replacing it with the addition scheme for G', possibly with a re-indexing of the vectors in the new addition scheme.

The set B' is in the work [8] chosen as contiguous segments of the ordered set B. Specifically, a sliding window of a certain size $s = |B| - 1$ is constructed, all contiguous windows of size s are used as the set B'. Whenever an improvement can be made by way of optimizing over the window induced by s, with one of our heuristics, we reset $s \leftarrow |B| - 1$, but if no optimization for any window of a given size s is found, then we reduce s by 1. At some point s becomes less than 2, in which case we terminate the process.

Example 4.3. Given an instance of Problem 2.1 defined by the matrix $G \in \mathbb{F}_2^{4\times 5}$, as in equation (2.11), with an addition scheme given by

$$\begin{aligned} B = \{&b_1 = x_1, b_2 = x_2, b_3 = x_3, b_4 = x_4, b_5 = x_5 \\ &b_6 = s_1 = x_1 + x_3, b_7 = y_2 = x_2 + x_3, b_8 = s_2 = x_2 + x_4, b_9 = s_3 = s_2 + x_5, \\ &b_{10} = y_1 = s_1 + x_5, b_{11} = y_3 = y_1 + x_4, b_{12} = y_4 = s_3 + x_1\}, \end{aligned}$$

which is illustrated in DAG form, in Figure 23. We can take $B' = \{s_2, s_3, y_1, y_3, y_4\}$, and get $S_o = \{y_1, y_3, y_4\}$ and $S_i = \{x_1, x_2, x_4, x_5, s_1\}$, as illustrated in Figure 23. Write each element in S_o as a sum of elements in S_i, which we can do as per the construction of S_i:

$$y_1 = x_5 + s_1, \tag{4.19}$$
$$y_3 = x_4 + x_5 + s_1, \tag{4.20}$$
$$y_4 = x_1 + x_2 + x_4 + x_5. \tag{4.21}$$

Equations (4.19)–(4.21) can be seen as an instance of Problem 2.1 defined by the matrix

$$G' = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

Computing Gx naively would require 6 additions, but by pre-computing $t_1 = x_4 + x_5$, we can reduce the cost to 5 additions:

$$\begin{aligned} t_1 &= x_4 + x_5, \\ y_1 &= x_5 + s_1, \\ y_3 &= t_1 + s_1, \\ t_2 &= x_1 + x_2 \\ y_4 &= t_2 + t_1. \end{aligned}$$

We can re-insert this addition scheme for the B'-derived instance of Problem 2.1 back into the original addition scheme from which it was derived, see Figure 24.

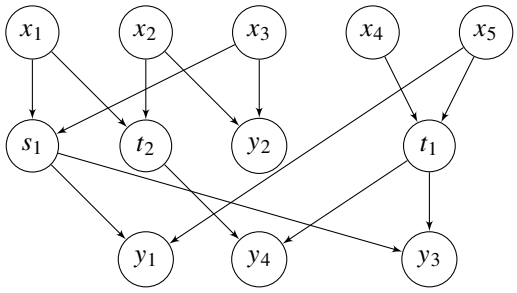


Fig. 24: Result of back-substituting the addition scheme for S_o, and S_i into the original addition scheme. Note that in the original addition scheme we had $s_2, s_3 \in B'$, hence why they were removed.

The scheme in Figure 24 require as many additions as the original scheme in Figure 23 while computing the same outputs. For examples of reductions induced by the postprocessing method, see Appendix C.

In [8], every element of S_o is expanded fully when generating S_i, such that $S_i \subseteq E$, with E defined as in the definition of Problem 2.1. We compare the two approaches and illustrate the result in Figure 33, and we find that expanding every element of S_o fully so that $S_i \subseteq E$ generally gives better results.

4.5 To parallelize or not to parallelize?

The framework described in Section 4.4 lends itself well to parallelization. Similarly performing the same randomized heuristic such as RNBP, A1, and A2 repeatedly also lends itself well to parallelization. Parallelizing these optimization methods would make optimizing an individual instance of Problem 2.1 much faster. That said, when the task is to optimize over multiple distinct instances of Problem 2.1 it can be more efficient to parallelize over the distinct instances, making parallelization over any given single instance undesired.

The main focus of this thesis is based on the 3×3 matrix multiplication dataset introduced in Section 3.1. When factoring in the basis change method described in Section 3.1.1, the dataset of general 3×3 matrix

multiplication schemes, can become exceedingly large. For this reason, we decide not to parallelize our framework internally, so as to allow for effective parallelization over multiple distinct schemes instead.

Chapter 5

Results

5.1 Individual Heuristic Optimizations

In Section 4.2 we detail and improve upon the most common heuristics used for solving Problem 2.1. The proposed improvements to each heuristic were justified theoretically in Section 4. In this section, we verify these improvements experimentally.

We note that we did not improve upon Paar's heuristic, introduced in Section 4.2.1, Paar's heuristic was initially posed in the $\mathbb{F}_2$ setting with the $\mathbb{F}_2$ speedups introduced in Section 4.2.1 [44]. We also note that the MIP method was not applied in any of the datasets below, as it turns out to be incredibly slow.

5.1.1 Greedy Potential

The greedy potential heuristic was introduced in the context of optimizing additions over $\{-1, 0, 1\}$. In this context we introduce the concept of computing the potential change, rather than the full potential, at each iteration. Computing the change in potential gave significant theoretical improvements to the time complexity of the greedy potential algorithm, see Section 4.2.2, and Figure 25 shows that this improvement indeed translates to a real practical speedup. In Section 4.2.2 we convert the greedy potential algorithm to be over $\mathbb{F}_2$ rather than $\{-1, 0, 1\}$, and show how this can induce certain speedups in the greedy potential algorithm. We verify that these speedups are significant in practice, see Figures 25 and 27. We compare our implementation of the greedy potential method to all other implementations we are aware of. Namely the two implementations provided in [32], and the implementation provided by [33].

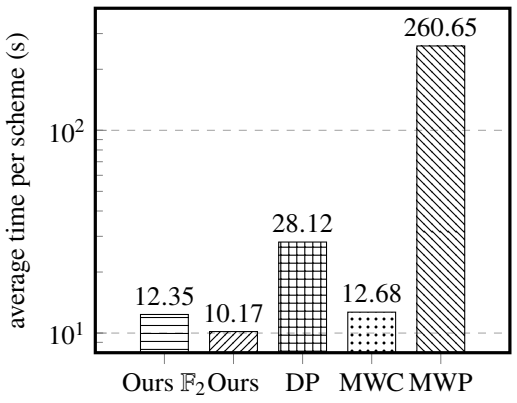


Fig. 25: Speed comparison between implementations of the greedy potential heuristic described in Section 4, 'DP' is the implementation by [49], 'MWC' is the C-implementation provided by [25], and 'MWP' the Python implementation also provided by [25]. The comparison was done on the 3×3 matrix multiplication scheme dataset provided by [4], see Section 3.1. Note that our $\mathbb{F}_2$ implementation is working over $\mathbb{F}_2$, while all other implementations work over $\{-1, 0, 1\}$. Also note that 'MWP' did not go through the whole dataset within the 72 hours of compute we allotted to it, while the other methods did manage to go through the whole dataset. All runs were allotted a total of 72 hours on a single core on an intel Xeon E5-2660 v3 CPU to go through the entire dataset.

We remark to the reader here, that the timings in Figure 25 measures the complete lifetime of the algorithm, including reading from standard input and writing to standard output, and therefore there is room for some uncertainty in terms of the relative speed of the actual algorithm implementations.

From Figure 25 we see that while we do get speedups when applying the change in potential optimization described in Section 4.2.2, we unexpectedly see less of a speedup when working over $\mathbb{F}_2$ as compared to over $\{-1, 0, 1\}$. We initially suspected that the lack of a significant speedup in the $\mathbb{F}_2$ case as compared to looking at the matrices over the set of values $\{-1, 0, 1\}$, was due to the dataset consisting of relatively sparse matrices.

For sparse matrices with entries in $\{-1, 0, 1\}$ we can compare pairs of columns, as is done in the greedy potential heuristic, quickly by only considering the intersection of the support for each of the two columns we are comparing.

At first we tested this hypothesis on the 3×3 matrix multiplication dataset used to compute the data for Figure 25, and found that we do not get a significant speedup working over $\mathbb{F}_2$ as compared to $\{-1, 0, 1\}$, when working over denser matrices in the dataset, see Figure 26.

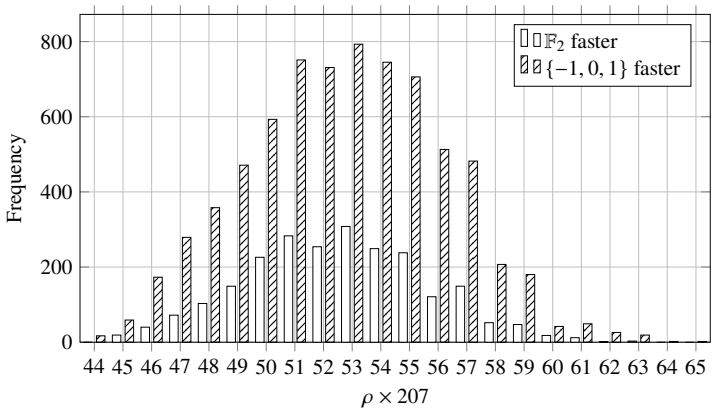


Fig. 26: Distribution over the schemes in the dataset described in Section 3.1 for which the $\mathbb{F}_2$ extension of the greedy potential method outperformed the corresponding ternary greedy potential method described in Section 4.2.2. ρ denotes the density, i.e. number of non-zero entries over the total number of entries. All matrices in the schemes in our dataset consist of $207 = 23 \cdot 9$ entries, hence making the scaled density $\rho \cdot 207$ integer valued. When the time required to compute a scheme was equal, we add an entry for that frequency to both bars. The data used here is derived from that which was used to compute Figure 25, i.e. it was not computed anew.

However, when constructing a dataset of random ternary matrices and comparing the $\mathbb{F}_2$ and $\{-1, 0, 1\}$ algorithms, see Section 3.4.1, we see the expected speedup derived from the methods described in Section 4.2.2, see Figure 27. The stark contrast between the expected $\mathbb{F}_2$-speedups we see in Figure 27, and the non-speedup in Figure 26 for the same pair of algorithms suggests that the dataset described in Section 3.1 has a very distinct structure, which lends itself well to the ternary implementation of the greedy potential method. We leave further exploration of the distribution of 3×3 matrix multiplication schemes to future work.

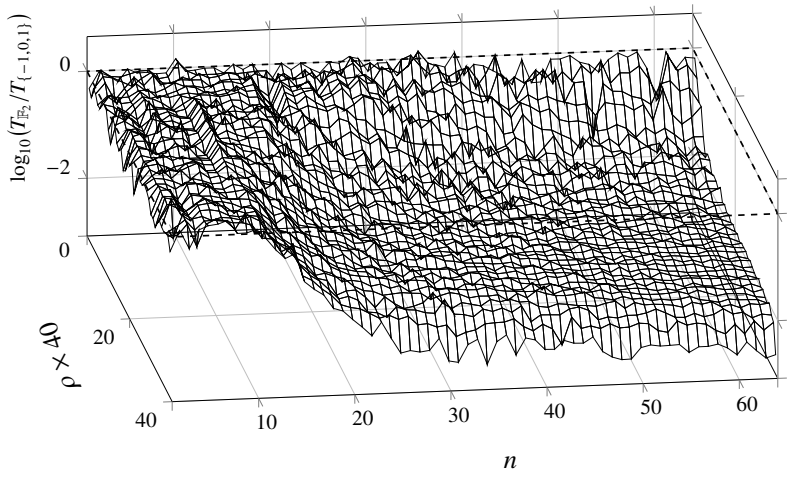


Fig. 27: Surface plot comparing the time, $T_{\mathbb{F}_2}$, required to compute each $\mathbb{F}_2$ scheme of the dataset in Section 3.4.1 to the time, $T_{\{-1,0,1\}}$, required to compute each of the $\{-1, 0, 1\}$ schemes of the same dataset. All instances were run on a single core on an Intel Xeon E5-2660 v3 CPU model.

We remind the reader that the 3×3 matrix multiplication dataset described in Section 3.1 is the only one of the four main datasets we consider that contain a meaningful formulation over $\{-1, 0, 1\}$, hence why we mainly test the greedy potential method on this dataset.

5.1.2 Boyar Peralta

We compare all proposed REACHABLE strategies with the baseline Boyar-Peralta algorithm used in [26]. The baseline BP algorithm used is an almost exact copy of the code developed for the original implementation of the BP algorithm [37]. One minor difference of the implementation in [26] to the one provided with the work in [37] being that the implementation provided in the repository for [26] takes matrices from standard input. This makes it easier to make a fair comparison against our implementation, which can do the same.

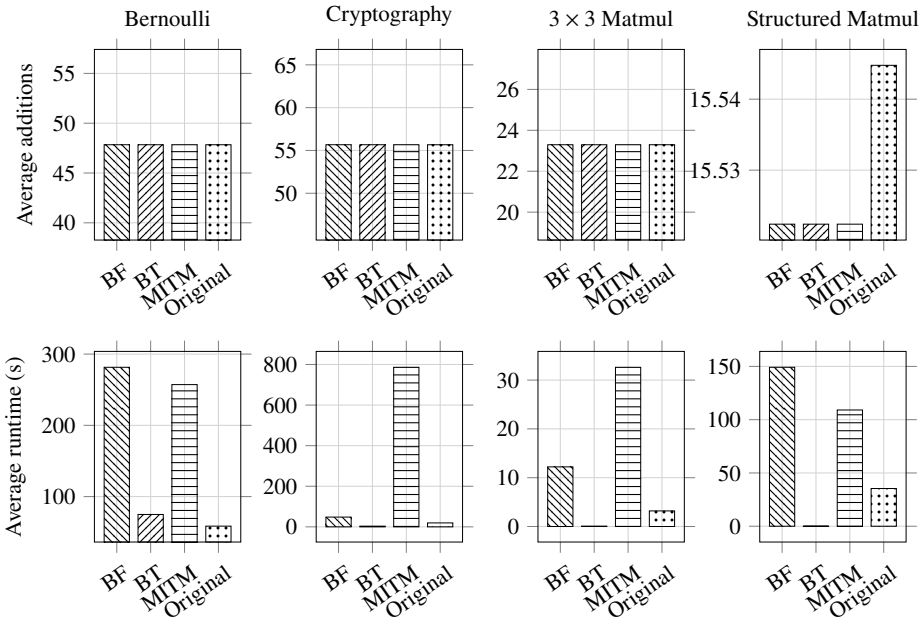


Fig. 28: Average number of additions and average runtime for brute force, sparsity-aware backtracking, meet-in-the-middle, and the original Boyar–Peralta SLP heuristic on the Bernoulli, Cryptography, 3×3 matrix multiplication, and structured matrix multiplication datasets. No preprocessing or postprocessing was applied. Each run was given a maximum wall-clock time of 3 hours per instance and 48 GB of RAM, and averages are computed only over instances for which all methods completed within these limits. The brute-force method is denoted "*BF*", the sparsity-aware backtracking method "*BT*", the meet-in-the-middle method "*MITM*", and "*Original*" denotes the implementation from [26]. All inputs were supplied via standard input. All experiments were run on a single core of an Intel Xeon E5-2660 v3 CPU.

Figure 28 shows that we achieve significant speedups using the proposed sparsity-aware backtracking method. This aligns with our expectations, as the sparsity-aware backtracking method was designed to be efficient in practice on relatively sparse matrices. These speedups should also apply to the algorithms RNBP, A1 and A2, as these methods also rely on the REACHABLE function, and has computing it as their bottlenecks [39].

The average number of additions in Figure 28 is included as a sanity check, and align with one another as expected. The very small deviation in the structured matrix multiplication dataset results, between our implementations and the one from [26] is likely due to random chance, as there are very slight differences in how we implement the case when there is some index $i \in [m]$ with $D_i = 1$.

5.1.3 Comparison of Single-Shot Heuristics

We compare the heuristics for finding $\mathbb{F}_2$ addition schemes described in Section 4 to one another on the benchmarks introduced in Section 3. The results are shown in Figures 29–32. The mixed integer programming method introduced in Section 4.2.6, and the Paar2 heuristic introduced in Section 4.2.1, are both too slow to run and compare against the heuristics on our benchmark datasets, and are therefore omitted below.

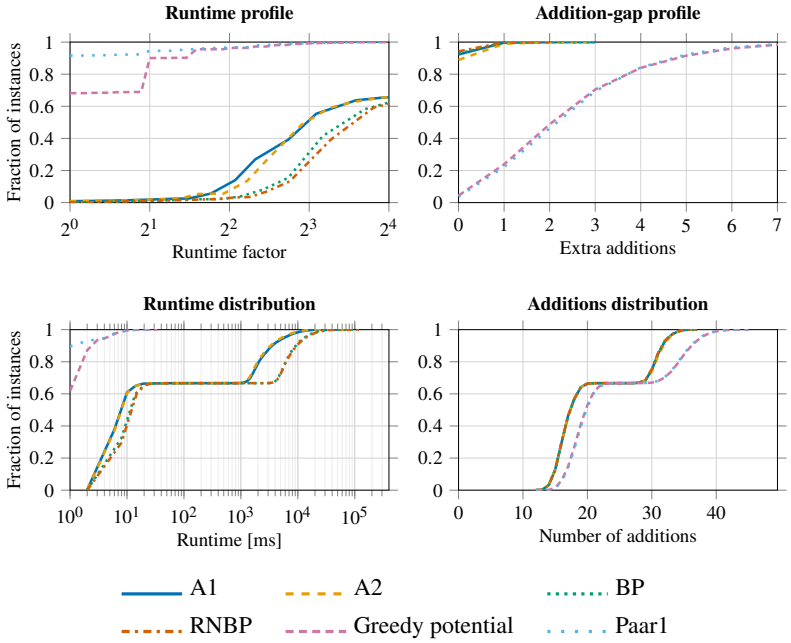


Fig. 29: Comparison of the six heuristics on the 3×3 matrix multiplication dataset, see Section 3.1. The runtime-profile and addition-profile show the fraction of instances solved within a given factor of the fastest method and best addition scheme respectively, on the same instance. No preprocessing or postprocessing was applied. 32 Bernoulli-random invertible basis change matrices drawn with Bernoulli bias parameter $\rho = 0.04$, were used for each data point, and applied as detailed in Section 3.1.1. All instances were run on a single core on an Intel Xeon E5-2660 v3 CPU model, and the time limit given was 1200 seconds per instance of SLP we solve.

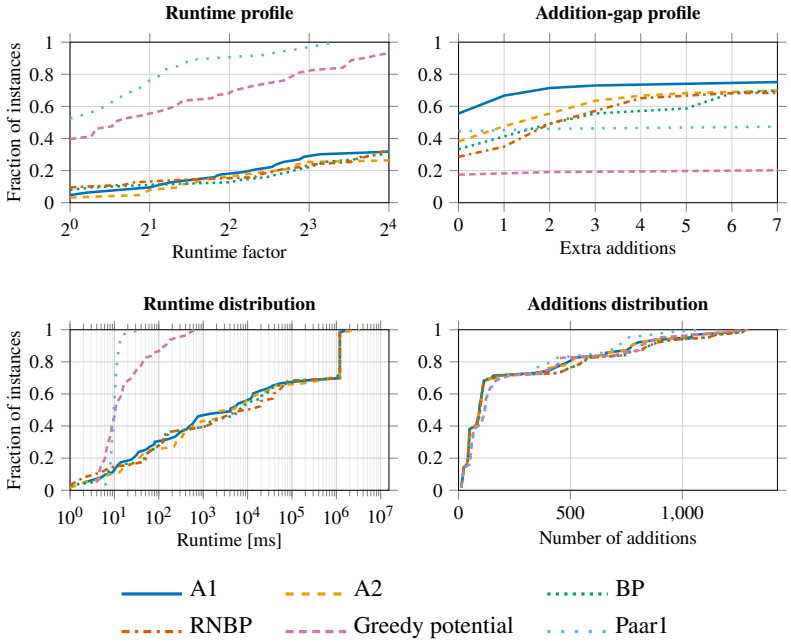


Fig. 30: Comparison of the six heuristics on the Crypt dataset. The runtime-profile and addition-profile show the fraction of instances solved within a given factor of the fastest method and best addition scheme respectively, on the same instance. No preprocessing or postprocessing was applied. All instances were run on a single core on an Intel Xeon E5-2660 v3 CPU model, and the time limit given was 1200 seconds per instance of SLP we solve - hence the sharp cutoff of the runtime at 1200000 milliseconds.

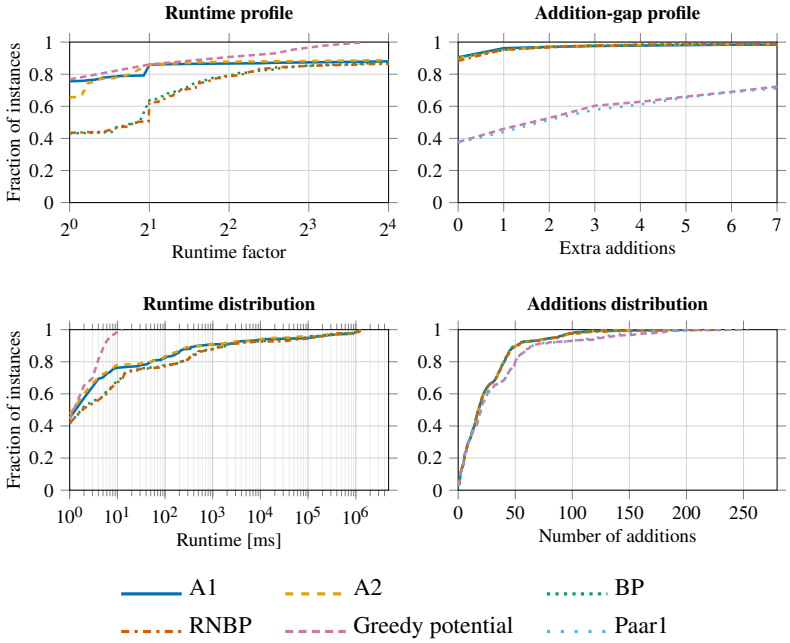


Fig. 31: Comparison of the six heuristics on the structured matrix multiplication dataset. We remark that the Paar1 heuristic took 0 milliseconds (we round to nearest millisecond) on most data points, hence its missing representation on the runtime plots. No preprocessing or postprocessing was applied. All instances were run on a single core on an Intel Xeon E5-2660 v3 CPU model, and the time limit given was 1200 seconds per instance of SLP we solve - hence the sharp cutoff of the runtime at 1200000 milliseconds.

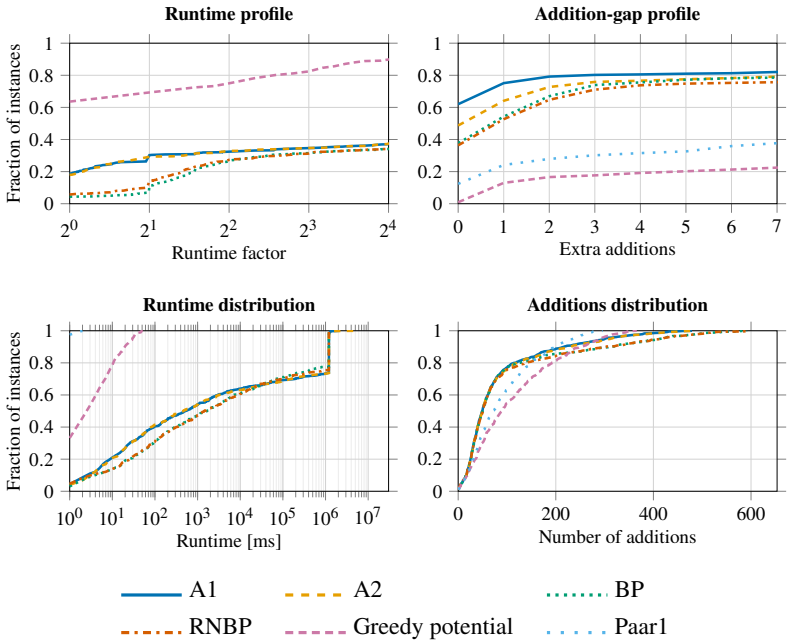


Fig. 32: Comparison of the six heuristics on the Bernoulli dataset. No preprocessing or postprocessing was applied. We remark that the Paar1 heuristic took 0 milliseconds (we round to nearest millisecond) on most data points, hence its missing representation on the runtime plots. All instances were run on a single core on an Intel Xeon E5-2660 v3 CPU model, and the time limit given was 1200 seconds per instance of SLP we solve.

5.1.4 Framework

We propose novel preprocessing and postprocessing schemes in Sections 4.1 and 4.3, moreover in Section 4.4 we describe how we implement the framework proposed in [8]. We show here the effectiveness of each part of this setup, and compare it against simply doing repetitions of our randomized algorithms with different random seeds, see Sections 4.2.4 and 4.2.5.

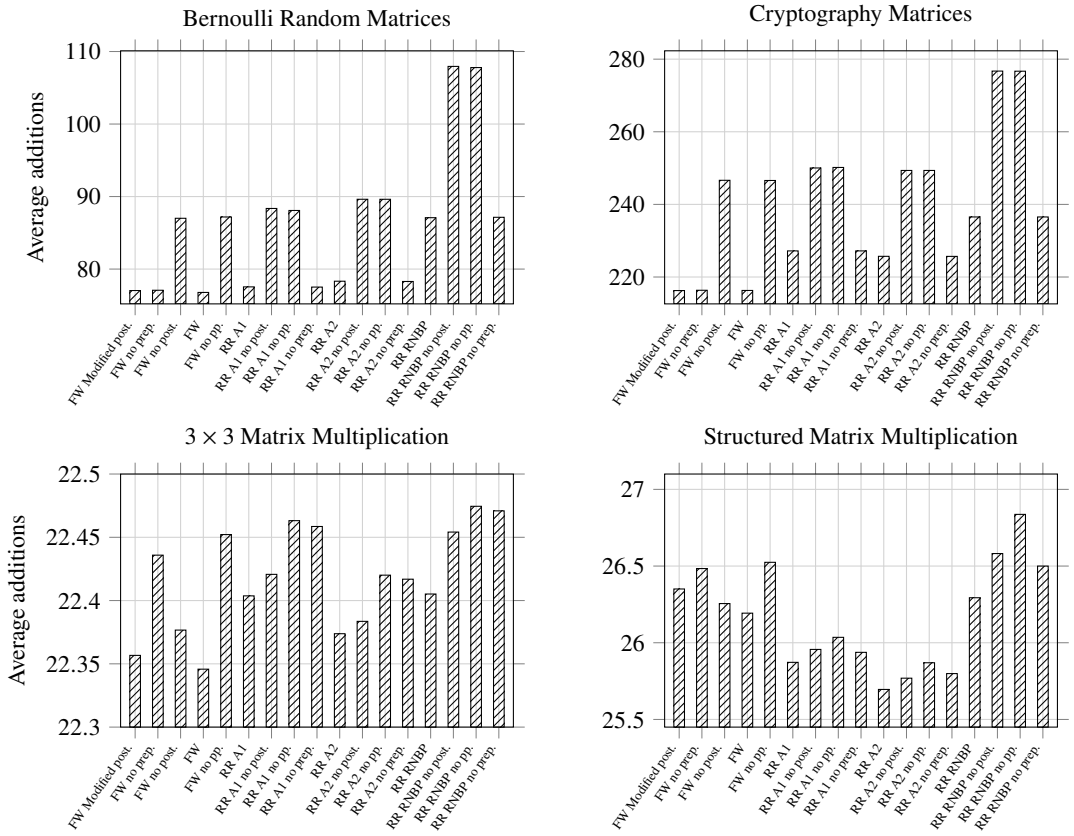


Fig. 33: Average number of additions found by each method on the Bernoulli, Cryptography, 3×3 matrix multiplication, and structured datasets. Each data point of the Bernoulli dataset is run for 350 seconds, each data point in the Cryptography dataset is run for 4000 seconds, each data point in the 3×3 matrix multiplication dataset is run for 4.8 seconds, and each data point in the structured matrix multiplication dataset is run for 350 seconds. The method *FW Modified post.* is the full framework with both preprocessing and postprocessing, where the postprocessing is as described in Section 4.3. The method *FW no prep.* is the framework without preprocessing but with postprocessing, *FW no post.* is the framework with preprocessing but without postprocessing, *FW* is the framework with the original full-extension postprocessing as proposed in [8], and *FW no pp.* is the framework with neither preprocessing nor postprocessing. The *RR* algorithms repeatedly apply the same randomized algorithm, namely A1, A2, or RNBP. The variants *RR no post.*, *RR no pp.*, and *RR no prep.* remove postprocessing, both preprocessing and postprocessing, and preprocessing, respectively. The time limits were chosen so as to complete the benchmark within 72 hours. All instances were run on a single core on an Intel Xeon E5-2660 v3 CPU model.

Figure 33 illustrates the effectiveness of our framework setup in finding good solutions to instances of Problem 2.1. Interestingly, the repeated randomized algorithms perform better on the structured matrix multiplication dataset than the framework, whereas our framework performs better on all other datasets we have. This suggests that there might be performance gains to be had in future work by picking the solver algorithm based on criteria of the specific instance to be solved. Moreover, our novel preprocessing methods clearly improve the performance of all the methods on all the datasets.

5.2 Rank-23 General 3×3 Matrix Multiplication

By applying the framework with preprocessing and postprocessing to the dataset described in Section 3.1, we obtain an empirical upper bound on the number of additions required to compute each 3×3 matrix multiplication scheme in the dataset, see Figure 34.

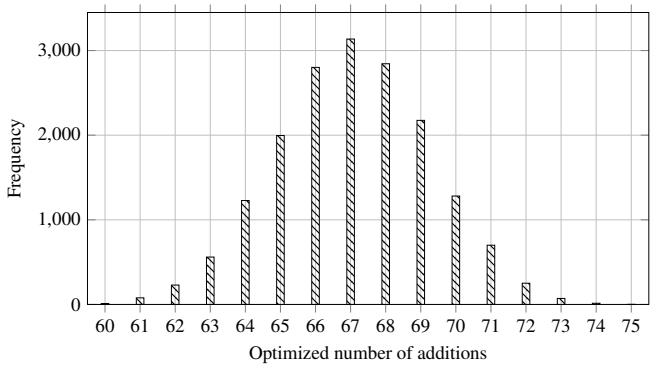


Fig. 34: For each Strassen-type matrix-matrix multiplication scheme from the dataset described in Section 3.1, the figure counts the total number of additions required to compute the three linear parts U, V, and W, see equation (2.3), after optimization by our framework. All instances were run on a single core on an Intel Xeon E5-2660 v3 CPU model, and each SLP instance was given a time limit of 4.8 seconds.

The distribution in Figure 34 includes 11 distinct 60-addition rank-23 general 3×3 matrix multiplication schemes, each of which would have been a unique state-of-the-art scheme prior to the work [32], published in December 2025. Moreover, our framework was only given 4.8 seconds to find a good addition scheme for each matrix U, V, and W, due to the large amount of data, hence giving more time to each scheme could lead to further improvements. These 11 schemes only work in $\mathbb{F}_2$ as given, but using the lifting method detailed in Section 2.3.2 we can lift all of them so that they work in general fields $\mathbb{F}$, see Appendix D for an example of a lifted 60-addition 3×3 rank-23 matrix multiplication scheme.

It is possible to further reduce the number of additions required to compute a given scheme by applying Theorem 3.1 with Bernoulli-random invertible matrices. Increasing the size of our dataset reduces the time spent on each instance, given a fixed time budget. The best one of our methods for any fixed data point is the framework method, as illustrated in Figure 33, while the fastest methods are the Paar1 and greedy-potential methods. We apply the framework, and the fast individual heuristic methods to the 3×3 matrix multiplication dataset using invertible Bernoulli-random basis change matrices, and obtain the results illustrated in Figure 35.

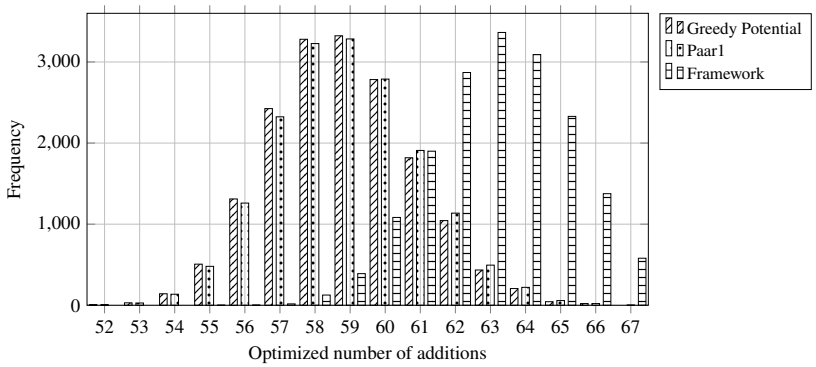


Fig. 35: Frequency distribution of the result of running a 32-thread parallelized solver over the 3×3 dataset with basis-change matrices. The best scheme found for each data point is recorded, together with the corresponding best basis change matrix B_U, B_V, B_W. The framework results were given a time limit of 2.4 seconds per instance, and 128 basis change matrices were used. The greedy-potential and Paar1 methods were run with $500,000$ basis change matrices drawn from the set of invertible Bernoulli-random matrices with Bernoulli bias $\rho = 0.04$.

From the greedy-potential heuristic we obtain 685 multiplication schemes for 3×3 matrices requiring a basis change, and 55 or fewer total additions. Each of these schemes beat the previous state-of-the-art, which required 56 additions with no basis change [3]. Of these 685 multiplication schemes 6 are distinct 52-addition 3×3 general matrix multiplication schemes with a basis change. The first one found was lifted from $\mathbb{F}_2$ to work over all fields $\mathbb{F}$, the lifting was done using the method proposed in Section 2.3.2. The resulting state-of-the-art 52-addition scheme can be seen in Appendix D.

5.3 Fast Structured Matrix Multiplication

The structured matrix multiplication dataset proposed in Section 3.2 is novel, and therefore has no good baseline to compare against. For readers interested in implementing some of the state-of-the-art structured matrix multiplication schemes introduced in [5], we refer the reader to our github repository, see Appendix E, where our optimized addition schemes for structured matrix multiplication are provided.

During our work we also apply our framework to the improved 3×3 structured matrix multiplication scheme for computing $AA^\intercal$ over $\mathbb{F}_2$ introduced in Appendix A and Example 3.3. The resulting optimized addition scheme is shown below. First we compute the 16 additions in the sub-matrices of A:

$$
\begin{aligned}
s_1 &= A_{2,1} + A_{3,1}, & s_2 &= A_{1,1} + A_{1,2}, & s_3 &= A_{1,2} + A_{2,2},\\
s_4 &= A_{2,2} + A_{3,2}, & s_5 &= A_{2,1} + A_{2,3}, & s_6 &= A_{1,3} + A_{3,3},\\
s_7 &= A_{1,1} + s_3, & s_8 &= A_{3,3} + s_4, & s_9 &= s_1 + s_4,\\
s_{10} &= A_{3,1} + s_5, & s_{11} &= A_{1,1} + s_5, & s_{12} &= A_{2,1} + s_7,\\
s_{13} &= A_{1,3} + s_8, & s_{14} &= A_{3,3} + s_{10}, & s_{15} &= s_2 + s_{13},\\
s_{16} &= s_5 + s_{15}.
\end{aligned}
$$

From this we can compute the desired recursive calls, and general matrix products:

$$
\begin{aligned}
S_1 &= A_{2,2}A_{2,2}^\intercal, & S_2 &= A_{3,1}A_{3,1}^\intercal, & S_3 &= A_{1,3}A_{1,3}^\intercal,\\
S_4 &= s_7 s_7^\intercal, & S_5 &= s_{13}s_{13}^\intercal, & S_6 &= s_{10}s_{10}^\intercal,\\
M_7 &= s_6 s_8^\intercal, & M_8 &= s_{12}A_{1,1}^\intercal, & M_9 &= A_{2,3}A_{1,3}^\intercal,\\
M_{10} &= A_{2,2}A_{3,2}^\intercal, & M_{11} &= s_{14}A_{2,3}^\intercal, & M_{12} &= s_5 s_1^\intercal,\\
M_{13} &= s_3 s_2^\intercal, & M_{14} &= s_9 s_{11}^\intercal, & M_{15} &= s_{16}s_4^\intercal.
\end{aligned}
$$

Finally, we can form the output entries of $AA^\intercal$ using 23 additions, as follows:

$$
\begin{aligned}
t_1 &= S_1 + M_{13}, & t_2 &= S_5 + M_7^\intercal,\\
t_3 &= S_6 + M_{12}^\intercal, & t_4 &= S_4 + t_1,\\
t_5 &= S_3 + t_2, & t_6 &= S_2 + t_3,\\
t_7 &= M_8^\intercal + t_4, & t_8 &= S_3 + t_4,\\
t_9 &= S_2 + t_5, & t_{10} &= M_{10} + t_6,\\
t_{11} &= M_{12} + t_6, & t_{12} &= M_{12} + t_7,\\
t_{13} &= M_{10} + t_9, & t_{14} &= S_1 + t_{11},\\
t_{15} &= t_5 + t_{12}, & t_{16} &= S_1 + t_{13},\\
t_{17} &= t_{16} + M_7, & t_{18} &= t_{15} + M_{15},\\
t_{19} &= t_7 + M_9^\intercal, & t_{20} &= t_{17} + M_{10}^\intercal,\\
t_{21} &= t_{10} + M_{11}^\intercal, & t_{22} &= t_8 + M_{13}^\intercal,\\
t_{23} &= t_{18} + M_{14}^\intercal,\\
w_1 &= t_{22}, & w_2 &= t_{19},\\
w_3 &= t_{23}, & w_4 &= t_{14},\\
w_5 &= t_{21}, & w_6 &= t_{20}.
\end{aligned}
$$

Such that

$$AA^\intercal = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_2^\intercal & w_4 & w_5 \\ w_3^\intercal & w_5^\intercal & w_6 \end{bmatrix}$$

Hence, over $\mathbb{F}_2$ we can compute the structured matrix product $AA^\intercal$ using only $16 + 23 = 39$ additions at each recursive level, rather than the $40 + 49 = 89$ additions we would perform using a naive implementation. This result is by no means the best we can do, especially factoring in that no basis-change has been applied in obtaining this result. It does however illustrate the massive performance gains that our framework can discover for these types of structured matrix multiplication methods.

5.4 Linear Cryptographic Layers

We compare the results obtained by applying our framework to the cryptographic dataset with those data points that are reported by both Kranz et al. in [26] and by Xiang et al. in [8], as shown in Table 2.

Table 2: Comparison of the number of additions obtained by our framework on the cryptographic dataset with the counts reported by [26] and [8]. We follow the instance naming convention used in [26]. Our results were obtained with a fixed time limit of 120, 000 seconds per SLP instance. Bold entries indicate the lowest reported count for that instance.

Instance / Source	Kranz et al. [26]	Xiang et al. [8]	Our Work
ACISP_SarSye17_8x8_4	201	**177**	284
AES	97	**91**	94
Anubis	113	**96**	100
C_BeiKraLea16_4x4_4	42	**40**	**40**
C_BeiKraLea16_4x4_8	110	105	**104**
Clefia_M0	106	**96**	100
Clefia_M1	111	**108**	**108**
FSE_LiWang16_4x4_4	44	**43**	**43**
FSE_LiWang16_4x4_4_i	48	**43**	**43**
FSE_LiWang16_4x4_8	104	**99**	**99**
FSE_LiWang16_4x4_8_i	91	**86**	**86**
FSE_LiuSim16_4x4_4	44	**43**	**43**
FSE_LiuSim16_4x4_8	112	**107**	109
FSE_SKOP15_4x4_4	48	**44**	45
FSE_SKOP15_4x4_4_i	48	**43**	**43**
FSE_SKOP15_4x4_8	100	**91**	**91**
FSE_SKOP15_4x4_8_i	100	89	**88**
FSE_SKOP15_8x8_4	194	**169**	286
FSE_SKOP15_8x8_4_i	217	**172**	360
Fox_Mu4	137	**130**	132
Joltik	48	**43**	**43**
SmallScale_AES	47	**43**	**43**
ToSC_SarSye16_4x4_4	43	**40**	**40**
ToSC_SarSye16_4x4_4_i	42	**37**	**37**
ToSC_SarSye16_4x4_8	107	**100**	101
ToSC_SarSye16_4x4_8_i	100	**92**	94
Twofish	129	**121**	138
Whirlwind_M0	212	**183**	360
Whirlwind_M1	235	**180**	379
ePrint_JeaPeySim_4x4_4	43	**40**	**40**
ePrint_JeaPeySim_4x4_4_i	47	**41**	**41**
ePrint_JeaPeySim_4x4_8	86	**80**	**80**
ePrint_JeaPeySim_4x4_8_i	91	**84**	86

Our results generally improve on the addition counts reported in [26], while the results reported in [8] generally remain better on the overlapping instances. However, these comparisons should be interpreted with some care. Neither [26] nor [8] reports both the hardware used and the exact optimization time. From what is reported in [8, 26] we can infer that Kranz et al. likely used a substantially smaller computational budget than ours, whereas Xiang et al. likely used a substantially larger one. Thus, the table should be read primarily as a comparison of obtained addition counts, rather than as a runtime-normalized benchmark.

The Bernoulli dataset does not have any inherent meaning or utility outside of comparing different optimization methods. Therefore no results section is included for this dataset.

Chapter 6

Conclusion and Outlook

In this work we improve upon several heuristics designed for solving instances of the SLP problem. Our most significant improvements are to the well-known Boyar-Peralta heuristic, see Figure 28, and to the greedy potential heuristic, see Figures 25 and 27. We detail how reducing the number of additions required to compute Strassen-type matrix multiplication schemes can be expressed as instances of the SLP problem. Using the improved heuristics we find several state-of-the-art implementations of multiple well-known linear layers, arising both in cryptography and in Strassen-type matrix multiplication. Of special interest is the new state-of-the-art 52-addition rank-23 Strassen-type matrix multiplication scheme for 3×3 matrices, which improves upon the state-of-the-art in a very active area of research [2, 3, 25, 29, 32, 33].

In addition to these advancements, we also provide multiple new state-of-the-art structured matrix multiplication schemes for computing $AA^\intercal$ over $\mathbb{F}_2$. The best of these improves the best known asymptotic ratio for structured gt matrix multiplication from 0.595 as found in [5] to 0.5678, see Appendix A.

6.1 Future Work

Framework Improvements. While the framework and optimizations introduced in this thesis perform very well, there is certainly space to improve. Developing more efficient preprocessing methods, both lossy and lossless, could significantly improve the framework's performance. Discovering new heuristics to incorporate into the framework, and figuring out good instance-dependent probability distributions over the implemented heuristics given certain metrics, such as the hardware it is running on, the time limits provided, etc., could lead to significant performance improvements. Finally, developing a faster, more complete postprocessing methods could also be an interesting direction to take further research. Another interesting improvement could be to utilize symmetry detection tools, such as [50], for removing redundant search trajectories and speeding up various heuristics for solving instances of Problem 2.1.

Framework Parallelization. As detailed in Section 4.5, we do not parallelize our framework, but we do parallelize over our datasets, see Section 5. This works well when we want to optimize over huge datasets, but when the desire is to spend the available computational power on a few, or even an individual, instance of Problem 2.1 this approach is not optimal. Working to efficiently parallelize the framework for such cases could help improve state of the art for certain individual important instances of Problem 2.1.

Information Set Decoding Methods. Information Set Decoding (ISD) refers to the problem of finding the additive distance used in BP and BP-inspired algorithms, i.e. exactly what Algorithms 4–6 compute [48]. It is a well-studied problem, and taking advantage of more advanced, state-of-the-art algorithms for ISD, might speed up the BP and BP-inspired heuristics significantly.

Large Matrices. In this work we assume matrices $G \in \mathbb{F}_2^{m \times n}$ with $m \leq 64$ and $n \leq 64$, as these assumptions lend themselves to certain optimization tricks, and representations using 64-bit unsigned integers. Moreover, the performance of most heuristics degrade rapidly as the matrix size increases. As such, it could be interesting to design and explore various heuristics for larger matrices.

Strassen Parallelism. Let an addition scheme be given in the DAG form, and define the depth of the addition scheme as the longest path from an input node x_i to an output node y_j. For practical Strassen-type matrix multiplication algorithms, having a smaller depth lends a given matrix multiplication scheme better to parallelization, as the depth of the DAG is exactly the length of the longest sequential number of additions required before recursing down the multiplication chain. As such, looking into minimizing not only the number of additions in a Strassen-type scheme, but also the depth of such schemes, could significantly improve the performance of Strassen-type matrix multiplication schemes in practice on parallel hardware.

Block Matrix Addition Improvements. Theorem 4.4 tells us that we might be able to reduce the number of additions beyond what is required to compute $U\vec{A}$, and $V\vec{B}$ separately in equation (2.3) by viewing the problem as an individual block-matrix computation

$$\begin{bmatrix} U & 0 \\ 0 & V \end{bmatrix} \begin{bmatrix} \vec{A} \\ \vec{B} \end{bmatrix}.$$

Reducing the additions required to compute the linear maps in Strassen-type schemes is an extensively researched area, and so reducing the number of additions in such a scheme via this method would be very exciting, and could translate to immediate tangible speedups in many matrix multiplication libraries that already employ Strassen-type algorithms.

Structured Matrix Multiplication Scheme Discovery. We find multiple new state-of-the-art multiplication schemes for computing $AA^\intercal$, see Appendix A, using a SAT-solver method inspired by [4]. Extending this method to more types of structured matrix multiplication could yield new state-of-the-art results for other structured matrix multiplication schemes. The structured matrix multiplication schemes found in Appendix A only work over $\mathbb{F}_2$, extending them to general fields $\mathbb{F}$ would also be an interesting direction to take future work.

All these examples show that there is a lot of exciting future research to do in terms of efficiently solving SLP instances and improving Strassen-type matrix multiplication schemes.

Use of Generative AI

ChatGPT was used in a limited capacity for debugging code. ChatGPT was used to assist with grammar and language formulation.

References

[1] Schwartz, O., Vaknin, N.: Pebbling game and alternative basis for high performance matrix multiplication. SIAM Journal on Scientific Computing **45**(6), 277–303 (2023) https://doi.org/10.1137/22M1502719 https://doi.org/10.1137/22M1502719

[2] Karstadt, E., Schwartz, O.: Matrix multiplication, a little faster. Journal of the ACM **67**(1) (2020) https://doi.org/10.1145/3364504

[3] Sun, Y.: An Exact 56-Addition, Rank-23 Scheme for General 3 × 3 Matrix Multiplication (2026). https://doi.org/10.48550/arXiv.2604.27645 . https://arxiv.org/abs/2604.27645

[4] Heule, M.J.H., Kauers, M., Seidl, M.: New ways to multiply 3 × 3-matrices. Journal of Symbolic Computation **104**, 899–916 (2021) https://doi.org/10.1016/j.jsc.2020.10.003

[5] Khoruzhii, K., Gelß, P., Pokutta, S.: Faster Algorithms for Structured Matrix Multiplication via Flip Graph Search (2025). https://arxiv.org/abs/2511.10786

[6] Dongarra, J.J., Du Croz, J., Hammarling, S., Duff, I.S.: A set of level 3 basic linear algebra subprograms. ACM Trans. Math. Softw. **16**(1), 1–17 (1990) https://doi.org/10.1145/77626.79170

[7] Fawzi, A., Balog, M., Huang, A., Hubert, T., Romera-Paredes, B., Barekatain, M., Novikov, A., Ruiz, F.J.R., Schrittwieser, J., Swirszcz, G., Silver, D., Hassabis, D., Kohli, P.: Discovering faster matrix multiplication algorithms with reinforcement learning. Nature **610**(7930), 47–53 (2022) https://doi.org/10.1038/s41586-022-05172-4

[8] Lin, D., Xiang, Z., Zeng, X., Zhang, S.: A framework to optimize implementations of matrices. In: Topics in Cryptology – CT-RSA 2021: Cryptographers' Track at the RSA Conference 2021, Virtual Event, May 17–20, 2021, Proceedings, pp. 609–632. Springer, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-030-75539-3_25 . https://doi.org/10.1007/978-3-030-75539-3_25

[9] Strassen, V.: Gaussian elimination is not optimal. Numerische Mathematik **13**(4), 354–356 (1969) https://doi.org/10.1007/BF02165411

[10] Pan, V.Y.: Strassen's algorithm is not optimal: Trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations. In: 19th Annual Symposium on Foundations of Computer Science (SFCS 1978), pp. 166–176. IEEE, Los Alamitos, CA, USA (1978). https://doi.org/10.1109/SFCS.1978.34

[11] Bini, D.A., Capovani, M., Romani, F., Lotti, G.: $O(n^{2.7799})$ complexity for $n \times n$ approximate matrix multiplication. Information Processing Letters **8**(5), 234–235 (1979) https://doi.org/10.1016/0020-0190(79)90113-3

[12] Schönhage, A.: Partial and total matrix multiplication. SIAM Journal on Computing **10**(3), 434–455 (1981) https://doi.org/10.1137/0210032

[13] Romani, F.: Some properties of disjoint sums of tensors related to matrix multiplication. SIAM Journal on Computing **11**(2), 263–267 (1982) https://doi.org/10.1137/0211020

[14] Coppersmith, D., Winograd, S.: On the asymptotic complexity of matrix multiplication. SIAM Journal on Computing **11**(3), 472–492 (1982) https://doi.org/10.1137/0211038

[15] Strassen, V.: The asymptotic spectrum of tensors and the exponent of matrix multiplication. In: Proceedings of the 27th Annual Symposium on Foundations of Computer Science (FOCS 1986), pp. 49–54. IEEE Computer Society, Los Alamitos, CA, USA (1986). https://doi.org/10.1109/SFCS.1986.52

[16] Coppersmith, D., Winograd, S.: Matrix multiplication via arithmetic progressions. Journal of Symbolic Computation **9**(3), 251–280 (1990) https://doi.org/10.1016/S0747-7171(08)80013-2

[17] Davie, A.M., Stothers, A.J.: Improved bound for complexity of matrix multiplication. Proceedings of the Royal Society of Edinburgh Section A: Mathematics **143**(2), 351–369 (2013) https://doi.org/10.1017/S0308210511001648

[18] Vassilevska Williams, V.: Multiplying matrices faster than coppersmith-winograd. In: Proceedings of the 44th Annual ACM Symposium on Theory of Computing. STOC '12, pp. 887–898. Association for Computing Machinery, New York, NY, USA (2012). https://doi.org/10.1145/2213977.2214056

[19] Le Gall, F.: Powers of tensors and fast matrix multiplication. In: Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation. ISSAC '14, pp. 296–303. Association for Computing Machinery, New York, NY, USA (2014). https://doi.org/10.1145/2608628.2608664

[20] Alman, J., Vassilevska Williams, V.: A refined laser method and faster matrix multiplication. In: Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 522–539. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA (2021). https://doi.org/10.1137/1.9781611976465.32

[21] Duan, R., Wu, H., Zhou, R.: Faster matrix multiplication via asymmetric hashing. In: 2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS), pp. 2129–2138. IEEE, Los Alamitos, CA, USA (2023). https://doi.org/10.1109/FOCS57990.2023.00130

[22] Vassilevska Williams, V., Xu, Y., Xu, Z., Zhou, R.: New bounds for matrix multiplication: From alpha to omega. In: Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 3792–3835. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA (2024). https://doi.org/10.1137/1.9781611977912.134

[23] Alman, J., Duan, R., Vassilevska Williams, V., Xu, Y., Xu, Z., Zhou, R.: More asymmetry yields faster matrix multiplication. In: Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 2005–2039. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA (2025). https://doi.org/10.1137/1.9781611978322.63

[24] Pan, V.Y.: Matrix Multiplication, Trilinear Decompositions, APA Algorithms, and Summation. arXiv:1412.1145v3, revised February 5, 2015 (2014). https://doi.org/10.48550/arXiv.1412.1145

[25] Mårtensson, E., Wagner, P.S.: The number of the beast: Reducing additions in fast matrix multiplication algorithms for dimensions up to 666. In: Conway, A., Pothen, A., Farach-Colton, M., U çar, B. (eds.) Proceedings of the 3rd Conference on Applied and Computational Discrete Algorithms (ACDA 2025), pp. 47–60. SIAM, Philadelphia, PA (2025). https://doi.org/10.1137/1.9781611978759.4 . https://doi.org/10.1137/1.9781611978759.4

[26] Kranz, T., Leander, G., Stoffelen, K., Wiemer, F.: Shorter linear straight-line programs for mds matrices. IACR Transactions on Symmetric Cryptology **2017**(4), 188–211 (2017) https://doi.org/10.13154/tosc.v2017.i4.188-211

[27] Williams, V.V.: An overview of the recent progress on matrix multiplication. ACM SIGACT News **34**(3), 2–13 (2003)

[28] Winograd, S.: On multiplication of 2×2 matrices. Linear Algebra and its Applications **4**(4), 381–388 (1971) https://doi.org/10.1016/0024-3795(71)90009-7

[29] Stapleton, J.: A 60-Addition, Rank-23 Scheme for Exact 3x3 Matrix Multiplication (2025). https://doi.org/10.48550/arXiv.2508.03857 . https://arxiv.org/abs/2508.03857

[30] Johnson, R.W., McLoughlin, A.M.: Noncommutative bilinear algorithms for 3 x 3 matrix multiplication. SIAM Journal on Computing **15**(2), 595–603 (1986) https://doi.org/10.1137/0215043

[31] Laderman, J.D.: A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. Bull. Amer. Math. Soc. **82**(1), 126–128 (1976) https://doi.org/10.1090/S0002-9904-1976-13988-2

[32] Mårtensson, E., Stankovski Wagner, P., Stapleton, J.: A Rank 23 Algorithm for Multiplying 3 x 3

Matrices with an Arithmetic Complexity of 59 (2025). https://doi.org/10.48550/arXiv.2601.05272 . https://arxiv.org/abs/2601.05272

[33] Perminov, A.I.: A 58-Addition, Rank-23 Scheme for General 3x3 Matrix Multiplication (2025). https://doi.org/10.48550/arXiv.2512.21980 . https://arxiv.org/abs/2512.21980

[34] Kauers, M., Moosbauer, J.: Flip graphs for matrix multiplication. In: Proceedings of the 2023 International Symposium on Symbolic and Algebraic Computation. ISSAC '23, pp. 381–388. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3597066.3597120 . https://doi.org/10.1145/3597066.3597120

[35] Courtois, N.T., Hulme, D., Mourouzis, T.: Multiplicative complexity and solving generalized brent equations with sat solvers. In: COMPUTATION TOOLS 2012: The Third International Conference on Computational Logics, Algebras, Programming, Tools, and Benchmarking, pp. 22–27. IARIA, Nice, France (2012). https://www.thinkmind.org/library/COMPUTATION_TOOLS/COMPUTATION_TOOLS_2012/computation_tools_2012_1_40_80034.html

[36] Rybin, D., Zhang, Y., Luo, Z.-Q.: XX^t Can Be Faster. arXiv preprint arXiv:2505.09814v2 (2025). https://doi.org/10.48550/arXiv.2505.09814

[37] Boyar, J., Peralta, R.: A new combinational logic minimization technique with applications to cryptology. In: Festa, P. (ed.) Experimental Algorithms, pp. 178–189. Springer, Berlin, Heidelberg (2010)

[38] Visconti, A., Schiavo, C.V., Peralta, R.: Improved upper bounds for the expected circuit complexity of dense systems of linear equations over GF(2). Cryptology ePrint Archive, Paper 2017/194 (2017). https://eprint.iacr.org/2017/194

[39] Tan, Q.Q., Peyrin, T.: Improved Heuristics for Short Linear Programs. Cryptology ePrint Archive, Paper 2019/847 (2019). https://doi.org/10.13154/tches.v2020.i1.203-230 . https://eprint.iacr.org/2019/847

[40] Jukna, S., Sergeev, I.: Complexity of linear Boolean operators. Foundations and Trends in Theoretical Computer Science **9**(1), 1–123 (2013) https://doi.org/10.1561/0400000063

[41] Nechiporuk, É.I.: Rectifier networks. Doklady Akademii Nauk SSSR **148**(1), 50–53 (1963). published in Soviet Physics Doklady, 8 (1963), 5–7. Accessed 2026-05-11

[42] Jukna, S.: Solution of Problem 7.7. https://web.vu.lt/mif/s.jukna/Knizka/problem-7.7.html. Added material dated June 4, 2017 (2015)

[43] Yuan, Y., Wu, W., Shi, T., Zhang, L., Zhang, Y.: A framework to improve the implementations of linear layers. IACR Transactions on Symmetric Cryptology **2024**(2), 322–347 (2024) https://doi.org/10.46586/tosc.v2024.i2.322-347

[44] Paar, C.: Optimized arithmetic for reed-solomon encoders. In: Proceedings of IEEE International Symposium on Information Theory, p. 250 (1997). https://doi.org/10.1109/ISIT.1997.613165

[45] Intel Corporation: Intel 64 and IA-32 Architectures Software Developer's Manual. Intel Corporation, (2026). Intel Corporation. Version 091. See Volume 2: Instruction Set Reference, A–Z. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html Accessed 2026-06-14

[46] Sun, Y., Yang, R., Li, T.: Revisit the Boyar-Peralta Algorithm to Solve the Shortest Linear Program Problem. Cryptology ePrint Archive, Paper 2025/1493 (2025). https://eprint.iacr.org/2025/1493

[47] Berlekamp, E.R., McEliece, R.J., Tilborg, H.C.A.: On the inherent intractability of certain coding problems. IEEE Transactions on Information Theory **24**(3), 384–386 (1978) https://doi.org/10.1109/TIT.1978.1055873

[48] May, A., Ozerov, I.: On computing nearest neighbors with applications to decoding of binary linear codes. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology – EUROCRYPT 2015. Lecture Notes in Computer Science, vol. 9056, pp. 203–228. Springer, Berlin, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46800-5_9

[49] Perminov, A.I.: Parallel heuristic exploration for additive complexity reduction in fast matrix multiplication. arXiv preprint arXiv:2512.13365 (2025)

[50] Anders, M., Schweitzer, P.: Parallel Computation of Combinatorial Symmetries. In: 29th Annual European Symposium on Algorithms (ESA 2021). Leibniz International Proceedings in Informatics (LIPIcs), vol. 204, pp. 6–1618. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021). https://doi.org/10.4230/LIPIcs.ESA.2021.6

[51] Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F.: CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024. In: Heule, M., Iser, M., Järvisalo, M., Suda, M. (eds.) Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions. Department of Computer Science Report Series B, vol. B-2024-1, pp. 8–10. University of Helsinki, Helsinki, Finland (2024)

Appendix

A Recursive Scheme for $AA^\intercal$

To the best of our knowledge, the result for gt structured matrix multiplication presented in [5] was previously state of the art. In this appendix we introduce the methods used to find the new state of the art method for gt structured matrix multiplication over $\mathbb{F}_2$, which was used in the example in Section 3.2.

We remind the reader that most work on structured matrix-matrix multiplication aims to find Strassen-type schemes with as many of the resulting sub-matrix multiplications being structured themselves, see the example in Section 3.2. Such setups lead to recurrence relations of the form

$$S(n) = aS(n/k) + bM(n/k) + O(n^2), \quad S(1) = M(1) = 1, \tag{A.1}$$

where $S(n)$ denotes the number of arithmetic operations required to compute the structured $n \times n$ matrix-matrix product, $M(n)$ the number of arithmetic operations to perform general $n \times n$ matrix product, k the size of the block matrix used, a the number of structured block matrix multiplications induced by the scheme, and b the number of general block-matrix multiplication products. If we let $M(n) = Cn^\omega + O(n^2)$ with $\omega > 2$, and $a < k^2$, then the solution to the recurrence relation equation (A.1) is given by:

$$S(n) = \frac{b}{k^\omega - a} M(n) + O(n^2), \tag{A.2}$$

as can be easily verified by substituting the claimed solution back into equation (A.1):

$$S(n) \overset{(A.2)}{=} \frac{b}{k^\omega - a} M(n) + O(n^2) = \frac{1}{k^\omega} \left(\frac{ab + bk^\omega - ab}{k^\omega - a} \right) M(n) + O(n^2) \tag{A.3}$$

$$= \frac{1}{k^\omega} \left(a \frac{b}{k^\omega - a} + b \right) M(n) + O(n^2) = a \left(\frac{b}{k^\omega - a} \frac{M(n)}{k^\omega} \right) + b \frac{M(n)}{k^\omega} + O(n^2) \tag{A.4}$$

$$= aS(n/k) + bM(n/k) + O(n^2), \tag{A.5}$$

as was to be shown, where we have used $M(n/k) = M(n)/k^\omega + O(n^2)$. As a consequence of equations (A.3)–(A.5) the relative savings in number of operations using a given structured scheme as compared to using regular matrix multiplication is given by:

$$S(n)/M(n) = \frac{b}{k^\omega - a} + o(1), \tag{A.6}$$

and is usually considered in the limiting case such that the $o(1)$ term vanishes. We remark here that the pre-requisite $a < k^2$ might not necessarily always be the case, but it suffices for our considerations, see Table 3, moreover the ratio will asymptotically stay the same, even if $a \geq k^2$ [5].

For gt structured matrix-matrix multiplication, see Table 1, the previously best known scheme had a ratio of the form equation (A.6) of 0.595 with $\omega = \log_2 7$ [5]. Thus to improve upon this ratio, we need a scheme for gt matrix multiplication with values $a, b, k \in \mathbb{N}$ satisfying

$$0.595 \left(k^{\log_2 7} - a \right) \geq b. \tag{A.7}$$

For $k = 3$ equation (A.7) implies $b + 0.595a \leq 13.00067$, which can only be satisfied when integers $a \leq 21$ and $b \leq 13$, hence we can determine existence or non-existence of schemes satisfying any such a, b-combination. To this end we setup the polynomial equations corresponding to gt-structured matrix multiplication schemes, see the work [4] for more on how to construct such polynomial equations, the code used to generate the polynomial equations can be seen in https://github.com/dragonoverlord3000/AAT-SAT. We determine the solution or infeasibility of solving these polynomial equations over $\mathbb{F}_2$ using a SAT solver, just as in the work [4], but without the lossy simplifications they introduce, such that unsatisfiability translates directly to the polynomial equations having no solutions over $\mathbb{F}_2$. Through this process we find the results showcased in Table 3.

$a\backslash b$	1	2	3	4	5	6	7	8	9	10	11	12	13
1	NO*	NO*	NO*	NO*	?	?	?	?	?	?	?	?	
2	NO*	NO*	NO*	NO*	?	?	?	?	?	?	?		
3	NO*	NO*	NO*	NO*	?	?	?	?	?	?	YES		
4	NO*	NO*	NO*	NO*	?	?	?	?	?	?			
5	NO*	NO*	NO*	NO*	?	?	?	?	?	YES			
6	NO*	NO*	NO*	NO*	?	?	?	?	YES				
7	NO*	NO*	NO*	NO*	?	?	?	?					
8	NO*	NO*	NO*	NO*	?	?	?	?					
9	NO*	NO*	NO*	NO*	?	?	?						
10	NO*	NO*	NO*	NO*	?	?	?						
11	NO*	NO*	NO*	NO*	?	?							
12	NO*	NO*	NO*	NO*	?								
13	NO*	NO*	NO*	NO*	?								
14	NO*	NO*	NO*	NO									
15	NO*	NO*	NO*	NO									
16	NO*	NO*	NO										
17	NO*	NO											
18	NO*	NO											
19	NO												
20	NO												
21													

Table 3: The blank cells correspond to pairs (a, b) not satisfying $b + 0.595a \leq 13.00067$, the NO* are not computed directly, but are an inference from the result of a cell either above or to the right of it. The cells containing "?" denote that KISSAT [51] could not determine satisfiability for the conjunctive normal form of the set of multivariate polynomial corresponding to the cell within 72 hours, running on a single core, on a XeonE5_2660v3 CPU.

For verification purposes, the exact code used for computing the conjunctive normal forms we apply the KISSAT solver [51] to so as to fill out Table 3 can be found in the Github repository https://github.com/dragonoverlord3000/AAT-SAT.

As illustrated in Table 3, we found a scheme with $a = 5, b = 10$, which has $S(n)/M(n) = 0.5935 + o(1)$, we found a scheme with $a = 3, b = 11$ which has $S(n)/M(n) = 0.5836 + o(1)$, and finally we found a scheme with $a = 6, b = 9$ which has $S(n)/M(n) = 0.5678 + o(1)$. All these three new schemes represent state of the art [5, 36] over $\mathbb{F}_2$, and we consider extending these results to general fields $\mathbb{F}$ as an interesting direction for future work.

The best scheme found, namely $a = 6, b = 9$, is written out explicitly in Example 3.3. The method for computing the $a = 5, b = 10$ scheme is given by first computing:

$$
\begin{aligned}
S_1 &= \left(A_{1,2} + A_{2,2}\right)\left(A_{1,2} + A_{2,2}\right)^{\intercal}, \\
S_2 &= \left(A_{1,1} + A_{1,2} + A_{1,3}\right)\left(A_{1,1} + A_{1,2} + A_{1,3}\right)^{\intercal}, \\
S_3 &= \left(A_{2,3} + A_{3,3}\right)\left(A_{2,3} + A_{3,3}\right)^{\intercal}, \\
S_4 &= \left(A_{1,1} + A_{1,3} + A_{2,1} + A_{3,3}\right)\left(A_{1,1} + A_{1,3} + A_{2,1} + A_{3,3}\right)^{\intercal}, \\
S_5 &= \left(A_{1,1} + A_{1,3} + A_{2,2} + A_{3,1} + A_{3,2} + A_{3,3}\right)\left(A_{1,1} + A_{1,3} + A_{2,2} + A_{3,1} + A_{3,2} + A_{3,3}\right)^{\intercal}, \\
M_6 &= A_{3,1}\left(A_{1,1} + A_{1,2} + A_{1,3}\right)^{\intercal}, \\
M_7 &= \left(A_{1,2} + A_{2,2}\right)A_{3,2}^{\intercal}, \\
M_8 &= \left(A_{3,2} + A_{3,3}\right)A_{1,2}^{\intercal}, \\
M_9 &= \left(A_{3,1} + A_{3,3}\right)\left(A_{1,2} + A_{1,3}\right)^{\intercal}, \\
M_{10} &= \left(A_{1,1} + A_{1,3} + A_{3,1} + A_{3,3}\right)\left(A_{1,1} + A_{1,2} + A_{3,1} + A_{3,3}\right)^{\intercal}, \\
M_{11} &= \left(A_{1,1} + A_{1,3} + A_{3,1} + A_{3,3}\right)\left(A_{2,1} + A_{2,2} + A_{3,1} + A_{3,2}\right)^{\intercal}, \\
M_{12} &= \left(A_{1,1} + A_{1,2} + A_{1,3} + A_{3,1} + A_{3,3}\right)\left(A_{1,2} + A_{2,2} + A_{3,2} + A_{3,3}\right)^{\intercal}, \\
M_{13} &= A_{3,1}A_{2,1}^{\intercal}, \\
M_{14} &= \left(A_{1,1} + A_{1,3} + A_{2,1} + A_{2,3}\right)\left(A_{1,3} + A_{3,3}\right)^{\intercal},
\end{aligned}
$$

$$M_{15} = A_{3,3}\left(A_{1,2} + A_{2,3}\right)^{\intercal}.$$

And then $AA^{\intercal}$ is given by:

$$AA^{\intercal} = \begin{bmatrix} S_2 + M_9 + M_9^{\intercal} + M_{10} + M_{10}^{\intercal} & S_2 + M_6 + M_8^{\intercal} + M_9^{\intercal} + M_{10}^{\intercal} + M_{11} + M_{12} + M_{13} + M_{14}^{\intercal} + M_{15} & M_6^{\intercal} + M_8^{\intercal} + M_9^{\intercal} \\ \star & S_1 + S_2 + S_3 + S_4 + M_6 + M_6^{\intercal} + M_8 + M_8^{\intercal} + M_{11} + M_{11}^{\intercal} + M_{12} + M_{12}^{\intercal} + M_{13} + M_{13}^{\intercal} + M_{15} + M_{15}^{\intercal} & M_7 + M_8^{\intercal} + M_{13}^{\intercal} + M_{15}^{\intercal} \\ \star & \star & S_1 + S_2 + S_5 + M_6 + M_6^{\intercal} + M_7 + M_7^{\intercal} + M_{12} + M_{12}^{\intercal} \end{bmatrix}.$$

And for the scheme with $a = 3$, and $b = 11$ we first compute:

$$\begin{aligned}
S_1 &= \left(A_{3,1} + A_{3,2} + A_{3,3}\right)\left(A_{3,1} + A_{3,2} + A_{3,3}\right)^{\intercal}, \\
S_2 &= \left(A_{1,1} + A_{1,2} + A_{1,3}\right)\left(A_{1,1} + A_{1,2} + A_{1,3}\right)^{\intercal}, \\
S_3 &= \left(A_{2,1} + A_{2,2} + A_{2,3}\right)\left(A_{2,1} + A_{2,2} + A_{2,3}\right)^{\intercal}, \\
M_4 &= A_{1,3}\left(A_{2,1} + A_{2,2} + A_{2,3}\right)^{\intercal}, \\
M_5 &= A_{2,2}\left(A_{1,1} + A_{1,2}\right)^{\intercal}, \\
M_6 &= \left(A_{1,2} + A_{1,3}\right)\left(A_{1,1} + A_{1,3} + A_{2,1} + A_{2,3} + A_{3,1} + A_{3,3}\right)^{\intercal}, \\
M_7 &= \left(A_{2,1} + A_{2,3} + A_{3,2} + A_{3,3}\right)\left(A_{1,1} + A_{1,3} + A_{2,1} + A_{2,3} + A_{3,2} + A_{3,3}\right)^{\intercal}, \\
M_8 &= \left(A_{2,2} + A_{2,3} + A_{3,2} + A_{3,3}\right)\left(A_{1,1} + A_{1,3} + A_{2,1} + A_{2,3} + A_{3,1} + A_{3,3}\right)^{\intercal}, \\
M_9 &= \left(A_{2,1} + A_{2,2} + A_{2,3} + A_{3,1} + A_{3,2} + A_{3,3}\right)\left(A_{1,1} + A_{1,2} + A_{1,3}\right)^{\intercal}, \\
M_{10} &= \left(A_{1,1} + A_{1,3}\right)\left(A_{1,2} + A_{1,3} + A_{2,1} + A_{2,3} + A_{3,2} + A_{3,3}\right)^{\intercal}, \\
M_{11} &= \left(A_{2,1} + A_{2,3}\right)\left(A_{2,1} + A_{2,2}\right)^{\intercal}, \\
M_{12} &= \left(A_{2,1} + A_{2,2}\right)\left(A_{1,1} + A_{1,3} + A_{2,1} + A_{2,3}\right)^{\intercal}, \\
M_{13} &= \left(A_{3,2} + A_{3,3}\right)\left(A_{3,1} + A_{3,2}\right)^{\intercal}, \\
M_{14} &= \left(A_{1,3} + A_{3,1} + A_{3,2} + A_{3,3}\right)\left(A_{2,1} + A_{2,2} + A_{2,3}\right)^{\intercal}.
\end{aligned}$$

So that we can compute $AA^{\intercal}$ via:

$$AA^{\intercal} = \begin{bmatrix} S_2 + M_7 + M_7^{\intercal} + M_{10} + M_{10}^{\intercal} & M_4 + M_5^{\intercal} + M_{11} + M_{12}^{\intercal} & M_4 + M_5^{\intercal} + M_6 + M_7 + M_7^{\intercal} + M_9^{\intercal} + M_{10}^{\intercal} \\ \star & S_3 + M_{11} + M_{11}^{\intercal} & M_4^{\intercal} + M_7 + M_8 + M_{12} + M_{13} + M_{14}^{\intercal} \\ \star & \star & S_1 + M_{13} + M_{13}^{\intercal} \end{bmatrix}.$$

The code used to generate the set of polynomial equations that follow from letting U, V, and W in equation (2.3) be indeterminents and comparing coefficients on the left and right hand sides, with a fixed set of recursions (what we denote a in Table 3), and then translate the polynomial equations to the conjunctive normal form format required to apply a SAT-solver to it, is given in https://github.com/dragonoverlord3000/AAT-SAT.

B Strassen's 2×2 matrix multiplication implementation

The following is a small python implementation of Algorithm 2, with $k = 2$, created with the purpose of being succinct and easily readable, while also providing insight into the effects of Theorem 3.1.

```
import itertools, numpy as np

def strassen(U, V, Wt, A, B, _r, l) -> np.array:
    n = len(A)
    if n <= 1:
        C = A @ B
        return C
    sz = n // 2
    r = len(U)
    Alpha = [np.zeros((sz, sz)) for _ in range(r)]
    Beta = [np.zeros((sz, sz)) for _ in range(r)]

    for i, (row_U, row_V) in enumerate(list(zip(U,V))):
        for j in range(len(row_U)):
            Alpha[i] += row_U[j] * [A[:sz, :sz], A[sz:, :sz], A[:sz, sz:], A[sz:, sz:]][j]
            Beta[i] += row_V[j] * [B[:sz, :sz], B[sz:, :sz], B[:sz, sz:], B[sz:, sz:]][j]
    M = [np.zeros((sz, sz)) for _ in range(r)]
    for i in range(r):
        M[i] = strassen(U, V, Wt, Alpha[i], Beta[i], i, l + 1)

    C = [np.zeros((sz, sz)) for _ in range(len(Wt))]
    for i, row in enumerate(Wt):
        for j in range(len(row)):
            C[i] += row[j] * M[j]
    C_new = np.asarray([[C[0], C[2]], [C[1], C[3]]])

    output = np.zeros((n, n))
    for i1, j1, i2, j2 in itertools.product(range(2), range(2), range(sz), range(sz)):
        output[i1 * sz + i2, j1 * sz + j2] = C_new[i1][j1][i2, j2]
    return output

#### ORIGINAL STRASSEN START ####
Wt = np.asarray([
    [1, 0, 0, 1, -1, 0, 1],
    [0, 1, 0, 1, 0, 0, 0],
    [0, 0, 1, 0, 1, 0, 0],
    [1, -1, 1, 0, 0, 1, 0]
])
U = np.asarray([
    [1, 0, 0, 1],
    [0, 1, 0, 1],
    [1, 0, 0, 0],
    [0, 0, 0, 1],
    [1, 0, 1, 0],
    [-1, 1, 0, 0],
    [0, 0, 1, -1]
])
V = np.asarray([
    [1, 0, 0, 1],
    [1, 0, 0, 0],
    [0, 0, 1, -1],
    [-1, 1, 0, 0],
    [0, 0, 0, 1],
    [1, 0, 1, 0],
    [0, 1, 0, 1]
])
A = np.asarray([
    [1, -1, 0, 1],
    [0, 1, 1, 0],
```

```
    [-1, 0, 1, 1],
    [-1, 1, -1, 0]
])
#### ORIGINAL STRASSEN END ####

#### BASIS CHANGE ADAPTATION START ####
U2 = np.asarray([
    [0, 0, -1, 0],
    [0, 0, 0, -1],
    [0, 1, -1, 0],
    [0, -1, 0, 0],
    [1, 0, -1, 0],
    [0, 0, 1, -1],
    [1, 0, 0, 0]
])
A2 = np.asarray([
    [0, 1, 1, 3],
    [0, -2, 1, 0],
    [-1, 1, 0, 1],
    [0, -1, 1, -1]
])
#### BASIS CHANGE ADAPTATION END ####

B = np.asarray([
    [1, 0, 1, -1],
    [0, 0, 0, 1],
    [1, 1, -1, 0],
    [-1, 1, 0, 1]
])

# C = A @ B
# C1 = strassen(U, V, Wt, A, B)
C2 = strassen(U2, V, Wt, A2, B, 0, 0)
print(f"A: \n", A)
print(f"B: \n", B)
# print(f"C: \n", C)
# print(f"C1: \n", C1)
print(f"C2: \n", C2)
```

Listing 3: Illustrative implementation of Algorithm 2 with $k = 2$, and input matrices $A, B \in \mathbb{F}_2^{n \times n}$ assumed to satisfy $n = 2^\ell$ for some $\ell \in \mathbb{N}_0$.

We recommend using and modifying the implementation to get understanding of Strassens algorithm and how the basis change described in Section 3.1.1 works.

C Rules Covered By New Postprocessing

The single postprocessing method described in Section 4.3.2 covers all 11 postprocessing rules proposed in [8], except rule R7, we show this in order of the naming convention of the rules as they are stated in [8]. All rules are in $\mathbb{F}_2$, and we will go about proving that our postprocessing method implicitly contains all 11 rules from [8], by using the DAG representation of our addition scheme, see Figure 22.

Rule R1. Suppose that part of the addition scheme looks as follows:

$$\begin{aligned} &\dots \\ &s_3 = s_1 + s_2 \\ &s_4 = s_1 + s_3 \\ &\dots \end{aligned}$$

such that $s_4 = s_1 + s_3 = s_1 + (s_1 + s_2) = s_2$. The $R1$-rule simply removes $s_4 = s_1 + s_3$ from the addition scheme and replaces all following occurrences of s_4 with s_2, including if $s_4 \in T$, then we set $s_2 \in T$ in its place. This is modeled by the postprocessing rule described in Section 4.3.2 as per Figure 36, and will likewise be defined only over $\mathbb{F}_2$.

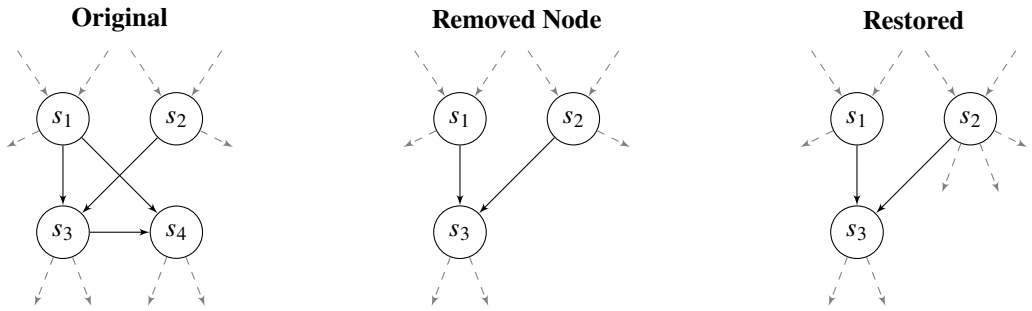


Fig. 36: DAG representation of the rule denoted $R1$ in [8]. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

The final outgoing edges of s_2, in Figure 36 are exactly those edges that were previously outgoing from s_4, and those that were previously outgoing edges of s_2 are still outgoing edges of s_2.

Rule R2. Suppose that part of the addition scheme looks as follows:

$$\begin{aligned} &\dots \\ &s_3 = s_1 + s_2 && \text{(C.1)} \\ &s_4 = s_1 + s_3 && \text{(C.2)} \\ &\dots \end{aligned}$$

moreover s_3 is only used in the addition $s_4 = s_1 + s_3$, and is not in T. The R2-rule then states that we can remove both equations (C.1) and (C.2) and replace all following occurrences of s_4 in the addition scheme with s_2, where if $s_4 \in T$ then now we denote that $s_2 \in T$.

Then running the postprocessing rule proposed in Section 4.3.2 would first apply the reduction in Figure 37. After this reduction, the node s_3 has no outgoing edges, since by assumption it was only used in the computation of s_4, and s_4 has now been removed. Furthermore, $s_3 \notin T$, so s_3 is no longer needed to complete the DAG. A second iteration of the postprocessing rule can therefore remove s_3, as shown in Figure 38.

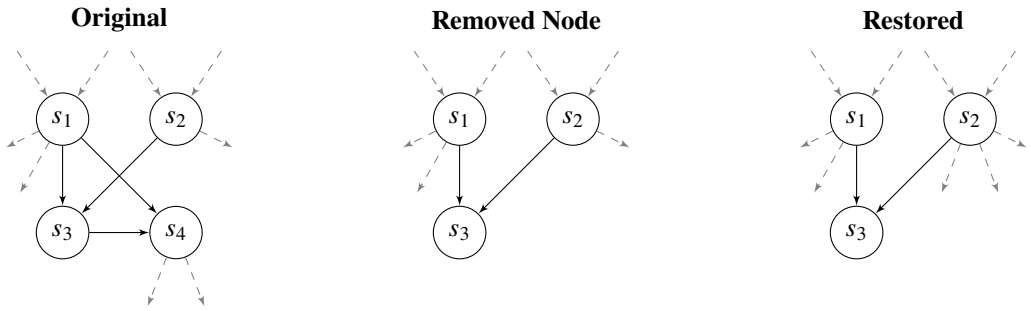


Fig. 37: DAG representation of the first iteration of the rule denoted $R2$ in [8], when applied through our postprocessing method. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

The outgoing edges of s_4 in Figure 37 are transferred to s_2 in the "restored" graph.

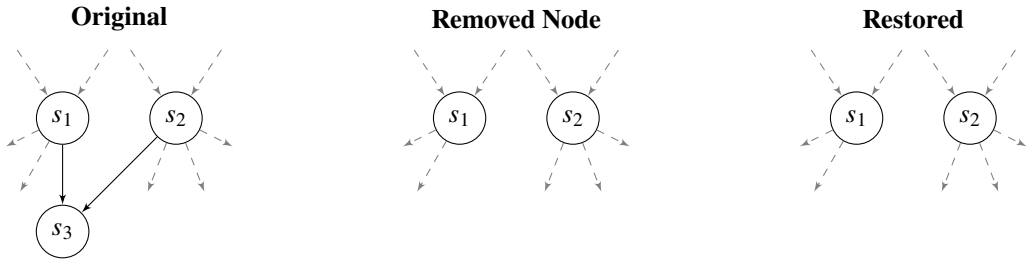


Fig. 38: DAG representation of the second iteration of the rule denoted $R2$ in [8], when applied through our postprocessing method. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

The result in Figure 38 coincides exactly with the results of applying $R2$.

Rule R3. The third rule states that if part of a given addition scheme looks like:

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 \\ &s_5 = s_4 + s_3 \\ &s_6 = s_1 + s_3 \\ &\dots \end{aligned}$$

where s_4 is only used in the addition $s_5 = s_4 + s_3$, and $s_4 \notin T$, then we can replace this part of the addition scheme with

$$\begin{aligned} &\dots \\ &s_6 = s_1 + s_3 \\ &s_5 = s_2 + s_6 \\ &\dots \end{aligned}$$

The postprocessing method proposed in Section 4.3.2 will also find this reduction, as shown in Figure 39.

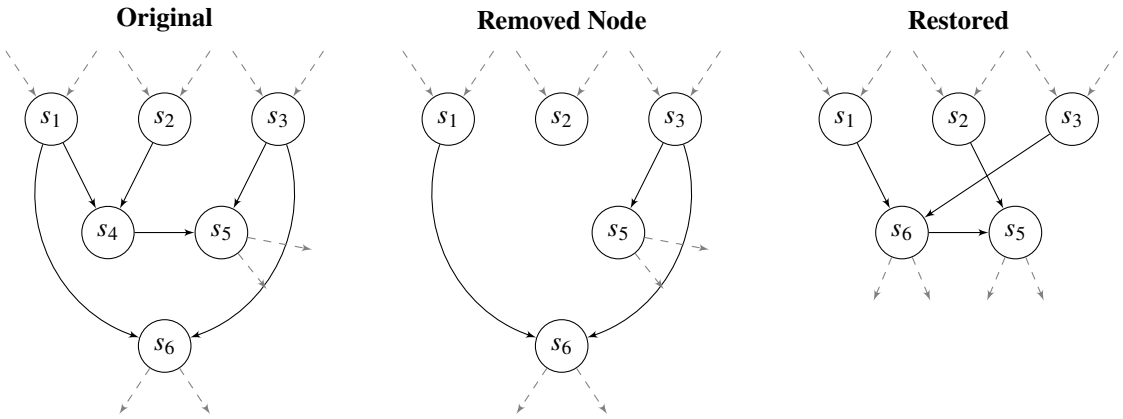


Fig. 39: DAG representation of the rule denoted $R3$ in [8]. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

The result in Figure 39 coincides exactly with the results of applying $R3$.

Rule R4. Rule R4 is obtained from a relabelling of the nodes of rule R3, see also [43]. Our proposed postprocessing method is invariant to label changes, as it considers each node independent of its label, hence Rule R4 is covered as a consequence of Rule R3 being covered by our postprocessing method.

Rule R5. The fifth rule states that if part of a given addition scheme looks like:

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 \\ &s_5 = s_1 + s_3 \\ &s_6 = s_2 + s_5 \\ &\dots \end{aligned}$$

where s_5 is only used in the addition $s_6 = s_2 + s_5$, and $s_5 \notin T$, then we can replace this part of the addition scheme with

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 \\ &s_6 = s_3 + s_4 \\ &\dots \end{aligned}$$

Our postprocessing method will also find this reduction, as shown in Figure 40.

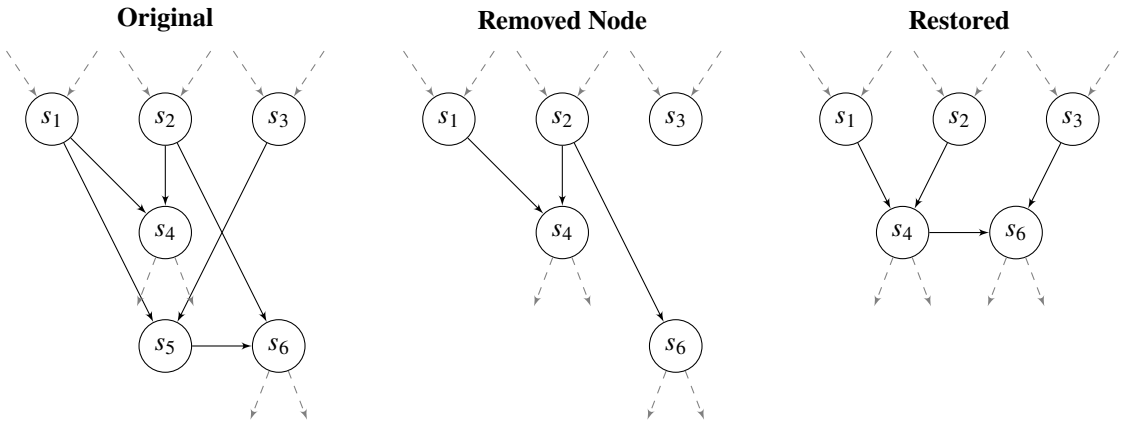


Fig. 40: DAG representation of the rule denoted $R5$ in [8]. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

As illustrated in Figure 40, the result of running our postprocessing method coincides exactly with the output of rule $R5$ on the given part of an addition scheme.

Rule R6. The sixth rule states that if part of a given addition scheme looks like:

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 \\ &s_5 = s_3 + s_4 \\ &s_6 = s_1 + s_5 \\ &\dots \end{aligned}$$

where $s_5 \notin T$ is used only in the addition $s_6 = s_1 + s_5$, then we can replace this part of the addition scheme with

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 \\ &s_6 = s_2 + s_3 \\ &\dots \end{aligned}$$

Our postprocessing method will also find this reduction, as shown in Figure 41.

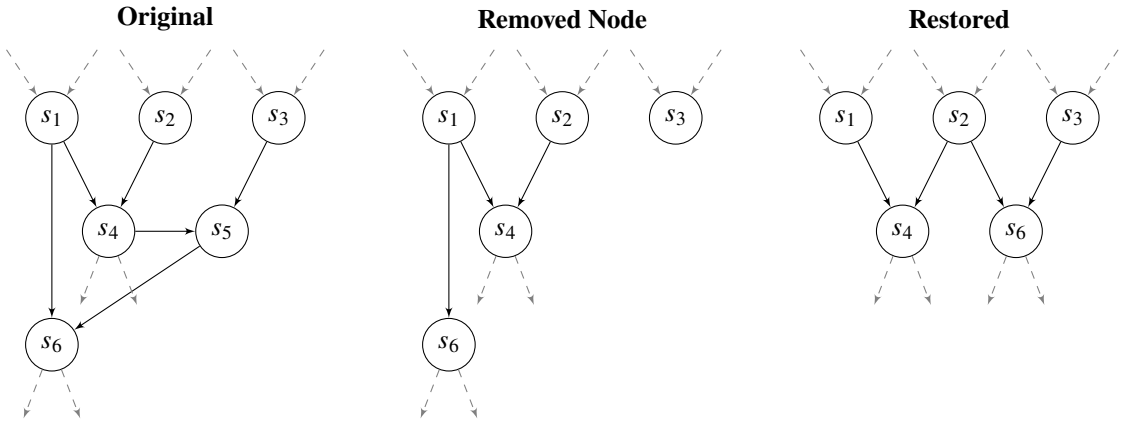


Fig. 41: DAG representation of the rule denoted $R6$ in [8]. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

As illustrated in Figure 41, the result of running our postprocessing method coincides exactly with the output of rule $R6$ on the given part of an addition scheme.

Rule R7. The seventh rule states that if part of a given addition scheme looks like:

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 \\ &s_5 = s_3 + s_4 \\ &s_6 = s_1 + s_5 \\ &\dots \end{aligned}$$

where s_4 is only used in the addition $s_5 = s_3 + s_4$, and $s_4 \notin T$, then we can replace this part of the addition scheme with

$$\begin{aligned} &\dots \\ &s_6 = s_2 + s_3 \\ &s_5 = s_1 + s_6 \\ &\dots \end{aligned}$$

note the similarity to rule R6, the only difference being that s_4 is not used outside this part of the addition scheme, while for rule R6 it was s_5 that was not used outside this part of the addition scheme.

If our postprocessing method worked for this reduction rule, then it would look something like Figure 42.

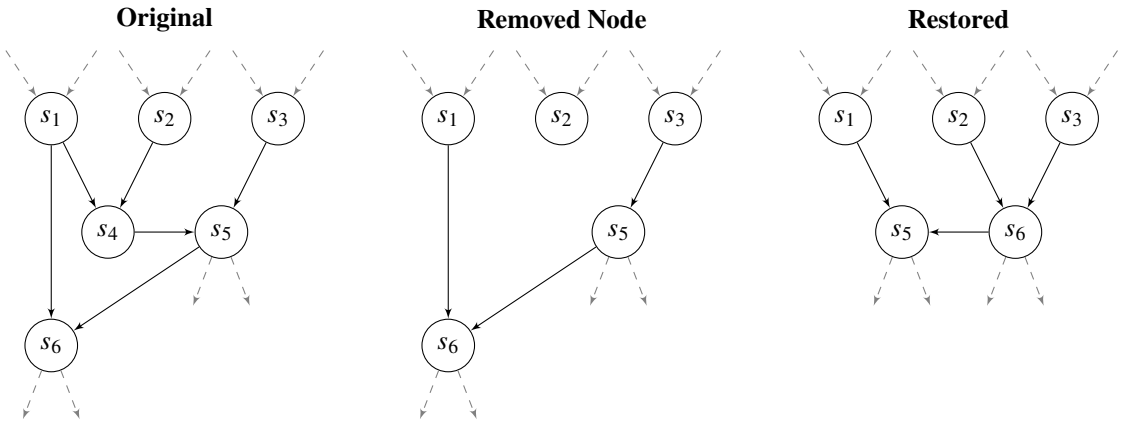


Fig. 42: Imagined DAG representation of the rule denoted $R7$ in [8]. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

Our postprocessing method would be trying to form s_5 from nodes that do not depend on s_4, in the sense that we can use s_i if there is no path from s_4 to s_i in the original graph. The problem is, as shown in Figure 42, that the Restored part forms s_5 using s_6, which depends on s_4 in the original graph, hence making it an optimization not found by our postprocessing method.

Rule R8. The eighth rule states that if part of a given addition scheme looks like:

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 && \text{(C.3)} \\ &s_5 = s_3 + s_4 && \text{(C.4)} \\ &s_6 = s_1 + s_5 && \text{(C.5)} \\ &\dots \end{aligned}$$

where $s_4 \notin T$ is only used in the addition $s_5 = s_3 + s_4$, similarly $s_5 \notin T$ is used only in the addition $s_6 = s_1 + s_5$, then we can replace this part of the addition scheme with

$$\begin{aligned} &\dots \\ &s_6 = s_2 + s_3 \\ &\dots \end{aligned}$$

Our proposed postprocessing method will find this reduction by first applying the same postprocessing step used for Rule R6, reducing equations (C.3)–(C.5) to equations (C.6) and (C.7), as given by:

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 && \text{(C.6)} \\ &s_6 = s_2 + s_3 && \text{(C.7)} \\ &\dots \end{aligned}$$

where $s_4 \notin T$ is not used elsewhere in the addition scheme and can hence simply be removed in any next iteration of our proposed postprocessing method. Thus completing the proof that our postprocessing method contains in it the reduction posed by rule R8 from [8].

Rule R9. The ninth rule states that if part of a given addition scheme looks like:

$$\begin{aligned} &\dots \\ &s_4 = s_1 + s_2 \\ &s_5 = s_1 + s_3 \\ &s_6 = s_4 + s_5 \\ &\dots \end{aligned}$$

where $s_4 \notin T$ is only used in the addition $s_6 = s_4 + s_5$, then we can replace this part of the addition scheme with

$$\begin{aligned} &\dots \\ &s_5 = s_1 + s_3 \\ &s_6 = s_2 + s_3 \\ &\dots \end{aligned}$$

Our proposed postprocessing method also finds this reduction, as shown in Figure 43.

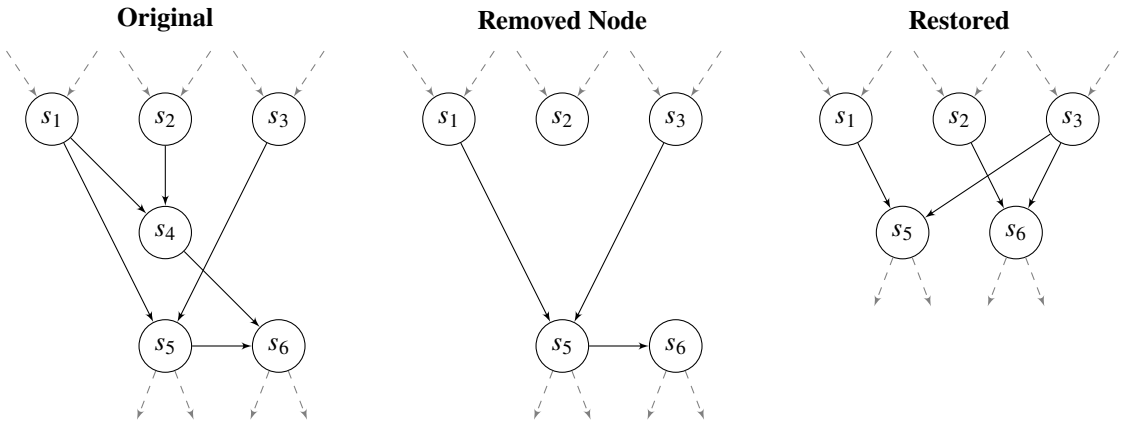


Fig. 43: DAG representation of the rule denoted $R9$ in [8]. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

Rule R10. The tenth rule states that if part of a given addition scheme looks like:

$$\begin{aligned}&\dots\\ s_4 &= s_1 + s_2\\ s_5 &= s_1 + s_3\\ s_6 &= s_4 + s_5\\ &\dots\end{aligned}$$

where $s_5 \notin T$ is only used in the addition $s_6 = s_4 + s_5$, then we can replace this part of the addition scheme with

$$\begin{aligned}&\dots\\ s_4 &= s_1 + s_2\\ s_6 &= s_2 + s_3\\ &\dots\end{aligned}$$

Our proposed postprocessing method also finds this reduction, as shown in Figure 44.

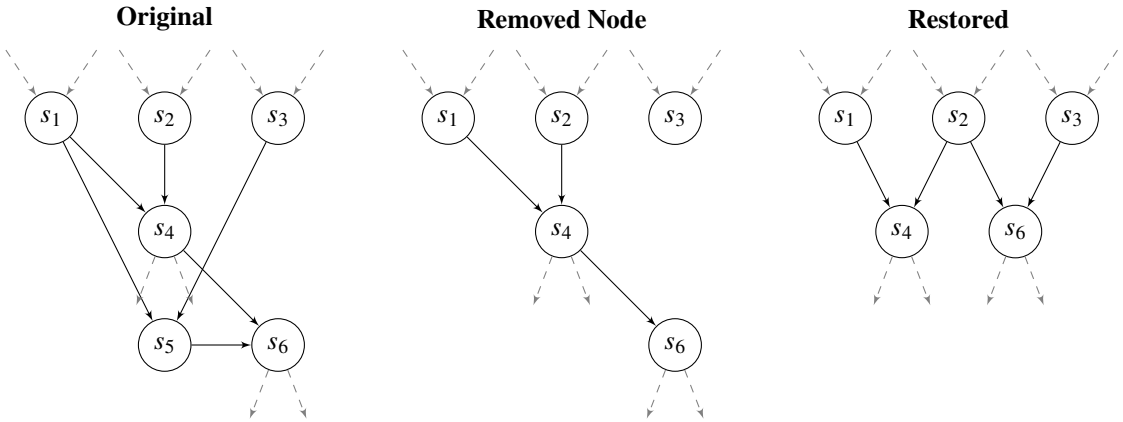


Fig. 44: DAG representation of the rule denoted $R10$ in [8]. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

Rule R11. The eleventh rule states that if part of a given addition scheme looks like:

$$\begin{aligned}&\dots\\ s_4 &= s_1 + s_2\\ s_5 &= s_1 + s_3\\ s_6 &= s_4 + s_5\\ &\dots\end{aligned}$$

where $s_4, s_5 \notin T$ are used only in the addition $s_6 = s_4 + s_5$, then we can replace this part of the addition scheme with

$$\begin{aligned}&\dots\\ s_6 &= s_2 + s_3\\ &\dots\end{aligned}$$

Our proposed postprocessing method also finds this reduction: by first applying the same postprocessing step used above for Rule R10 we end up with $s_4 \notin T$ being used nowhere. Hence a subsequent application of our postprocessing method would then remove s_4, providing the same reduction as the rule R11 from [8].

Non-Rule. The following is a reduction which is not one of the preceding 11 rules, but which our proposed postprocessing method successfully performs. Suppose part of the addition scheme is

$$\begin{aligned}
&\dots\\
&s_5 = s_1 + s_2\\
&s_6 = s_2 + s_3\\
&s_7 = s_4 + s_5\\
&s_8 = s_3 + s_4\\
&s_9 = s_6 + s_7\\
&\dots
\end{aligned}$$

where $s_6 \notin T$ and s_6 is not used outside the displayed part of the addition scheme. Moreover, assume that s_5, s_7, s_8 are either in T or used elsewhere, so that none of them may be removed by the preceding rules. Then we may replace this part by

$$\begin{aligned}
&\dots\\
&s_5 = s_1 + s_2\\
&s_7 = s_4 + s_5\\
&s_9 = s_1 + s_8\\
&\dots
\end{aligned}$$

Our proposed postprocessing method finds this reduction, as shown in Figure 45.

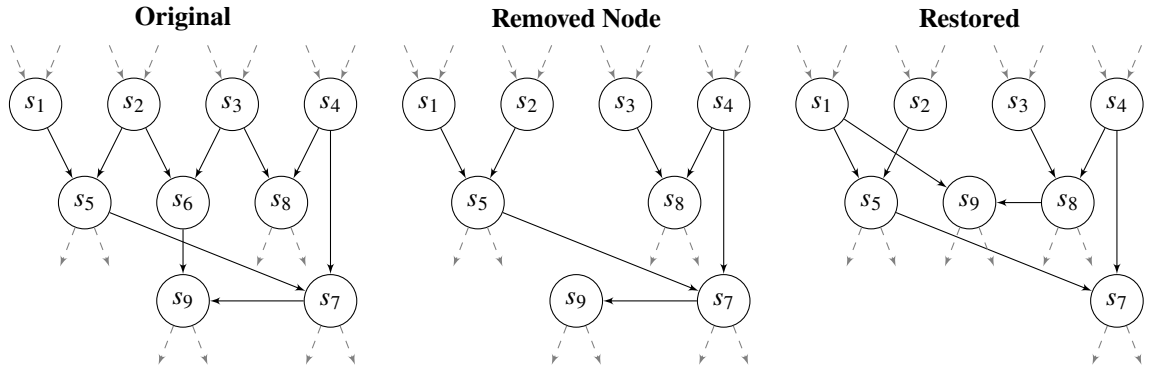


Fig. 45: DAG representation of the reduction not induced by any of the 11 preceding rules proposed in [8], but which our reduction method does successfully reduce. Note that s_6 is not used outside this part of our DAG. The dashed edges represent possible involvement from outside the explicit part of the DAG we're looking at.

D Rank-23 General 3×3 Matrix Multiplication Schemes

This appendix outlines some of the efficient rank-23 general 3×3 matrix-matrix multiplication schemes found throughout this work.

D.1 60**-addition scheme, no basis change.** This multiplication scheme is given by the 'x3+17x2+51x+4y6+2y5+3y4+14y3+z31+z29+z28/i133w207c347g-000.tab' datapoint in [4], and we introduce no basis change to find it, we only find efficient addition schemes to compute the three linear parts in equation (2.3), displayed below.

The following addition scheme computes $U\vec{A}$ in 15 additions, where u_i denotes entry i in the vector $U\vec{A}$.

$$
\begin{aligned}
s_1 &= A_{1,3} - A_{3,3}, & s_2 &= A_{3,1} - A_{3,2},\\
s_3 &= A_{1,3} + A_{3,1}, & s_4 &= A_{3,2} - A_{1,2},\\
s_5 &= A_{3,1} - A_{1,2}, & s_6 &= A_{1,1} - A_{3,1},\\
s_7 &= A_{1,1} + A_{1,3}, & s_8 &= A_{3,3} + s_2,\\
s_9 &= A_{2,1} + s_7, & s_{10} &= s_7 - A_{1,2},\\
s_{11} &= s_8 - A_{2,3}, & s_{12} &= s_9 - A_{1,2},\\
s_{13} &= A_{3,2} + s_{11}, & s_{14} &= s_{12} - s_{11},\\
s_{15} &= s_{14} - A_{2,2}, & u_1 &= s_{10},\\
u_2 &= s_{12}, & u_3 &= s_5,\\
u_4 &= s_9, & u_5 &= A_{2,1},\\
u_6 &= s_3, & u_7 &= s_6,\\
u_8 &= A_{2,1}, & u_9 &= A_{3,1},\\
u_{10} &= A_{1,2}, & u_{11} &= A_{2,2},\\
u_{12} &= s_2, & u_{13} &= s_{15},\\
u_{14} &= s_{11}, & u_{15} &= A_{2,2},\\
u_{16} &= s_4, & u_{17} &= A_{1,3},\\
u_{18} &= s_8, & u_{19} &= A_{1,3},\\
u_{20} &= A_{2,3}, & u_{21} &= s_{13},\\
u_{22} &= A_{2,3}, & u_{23} &= s_1.
\end{aligned}
$$

The following addition scheme computes $V\vec{B}$ in 16 additions, where v_i denotes entry i in the vector $V\vec{B}$.

$$
\begin{aligned}
s_1 &= B_{3,1} + B_{3,2}, & s_2 &= B_{3,2} - B_{1,2},\\
s_3 &= B_{2,2} - B_{3,1}, & s_4 &= B_{1,2} + B_{2,2},\\
s_5 &= B_{1,1} + B_{1,2}, & s_6 &= B_{3,1} - B_{1,1},\\
s_7 &= B_{1,1} - B_{2,2}, & s_8 &= B_{1,1} + B_{2,1},\\
s_9 &= B_{2,1} + B_{2,2}, & s_{10} &= B_{3,3} + s_1,\\
s_{11} &= B_{3,3} + s_2, & s_{12} &= B_{2,3} - s_8,\\
s_{13} &= s_4 + s_{11}, & s_{14} &= s_{12} - s_6,\\
s_{15} &= s_{12} - s_{11}, & s_{16} &= s_{15} + B_{1,3},\\
v_1 &= B_{1,1}, & v_2 &= s_7,\\
v_3 &= s_{12}, & v_4 &= s_4,\\
v_5 &= s_5, & v_6 &= s_{11},\\
v_7 &= B_{1,3}, & v_8 &= B_{1,3},\\
v_9 &= s_{16}, & v_{10} &= s_8,\\
v_{11} &= s_9, & v_{12} &= s_{14},\\
v_{13} &= B_{2,2}, & v_{14} &= s_3,\\
v_{15} &= B_{2,3}, & v_{16} &= B_{2,3},\\
v_{17} &= s_6, & v_{18} &= B_{3,1},
\end{aligned}
$$

$$
\begin{aligned}
v_{19} &= s_2, & v_{20} &= s_1,\\
v_{21} &= s_{13}, & v_{22} &= s_{10},\\
v_{23} &= B_{3,3}.
\end{aligned}
$$

Finally we denote $M_i = u_i \cdot v_i$, and compute the matrix product AB using 29 additions:

$$
\begin{aligned}
s_1 &= M_6 - M_{19}, & s_2 &= M_3 - M_{10},\\
s_3 &= M_{14} + M_{18}, & s_4 &= M_{20} - M_{22},\\
s_5 &= M_1 + M_{10}, & s_6 &= M_{23} - s_1,\\
s_7 &= s_2 - M_{16}, & s_8 &= s_2 - M_9,\\
s_9 &= M_{13} + s_3, & s_{10} &= M_8 - s_4,\\
s_{11} &= s_5 + M_{17}, & s_{12} &= s_3 - s_6,\\
s_{13} &= M_{18} - s_7, & s_{14} &= s_8 - s_1,\\
s_{15} &= M_1 - s_9, & s_{16} &= s_{10} + M_{15},\\
s_{17} &= M_{22} - s_{12}, & s_{18} &= s_{13} + M_{12},\\
s_{19} &= M_7 - s_{14}, & s_{20} &= M_{16} - s_{14},\\
s_{21} &= M_2 - s_{15}, & s_{22} &= s_{17} + M_{21},\\
s_{23} &= s_{20} - M_{23}, & s_{24} &= M_5 - s_{21},\\
s_{25} &= s_{21} + M_{11}, & s_{26} &= M_{20} + s_{24},\\
s_{27} &= s_9 + s_{24}, & s_{28} &= M_{19} - s_{27},\\
s_{29} &= s_{28} + M_4, & w_1 &= s_{11},\\
w_2 &= s_{25}, & w_3 &= s_{18},\\
w_4 &= s_{29}, & w_5 &= s_{26},\\
w_6 &= s_{22}, & w_7 &= s_{19},\\
w_8 &= s_{16}, & w_9 &= s_{23}.
\end{aligned}
$$

Where we then have

$$
AB = \begin{bmatrix} w_1 & w_4 & w_7 \\ w_2 & w_5 & w_8 \\ w_3 & w_6 & w_9 \end{bmatrix}.
$$

D.2 52**-addition scheme, with basis change.** This multiplication scheme is derived from the datapoint '15x2+54x+4y6+3y4+16y3+3z28/i2w205c38hj-000.tab', in [4]. We apply the following basis change methods:

$$
B_U^{-1} = \begin{bmatrix}
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0
\end{bmatrix},
\quad
B_V^{-1} = \begin{bmatrix}
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 \\
-1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0
\end{bmatrix},
\quad
B_W^{-1} = \begin{bmatrix}
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
\end{bmatrix}.
$$

First we compute the basis change induced by B_U and B_V to obtain $\tilde{A}, \tilde{B} \in \mathbb{F}^{3^\ell \times 3^\ell}$ from $A, B \in \mathbb{F}^{3^\ell \times 3^\ell}$ for some $\ell \in \mathbb{N}$, see the proof of Theorem 3.1. We compute the basis changed U', V' and W', see equation (3.1). Now we compute $U'\vec{\tilde{A}}$ in 12 additions using:

$$
\begin{aligned}
s_1 &= \tilde{A}_{1,1} - \tilde{A}_{3,2}, & s_2 &= \tilde{A}_{1,2} - \tilde{A}_{3,1},\\
s_3 &= \tilde{A}_{1,2} + \tilde{A}_{2,1}, & s_4 &= \tilde{A}_{1,3} + \tilde{A}_{2,3},\\
s_5 &= \tilde{A}_{1,3} + \tilde{A}_{3,3}, & s_6 &= \tilde{A}_{2,3} + s_1,
\end{aligned}
$$

$$
\begin{aligned}
s_7 &= s_1 - s_2, & s_8 &= \tilde{A}_{3,3} - s_3,\\
s_9 &= \tilde{A}_{3,3} + s_4, & s_{10} &= s_4 + s_2,\\
s_{11} &= s_3 + s_7, & s_{12} &= s_{10} + \tilde{A}_{2,2},\\
u_1 &= \tilde{A}_{3,3}, & u_2 &= s_3,\\
u_3 &= \tilde{A}_{1,3}, & u_4 &= s_8,\\
u_5 &= \tilde{A}_{2,1}, & u_6 &= s_5,\\
u_7 &= s_4, & u_8 &= \tilde{A}_{2,1},\\
u_9 &= s_9, & u_{10} &= \tilde{A}_{1,2},\\
u_{11} &= s_7, & u_{12} &= \tilde{A}_{3,2},\\
u_{13} &= s_{11}, & u_{14} &= \tilde{A}_{3,2},\\
u_{15} &= s_2, & u_{16} &= \tilde{A}_{1,1},\\
u_{17} &= s_1, & u_{18} &= \tilde{A}_{2,2},\\
u_{19} &= \tilde{A}_{2,2}, & u_{20} &= s_6,\\
u_{21} &= s_{12}, & u_{22} &= \tilde{A}_{3,1},\\
u_{23} &= \tilde{A}_{2,3}.
\end{aligned}
$$

Similarly, we find the following 13 addition scheme for computing $V'\vec{\tilde{B}}$:

$$
\begin{aligned}
s_1 &= \tilde{B}_{3,3} - \tilde{B}_{2,2}, & s_2 &= \tilde{B}_{1,1} - \tilde{B}_{2,2},\\
s_3 &= \tilde{B}_{3,2} - \tilde{B}_{3,1}, & s_4 &= \tilde{B}_{2,3} - \tilde{B}_{3,2},\\
s_5 &= \tilde{B}_{2,1} - s_1, & s_6 &= s_1 - \tilde{B}_{1,2},\\
s_7 &= s_2 - \tilde{B}_{1,2}, & s_8 &= \tilde{B}_{3,1} - s_2,\\
s_9 &= \tilde{B}_{3,1} + s_4, & s_{10} &= \tilde{B}_{1,3} + s_6,\\
s_{11} &= \tilde{B}_{1,3} + s_7, & s_{12} &= s_9 - \tilde{B}_{2,1},\\
s_{13} &= \tilde{B}_{2,3} - s_{10}, & v_1 &= \tilde{B}_{2,3},\\
v_2 &= s_9, & v_3 &= s_{13},\\
v_4 &= s_4, & v_5 &= s_3,\\
v_6 &= \tilde{B}_{3,2}, & v_7 &= s_{10},\\
v_8 &= \tilde{B}_{1,3}, & v_9 &= \tilde{B}_{1,3},\\
v_{10} &= s_{12}, & v_{11} &= s_5,\\
v_{12} &= s_5, & v_{13} &= \tilde{B}_{3,1},\\
v_{14} &= s_8, & v_{15} &= s_1,\\
v_{16} &= \tilde{B}_{2,2}, & v_{17} &= s_2,\\
v_{18} &= \tilde{B}_{3,3}, & v_{19} &= \tilde{B}_{1,1},\\
v_{20} &= \tilde{B}_{1,1}, & v_{21} &= s_6,\\
v_{22} &= \tilde{B}_{1,2}, & v_{23} &= s_{11}.
\end{aligned}
$$

Finally we denote $M_i = u_i \cdot v_i$, and compute the non-postprocessed matrix product AB using 27 additions:

$$
\begin{aligned}
s_1 &= M_{17} + M_{16}, & s_2 &= M_{16} + M_{15},\\
s_3 &= M_2 - M_1, & s_4 &= M_8 - M_{16},\\
s_5 &= s_1 - M_{14}, & s_6 &= s_1 - M_9,\\
s_7 &= s_2 + M_{18}, & s_8 &= M_5 + s_3,\\
s_9 &= s_4 + M_{22}, & s_{10} &= M_{20} - s_5,\\
s_{11} &= M_5 - s_5, & s_{12} &= M_{20} - s_6,\\
s_{13} &= M_7 + s_7, & s_{14} &= s_7 - M_1,
\end{aligned}
$$

$$
\begin{aligned}
s_{15} &= s_8 + M_4, & s_{16} &= s_8 + M_{18},\\
s_{17} &= s_{10} + M_6, & s_{18} &= s_{11} + M_{13},\\
s_{19} &= s_{12} - M_{23}, & s_{20} &= M_9 - s_{13},\\
s_{21} &= M_{12} - s_{14}, & s_{22} &= s_{15} + M_{19},\\
s_{23} &= s_{16} - M_{10}, & s_{24} &= s_{20} + M_{21},\\
s_{25} &= s_{20} - s_{21}, & s_{26} &= s_{21} + M_{11},\\
s_{27} &= s_{25} - M_3, & w_1 &= s_9,\\
w_2 &= s_{22}, & w_3 &= s_{18},\\
w_4 &= s_{24}, & w_5 &= s_{17},\\
w_6 &= s_{23}, & w_7 &= s_{26},\\
w_8 &= s_{27}, & w_9 &= s_{19}.
\end{aligned}
$$

Define the linear operator $T_\ell^{B_W}$ as in the proof of Theorem 3.1. We get the intermediate matrix:

$$
T_\ell^{B_W^{-1}} AB = \begin{bmatrix} w_1 & w_4 & w_7 \\ w_2 & w_5 & w_8 \\ w_3 & w_6 & w_9 \end{bmatrix}
$$

Which after postprocessing using the recursive linear transformation induced by the B_W matrix, becomes exactly the matrix product AB, see listing 4 below for verification.

```
"""
We remark for the reader, that Heule et. al. use a transposed version of the matrices
for their dataset, so even though my SLP algorithms represent elements in column major
format, parts of this algorithm is in row-major format.
"""

import sympy as sp

BU = sp.Matrix(
    [
        [0, 0, 0, 1, 0, 0, 0, 0, -1],
        [0, 1, 0, 0, 0, 0, 0, 0, 0],
        [0, 1, 0, 0, 1, 0, 0, 0, 0],
        [0, 0, 0, 1, 0, 0, 0, 0, 0],
        [1, 0, 0, 0, 0, 0, 1, 0, 0],
        [0, 0, 0, 0, 0, 0, 1, 0, 0],
        [0, 0, 1, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 1, 0],
        [0, 0, 0, 0, 0, 1, 0, 0, 0],
    ]
)

BV = sp.Matrix(
    [
        [0, 0, 0, 0, 0, 1, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 1, 0],
        [0, 0, 1, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 1, 0, 0, 0, 0, -1],
        [-1, 0, 0, 0, 0, 0, 1, 0, 0],
        [0, 0, 0, 0, -1, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 1],
        [1, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 1, 0, 0, 1, 0, 0, 0, 0],
    ]
)

BW = sp.Matrix(
    [
```

```
        [0, 0, 0, 0, 0, 0, 0, 1, 0],
        [0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, -1, 1, 0],
        [0, 0, 0, 0, 0, 1, 0, 0, 0],
        [1, 0, 0, 0, 0, 0, 0, 0, 0],
        [-1, 1, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, -1, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 1],
    ]
)

A_vec = sp.Matrix([sp.Symbol(f"A_{i, j}") for i in range(3) for j in range(3)])
B_vec = sp.Matrix([sp.Symbol(f"B_{i, j}") for i in range(3) for j in range(3)])

A_vec_prime = BU.inv() @ A_vec
B_vec_prime = BV.inv() @ B_vec


def compute_U_prime(A_vec):
    A11, A12, A13, A21, A22, A23, A31, A32, A33 = list(A_vec)

    s1 = A11 - A32
    s2 = A12 - A31
    s3 = A12 + A21
    s4 = A13 + A23
    s5 = A13 + A33
    s6 = A23 + s1
    s7 = s1 - s2
    s8 = A33 - s3
    s9 = A33 + s4
    s10 = s4 + s2
    s11 = s3 + s7
    s12 = s10 + A22

    u = {}
    u[1] = A33
    u[2] = s3
    u[3] = A13
    u[4] = s8
    u[5] = A21
    u[6] = s5
    u[7] = s4
    u[8] = A21
    u[9] = s9
    u[10] = A12
    u[11] = s7
    u[12] = A32
    u[13] = s11
    u[14] = A32
    u[15] = s2
    u[16] = A11
    u[17] = s1
    u[18] = A22
    u[19] = A22
    u[20] = s6
    u[21] = s12
    u[22] = A31
    u[23] = A23

    return u


```

```
def compute_V_prime(B_vec):
    B11, B12, B13, B21, B22, B23, B31, B32, B33 = list(B_vec)

    s1 = B33 - B22
    s2 = B11 - B22
    s3 = B32 - B31
    s4 = B23 - B32
    s5 = B21 - s1
    s6 = s1 - B12
    s7 = s2 - B12
    s8 = B31 - s2
    s9 = B31 + s4
    s10 = B13 + s6
    s11 = B13 + s7
    s12 = s9 - B21
    s13 = B23 - s10

    v = {}
    v[1] = B23
    v[2] = s9
    v[3] = s13
    v[4] = s4
    v[5] = s3
    v[6] = B32
    v[7] = s10
    v[8] = B13
    v[9] = B13
    v[10] = s12
    v[11] = s5
    v[12] = s5
    v[13] = B31
    v[14] = s8
    v[15] = s1
    v[16] = B22
    v[17] = s2
    v[18] = B33
    v[19] = B11
    v[20] = B11
    v[21] = s6
    v[22] = B12
    v[23] = s11

    return v


def compute_W_prime(M):
    s1 = M[17] + M[16]
    s2 = M[16] + M[15]
    s3 = M[2] - M[1]
    s4 = M[8] - M[16]
    s5 = s1 - M[14]
    s6 = s1 - M[9]
    s7 = s2 + M[18]
    s8 = M[5] + s3
    s9 = s4 + M[22]
    s10 = M[20] - s5
    s11 = M[5] - s5
    s12 = M[20] - s6
    s13 = M[7] + s7
    s14 = s7 - M[1]
    s15 = s8 + M[4]
    s16 = s8 + M[18]
    s17 = s10 + M[6]
```

```
    s18 = s11 + M[13]
    s19 = s12 - M[23]
    s20 = M[9] - s13
    s21 = M[12] - s14
    s22 = s15 + M[19]
    s23 = s16 - M[10]
    s24 = s20 + M[21]
    s25 = s20 - s21
    s26 = s21 + M[11]
    s27 = s25 - M[3]

    w = {}
    w[1] = s9
    w[2] = s22
    w[3] = s18
    w[4] = s24
    w[5] = s17
    w[6] = s23
    w[7] = s26
    w[8] = s27
    w[9] = s19

    return w


u = compute_U_prime(A_vec_prime)
v = compute_V_prime(B_vec_prime)

M = {i: u[i] * v[i] for i in range(1, 24)}

w_prime = compute_W_prime(M)
w_prime_vec = sp.Matrix([w_prime[i] for i in range(1, 10)])
C_vec_col_major = BW.inv() @ w_prime_vec

C = sp.expand(
    sp.Matrix(
        3,
        3,
        lambda i, j: C_vec_col_major[3 * j + i],
    )
)

print(C)


# easier to verify this way
A = sp.Matrix([[sp.Symbol(f"A_{i, j}") for j in range(3)] for i in range(3)])
B = sp.Matrix([[sp.Symbol(f"B_{i, j}") for j in range(3)] for i in range(3)])
diff = sp.simplify(C - A @ B)
print("-" * 32)
print(diff)
```

Listing 4: Symbolic verification of the 52-addition rank-23 exact 3×3 matrix multiplication algorithm described above.

E Shortest Linear Program Code

This appendix serves as an easy way for the reader to find our implementation, so that they can verify our results for themselves, or use our framework in their own work. The code can be found in https://github.com/dragonoverlord3000/shortest_linear_program.

The codebase includes implementations of every heuristic covered in Section 4, it includes the preprocessing and postprocessing methods covered in Section 4, it includes the benchmark datasets introduced in Section 3, and it includes an implementation of the lifting method described in Section 2.3.2.

We remark that the codebase is too big for it to be meaningful to include directly here.

A smaller codebase for the structured matrix multiplication pipeline is provided in https://github.com/dragonoverlord3000/AAT-SAT.